

Manual del Projecte KASBA

Sistema de Gestió i Control d'Assistència Acadèmica

Informació general

KASBA és una aplicació web per a la gestió d'assistència d'alumnes en un centre de formació/educació d'adults. Permet a professors, tutors i administradors gestionar grups, alumnes, matèries, horaris, dies no lectius i registrar/consultar l'assistència, amb informes i alertes automàtiques de baixa assistència.

Stack tecnològic

Component	Tecnologia
Frontend	React 19 + Vite
Backend	Node.js + Express 4
Base de dades	PostgreSQL (via Supabase)
Autenticació	Supabase Auth
Desplegament frontend	Vercel (previst)

Estructura de carpetes

```
KASBA/
├── KASBA-backend/
│   ├── index.js                ← Servidor Express principal (totes les
│   │                           rutes API)
│   ├── package.json
│   ├── .env                   ← Variables d'entorn (NO versionar)
│   ├── .env.example
│   ├── public/
│   │   └── index.html          ← Pàgina de prova de connexió a
│   │                           Supabase
│   └── src/
│       ├── config/
│       │   └── supabase.js     ← Client Supabase (CommonJS, no usat
│       │                           per index.js)
│       └── scripts/
│           ├── altaProfessor.js
│           ├── crearProfessor.js
│           └── testConnection.js
└── KASBA-frontend/
    ├── index.html
    ├── package.json
    ├── vite.config.js
    ├── eslint.config.js
    └── .env                    ← Variables d'entorn Vite (NO
                                versionar)
```

```

└─ src/
    ├── main.jsx          ← Punt d'entrada React
    ├── App.jsx           ← Component principal / navegació
    ├── App.css
    ├── index.css
    ├── api.js            ← Helper fetch amb autenticació
    ├── lib/
    │   └─ supabase.js    ← Client Supabase per al frontend
    ├── context/
    │   └─ AuthContext.jsx ← Gestió de sessió/usuari
    └─ components/
        ├── Login.jsx
        ├── AlumneManagement.jsx
        ├── AssistenciaManagement.jsx
        ├── ConfiguracioPrincipal.jsx
        ├── DiesNoLectiusManagement.jsx
        ├── GrupManagement.jsx
        ├── HorariGrup.jsx
        ├── HorariManagement.jsx (obsolet, sense auth)
        ├── HorariPerDia.jsx
        ├── MateriaManagement.jsx
        ├── ProfessorManagement.jsx
        └─ informes/
            ├── InformesPrincipal.jsx
            ├── InformeAssistencia.jsx
            ├── AssistenciaMateria.jsx
            ├── AssistenciaGrup.jsx
            └─ AlertesAssistencia.jsx

```

1. Model de dades (Supabase / PostgreSQL)

Segons el codi del backend (`index.js`), el model de dades inferit és el següent:

Taula professors

Vinculada a `auth.users` (mateix `id` = UUID d'Auth).

Camp	Tipus	Descripció
<code>id</code>	UUID (FK a <code>auth.users.id</code>)	Identificador, igual que l'usuari d'Auth
<code>email</code>	text	Correu electrònic
<code>nom</code>	text	Nom complet
<code>rol</code>	text	<code>admin</code> , <code>tutor</code> o <code>professor</code>

Taula grups

Camp	Tipus	Descripció
<code>id</code>	UUID	Identificador
<code>nom</code>	text	Nom del grup (ex: "1r ESO A")

curs	text	Curs (ex: "1r ESO")
professor_id	UUID (FK → professors.id)	Tutor assignat (opcional)
l·lindar_assistencia	numeric/int	% mínim d'assistència (per defecte 80)

Taula alumnes

Camp	Tipus	Descripció
id	UUID	Identificador
nom	text	Nom
cognoms	text	Cognoms
email	text	Correu (opcional)
dni	text	DNI (opcional)
grup_id	UUID (FK → grups.id)	Grup al qual pertany
actiu	boolean	Si l'alumne està actiu

Taula materies

Camp	Tipus	Descripció
id	UUID	Identificador
nom	text	Nom de la matèria
descripcio	text	Descripció opcional

Taula horaris

Defineix les franges horàries setmanals per grup.

Camp	Tipus	Descripció
id	UUID	Identificador
grup_id	UUID (FK → grups.id)	Grup
materia_id	UUID (FK → materies.id)	Matèria
dia_setmana	int (1-7)	1=Dilluns ... 7=Diumenge
hora_inici	time	Hora d'inici (format HH:MM:SS)
durada_min	int	Durada en minuts

Taula registres

Registres d'assistència, una fila per alumne+sessió+dia.

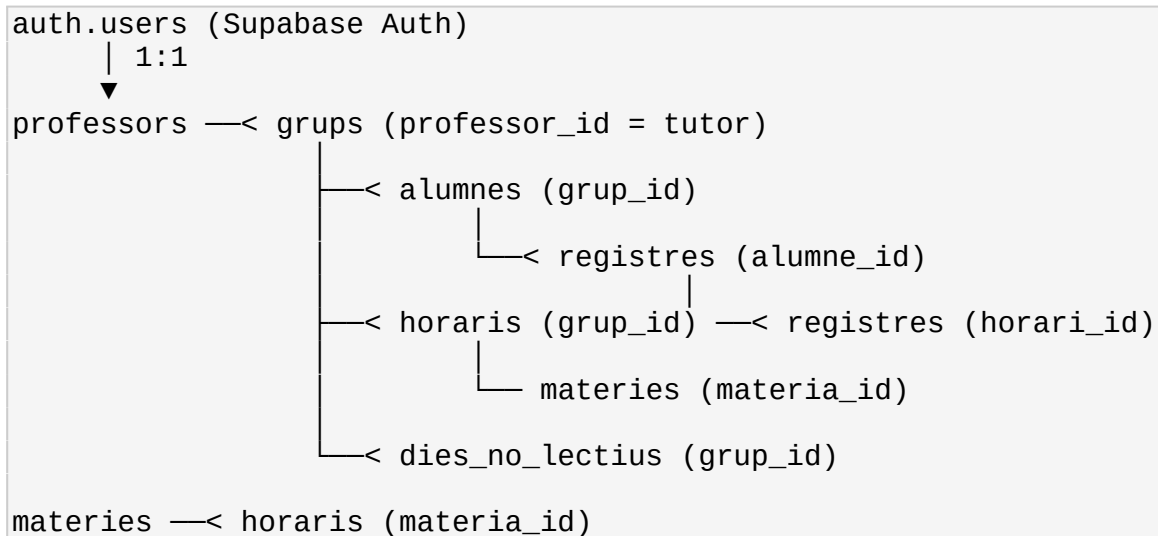
Camp	Tipus	Descripció
alumne_id	UUID (FK → alumnes.id)	Alumne
horari_id	UUID (FK → horaris.id)	Sessió/franja horària
data	date	Data de la sessió
minuts_assistits	int	Minuts assistits
minuts_justificats	int	Minuts justificats
observacions	text	Observacions opcionals

Clau d'unicitat (per a **upsert**): (**alumne_id**, **horari_id**, **data**) — definida a **ON CONFLICT** a **/assistencia/guardar**.

Taula **dies_no_lectius**

Camp	Tipus	Descripció
id	UUID	Identificador
grup_id	UUID (FK → grups.id)	Grup
data	date	Data no lectiva
motiu	text	Motiu (opcional)

Diagrama de relacions (resum)



2. Arquitectura general i autenticació

2.1. Flux d'autenticació

1. **Login** (**POST /login**): el frontend envia **email** + **password** al backend.
2. El backend crida **supabase.auth.signInWithPassword()**.
3. Si és correcte, el backend consulta la taula **professors** per obtenir **nom**, **email**, **rol**. Si l'usuari no és professor → error 403.
4. Si el **rol** és **tutor**, el backend cerca el grup on **professor_id = professor.id** per obtenir **grup_id**.
5. El backend retorna **{ token, user: { id, nom, email, rol, grup_id } }**.
6. El frontend guarda **token** i **user** a **localStorage** (**AuthContext.jsx**).
7. Totes les peticions posteriors envien **Authorization: Bearer <token>** (via **apiFetch** a **api.js**).

2.2. Middleware d'autenticació al backend (**index.js**)

- **autenticacio**: valida el token Bearer contra `supabase.auth.getUser(token)`, recupera el professor associat i adjunta `req.user = { id, email, nom, rol, grup_id? }`. Si el rol és **tutor**, també busca el `grup_id` del seu grup.
- **adminOnly**: middleware que comprova `req.user.rol === 'admin'`; si no, retorna 403.

2.3. Rols i permisos

Rol	Permisos
admin	Accés total: gestió de professors, grups, matèries, horaris, dies no lectius, alumnes, assistència i informes.
tutor	Accés al seu grup: pot gestionar alumnes del seu grup, registrar assistència, veure informes i horaris. NO pot gestionar professors, matèries, horaris ni dies no lectius.
professor	Pot registrar assistència i veure informes/horaris, però no gestionar alumnes, grups, etc.

2.4. Client Supabase al backend

El backend crea un client Supabase **amb la** `service_role` **key** (`getSupabaseAdmin()` a `index.js`), per cada petició, amb `persistSession: false`. Això **bypassa Row Level Security (RLS)**, de manera que tota la seguretat depèn del middleware `autenticacio/adminOnly` d'Express, **no** de les polítiques RLS de Supabase (encara que aquestes poden existir com a capa addicional).

⚠ **Important**: el fitxer `index.js` imprimeix per consola el rol de la clau de servei a l'arrencada:

```
console.log('ROL DE LA CLAU:',
JSON.parse(atob(process.env.SUPABASE_SERVICE_KEY.split('.')[1]))).role)
```

Això és útil per depurar però **no s'hauria de deixar en producció** (pot exposar informació sensible als logs).

2.5. CORS

Actualment configurat com:

```
app.use(cors({ origin: true }))) // Permet qualsevol origen – només per proves a la LAN
```

Recomanació: restringir-ho a l'origen real del frontend en producció (`cors({ origin: 'https://el-teu-domini.com' })`).

2.6. Variables d'entorn

Backend (`KASBA-backend/.env`):

```
NODE_ENV=...
PORT=...
SUPABASE_URL=...
SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_KEY=...
```

Frontend (**KASBA-frontend/.env**):

```
VITE_SUPABASE_URL=...
VITE_SUPABASE_ANON_KEY=...
```

*Nota: el client Supabase del frontend (**src/lib/supabase.js**) actualment **no s'utilitza** per a les operacions CRUD de l'app (totes passen pel backend via **apiFetch**). Només es fa servir (si s'arriba a usar) per a coses com auth directa client-side, si es vol implementar.*

2.7. URL de l'API

A **src/api.js**:

```
const API_URL = 'http://192.168.3.10:3000';
```

⚠ Aquesta URL està **hardcoded** amb una IP de LAN. Per a desplegament caldrà:

- Fer-la configurable via variable d'entorn Vite (**VITE_API_URL**).
- Apuntar-la al domini/IP del backend en producció.

2.8. Inici dels servidors

- **Backend**: **npm run dev** (nodemon) o **npm start** → escolta a **http://0.0.0.0:3000**.
- **Frontend**: **npm run dev** (Vite) → per defecte port **5173**.

3. Referència de l'API (Backend Express)

Totes les rutes (excepte **/login**) requereixen l'middleware **autenticacio** (capçalera **Authorization: Bearer <token>**). Les marcades amb **@Admin** també requereixen **adminOnly**.

3.1. Autenticació

Mètode	Ruta	Accés	Descripció
POST	/login	Públic	Login amb email+password . Retorna token i dades de l'usuari (id, nom, email, rol, grup_id).

3.2. Professors Admin

Mètode	Ruta	Descripció
GET	/professors	Llista tots els professors (ordenats per nom). Crea un professor: {email, nom, rol, password}. Crea l'usuari a Supabase Auth i després a la taula professors. Si falla la inserció a BD, esborra l'usuari d'Auth (rollback).
POST	/professors	Actualitza {nom, email, rol}. Si canvia l'email, també actualitza Auth.
PUT	/professors/:id	Esborra l'usuari d'Auth i el registre de la taula professors.
DELETE	/professors/:id	

3.3. Grups

Mètode	Ruta	Accés	Descripció
GET	/grups	Tots els autenticats	Llista grups amb el nom del professor tutor (*, professors(nom)). Crea grup: {nom, curs, professor_id?, llindar_assistencia?} (per defecte 80).
POST	/grups	 Admin	Actualitza grup.
PUT	/grups/:id	 Admin	Esborra grup.
DELETE	/grups/:id	 Admin	

3.4. Alumnes

Mètode	Ruta	Accés	Descripció
GET	/alumnes	Tots	Llista alumnes amb el nom del grup. Si el rol és tutor, només els del seu grup_id. Crea alumne: {nom, cognoms, email?, dni?, grup_id, actiu?}. Un tutor només pot crear alumnes al seu propi grup.
POST	/alumnes	Tots	Actualitza alumne. Tutor: només si l'alumne pertany al seu grup.
PUT	/alumnes/:id	Tots	Esborra alumne. Tutor: només si pertany al seu grup.
DELETE	/alumnes/:id	Tots	

3.5. Matèries Admin

Mètode	Ruta	Descripció
GET	/materies	Llista matèries (ordenades per nom).
POST	/materies	Crea: {nom, descripcio?}.
PUT	/materies/:id	Actualitza.
DELETE	/materies/:id	Esborra.

3.6. Horaris setmanals

Mètode	Ruta	Accés	Descripció
GET	/horaris	Tots	Llista horaris amb nom de matèria i grup (*, materies(nom), grups(nom)). Filtre opcional ?grup_id=. Ordenat per dia_setmana i hora_inici.
POST	/horaris	 Admin	Crea franja: {grup_id, materia_id, dia_setmana, hora_inici, durada_min}.
PUT	/horaris/:id	 Admin	Actualitza franja.
DELETE	/horaris/:id	 Admin	Esborra franja.

3.7. Horaris per dia (gestió ràpida)

Mètode	Ruta	Accés	Descripció
GET	/horaris/grup-dia	Tots	?grup_id=&dia_setmana=. Retorna les franges d'aquell grup i dia, amb info de matèria.
POST	/horaris/grup-dia	 Admin	{grup_id, dia_setmana, franges: [{hora_inici, durada_min, materia_id}, ...]}. Substitueix totes les franges existents per a aquell grup+dia (esborra i reinsereix). Valida que no hi hagi dues franges a la mateixa hora.

3.8. Assistència

Mètode	Ruta	Accés	Descripció
GET	/assistencia/config	Tots	?grup_id=&data=. Calcula el dia de la setmana de data, retorna {sessions, alumnes,

POST	/assistencia/guardar	Tots	registresExistents}: sessions del grup per aquell dia de la setmana, alumnes actius del grup, i registres ja existents per aquella data. {data, registres: [{alumne_id, horari_id, minuts_assistits, minuts_justificats, observacions}, ...]}. Fa upsert a registres amb onConflict: 'alumne_id, horari_id, data'.
------	----------------------	------	---

3.9. Dies no lectius Admin

Mètode	Ruta	Descripció
GET	/dies_no_lectius?grup_id=	Llista dies no lectius d'un grup, ordenats per data.
POST	/dies_no_lectius	Crea: {grup_id, data, motiu?}.
PUT	/dies_no_lectius/:id	Actualitza {data, motiu}.
DELETE	/dies_no_lectius/:id	Esborra.
POST	/dies_no_lectius/copiar	{origen_grup_id, desti_grup_id}. Copia tots els dies no lectius d'un grup a un altre (s'afegeixen, no substitueixen).

3.10. Informes (tots autenticats)

GET /informes/assistencia?grup_id=&data_inici=&data_fi=

Informe d'assistència **per alumne** dins d'un grup i període:

- Calcula minuts teòrics totals segons l'horari del grup i els dies no lectius del període.
- Per cada alumne actiu del grup: minuts assistits, justificats, % assistit i % assistit+justificat.
- Retorna {grup_id, data_inici, data_fi, minuts_teorics_totals, hores_teoric_total, alumnes: [...]}.

GET

/informes/assistencia_materia?grup_id=&materia_id=&data_inici=&data_fi=

Igual que l'anterior però **filtrat per una matèria concreta** (només compta les sessions d'horari corresponents a aquella matèria).

GET /informes/assistencia_grup?data_inici=&data_fi=&grup_id=(opcional)

Resum **agregat per grup** (no per alumne). Si no es passa grup_id, calcula per a **tots** els grups.

Retorna {data_inici, data_fi, grups: [{grup_id, grup_nom, hores_teoric,

```
hores_assistit, hores_justificat, percent_assistit, percent_assistit_justificat}, ...]]}.
```

GET

/informes/alertes?data_inici=&data_fi=&grup_id=(opcional)&llindar_personalitzat=(opcional)

Genera **alertes d'assistència baixa**:

- Per a cada grup (o un grup concret), calcula minuts teòrics del període.
- Per cada alumne actiu, calcula `% assistit`.
- Si `% assistit < llindar` (el del grup, o el `llindar_personalitzat` si es proporciona), genera una alerta.
- Retorna `{data_inici, data_fi, total_alertes, alertes: [...]}` ordenat per `percent_assistit` ascendent (els pitjors primers).

3.11. Funció auxiliar: càlcul de minuts teòrics

Tots els informes comparteixen aquesta lògica (repetida en diverses rutes — candidata a refactorització):

1. Es recorren totes les dates entre `data_inici` i `data_fi` (inclusiu).
2. Per cada dia que **no** sigui un dia no lectiu del grup, es calcula el dia de la setmana (1=dilluns...7=diumenge, amb `getDay() === 0 ? 7 : getDay()`).
3. Se sumen els `durada_min` de totes les franges horàries (`horaris`) que coincideixin amb aquell dia de la setmana.

3.12. Format d'hores

Funció `formatHoresMinuts(minuts)` (repetida en diverses rutes):

- Si `hores === 0` → "X min"
- Si `minutsRestants === 0` → "X h"
- Altrament → "X h Y min"

4. Frontend — Estructura i components

4.1. Punt d'entrada (`main.jsx`)

Munta `<AuthProvider>` que embolcalla `<App />` (que internament és `AppContent`, exportat com a `default` des de `App.jsx`).

4.2. `AuthContext.jsx`

Proporciona:

- `user`: objecte `{id, nom, email, rol, grup_id?}` o `null`.
- `token`: token JWT.
- `login(email, password)`: crida `POST /login`, desa `token` i `user` a `localStorage`.
- `logout()`: neteja `localStorage` i l'estat.
- `loading`: `true` mentre es comprova `localStorage` a l'inici.

4.3. `api.js` — Helper de peticions

`apiFetch(endpoint, options):`

- Afegeix automàticament `Content-Type: application/json`.
- Si hi ha token a `localStorage`, afegeix `Authorization: Bearer <token>`.
- Crida `${API_URL}${endpoint}`.

4.4. `App.jsx` — Navegació principal

Component `AppContent`:

- Si `loading` → missatge "Carregant..."
- Si no hi ha `user` → renderitza `<Login />`.
- Si hi ha `user`, mostra una capçalera amb el nom/rol i botó de tancar sessió, i una barra de navegació amb pestanyes:
 - o **Assistència** (tots) → `AssistenciaManagement`
 - o **Horaris** (tots) → `HorariGrup`
 - o **Informes** (tots) → `InformesPrincipal`
 - o **Alumnes** (admin i tutor) → `AlumneManagement`
 - o **Configuració** (només admin) → `ConfiguracioPrincipal`

4.5. `ConfiguracioPrincipal.jsx`

Component amb pestanyes (només accessible per `admin`):

- Professors → `ProfessorManagement`
- Grups → `GrupManagement`
- 📖 Matèries → `MateriaManagement`
- Horari (Disseny) → `HorariPerDia`
- Dies no lectius → `DiesNoLectiusManagement`

4.6. `InformesPrincipal.jsx`

Component amb pestanyes per als 4 tipus d'informes:

- 📈 Assistència per alumne → `InformeAssistencia`
- 📖 Assistència per matèria → `AssistenciaMateria`
- Assistència per grup → `AssistenciaGrup`
- ⚠️ Alertes → `AlertesAssistencia`

4.7. Resum funcional de cada component

`Login.jsx`

Formulari email/contrasenya. Crida `login()` del context i recarrega la pàgina (`window.location.reload()`) per aplicar els canvis d'estat global.

`ProfessorManagement.jsx (admin)`

- CRUD complet de professors (`/professors`).
- Formulari amb `email`, `nom`, `rol` (`professor/tutor/admin`), `password`.
- **Importació CSV**: llegeix un fitxer amb columnes `email`, `nom`, `rol`; crea professors amb contrasenya temporal `temp123456`.

- **Exportació CSV:** descarrega `email, nom, rol` de tots els professors.

GrupManagement.jsx (admin)

- CRUD de grups (`/grups`).
- Formulari: `nom, curs, professor_id (tutor), lllindar_assistencia`.
- Taula amb codi de colors per al lllindar (verd $\geq 80\%$, taronja 50-79%, vermell $< 50\%$).

MateriaManagement.jsx (admin)

- CRUD simple de matèries (`/materies`): `nom, descripcio`.

AlumneManagement.jsx (admin/tutor)

- CRUD d'alumnes (`/alumnes`).
- Formulari: `nom, cognoms, email, grup_id, actiu`.
- **Importació CSV:** columnes `nom, cognoms, email` (obligatori `nom` i `cognoms`); els alumnes importats no tenen grup assignat (`grup_id: null`).
- **Exportació CSV:** `nom, cognoms, email`.
- Taula amb indicador visual d'actiu/inactiu.

HorariPerDia.jsx (admin) — Disseny d'horaris

- Selecció de grup + dia de la setmana → carrega les franges existents (`GET /horaris/grup-dia`).
- Permet afegir/eliminar/editar franges (hora d'inici, durada, matèria) dinàmicament.
- En desar (`POST /horaris/grup-dia`), **substitueix totes les franges** d'aquell grup+dia.
- Valida que cada franja tingui matèria, hora i durada vàlides abans de desar.

HorariGrup.jsx (tots) — Visor d'horari

- Selecció de grup → carrega tot l'horari (`GET /horaris?grup_id=`).
- Mostra l'horari agrupat per dia de la setmana, en taules amb hora, durada i matèria.

HorariManagement.jsx ⚠ Component obsolet / sense ús actual

- Fa servir `fetch` directe a `http://localhost:3000` (sense `apiFetch`, sense token).
- No apareix referenciat des de `App.jsx` ni `ConfiguracioPrincipal.jsx`.
- **Recomanació:** eliminar-lo o migrar-lo a `apiFetch` si es vol recuperar.

DiesNoLectiusManagement.jsx (admin)

- Selecció de grup → CRUD de dies no lectius (`/dies_no_lectius`): `data, motiu`.
- **Copiar dies:** selecciona un grup origen i copia tots els seus dies no lectius al grup actual (`POST /dies_no_lectius/copiar`).

AssistenciaManagement.jsx (tots) — Registre d'assistència

Flux:

1. Selecciona grup + data → calcula dia de la setmana → carrega les sessions (**horaris**) d'aquell grup per aquell dia (**GET /horaris?grup_id=**, filtrat client-side per **dia_setmana**).
2. Si només hi ha una sessió, se selecciona automàticament; si n'hi ha vàries, l'usuari n'escull una.
3. Carrega alumnes actius del grup i els registres existents (**GET /assistencia/config**).
4. Per cada alumne, mostra controls ràpids: botons **0 / ¼ / ½ / ✓** (percentatge de la durada) + input numèric per a "minuts assistits" i "minuts justificats", més un camp d'observacions.
5. Validació: **minuts + justificats** no pot superar **durada_min** de la sessió.
6. **Guardar** (**POST /assistencia/guardar**) envia tots els registres de cop.

Informes (**components/informes/**)

- **InformeAssistencia.jsx**: per alumne, dins un grup i període. Mostra taula amb hores teòriques/assistides/justificades i percentatges, codi de color (vermell si <80%).
- **AssistenciaMateria.jsx**: igual, però filtrat per matèria dins el grup.
- **AssistenciaGrup.jsx**: resum agregat per grup (o tots els grups si no se'n selecciona cap).
- **AlertesAssistencia.jsx**: llista d'alumnes per sota del llindar d'assistència (del grup o personalitzat), amb codi de color segons gravetat (<50% vermell fort, <70% vermell suau).

Totes les pestanyes d'informes inicialitzen per defecte el període com **els últims 30 dies** (**data_fi = avui**, **data_inici = avui - 30 dies**).

5. Avisos de seguretat i recomanacions ⚠

Aquesta secció recull problemes detectats durant la revisió del codi font que **caldrà abordar abans de posar el projecte en producció**.

5.1. CRÍTIC: clau **service_role** exposada en codi font

El fitxer **KASBA-backend/src/scripts/crearProfessor.js** conté **hardcoded** la URL de Supabase i una clau **service_role** (JWT complet). Aquesta clau dona accés total a la base de dades, bypassant tota la RLS.

Accions recomanades immediates:

1. **Rotar (regenerar) la clau **service_role**** des del tauler de Supabase (Project Settings → API).
2. Eliminar el valor hardcoded del fitxer i substituir-lo per **process.env.SUPABASE_SERVICE_KEY** (com ja fa **index.js** i **altaProfessor.js**).
3. Assegurar-se que aquest fitxer (i qualsevol altre amb secrets) **no s'ha pujat mai a un repositori remot**; si s'ha pujat, cal reescriure l'històric de Git o, com a mínim, rotar totes les claus implicades.

4. Revisar `.gitignore` per assegurar que `.env` està exclòs (✓ ja ho està en aquest projecte).

5.2. ● Log de la clau de servei a la consola

A `index.js`:

```
console.log('ROL DE LA CLAU:',  
JSON.parse(atob(process.env.SUPABASE_SERVICE_KEY.split('.')[1])).role)
```

Encara que només imprimeix el `role` (no la clau completa), és una crida de depuració que s'hauria d'eliminar o protegir amb `if (process.env.NODE_ENV !== 'production')`.

5.3. ● CORS obert a qualsevol origen

```
app.use(cors({ origin: true })))
```

Vàlid per a proves a la LAN, però en producció s'hauria de restringir a l'origen real del frontend desplegat.

5.4. ● URL de l'API hardcoded al frontend

`src/api.js`:

```
const API_URL = 'http://192.168.3.10:3000';
```

Recomanat: moure-ho a una variable d'entorn Vite, p. ex. `VITE_API_URL`, amb un valor per defecte per a desenvolupament local.

5.5. ● Component `HorariManagement.jsx` sense autenticació

Fa peticions directes a `http://localhost:3000` sense token. No està enllaçat des de cap menú, però si algú l'importa accidentalment fallarà (token absent → 401) o, si les rutes corresponents no requereixin auth, seria una bretxa. **Recomanació**: eliminar el fitxer.

5.6. ● Contrasenyes temporals previsible

La importació CSV de professors assigna la contrasenya `temp123456` a tots els usuaris importats. **Recomanació**: generar contrasenyes aleatòries per usuari i/o forçar un canvi de contrasenya al primer login (Supabase Auth ho permet amb flux de "magic link" o "recovery").

5.7. ● Bones pràctiques ja aplicades

- Ús de `Authorization: Bearer <token>` per a totes les rutes protegides.
- Validació de rols (`admin/tutor/professor`) i d'abast (tutor limitat al seu grup) tant a `GET` com `POST/PUT/DELETE`.
- Rollback de la creació d'usuari a Auth si falla la inserció a `professors` (i viceversa en eliminació).
- `.env` correctament exclòs del control de versions.
- Ús de `express@4.18.2` (evitant els problemes coneguts amb `express@5`).

6. Instal·lació i posada en marxa (desenvolupament local)

6.1. Requisits previs

- Node.js 20.x
- npm
- Un projecte Supabase (URL + claus `anon` i `service_role`)
- Taules de BD creades a Supabase: `professors`, `grups`, `alumnes`, `materies`, `horaris`, `registres`, `dies_no_lectius`, amb les relacions descrites a la secció 1.

6.2. Backend

```
cd KASBA-backend
npm install

# Crear fitxer .env basat en .env.example:
cat > .env << 'EOF'
NODE_ENV=development
PORT=3000
SUPABASE_URL=https://<el-teu-projecte>.supabase.co
SUPABASE_ANON_KEY=<clau anon>
SUPABASE_SERVICE_KEY=<clau service_role – MAI exposar al client>
EOF

# Arrencar en mode desenvolupament (amb nodemon, recàrrega automàtica)
npm run dev

# 0 en mode producció
npm start
```

El servidor escoltarà a `http://0.0.0.0:3000` (accessible des de qualsevol IP de la xarxa, no només `localhost`).

6.3. Frontend

```
cd KASBA-frontend
npm install

# Crear fitxer .env:
cat > .env << 'EOF'
VITE_SUPABASE_URL=https://<el-teu-projecte>.supabase.co
VITE_SUPABASE_ANON_KEY=<clau anon>
EOF

# IMPORTANT: editar src/api.js i canviar API_URL per la IP/URL del
backend,
# o (recomanat) refactoritzar-ho a una variable d'entorn VITE_API_URL
```

```
npm run dev
```

Vite arrencarà per defecte a <http://localhost:5173>.

6.4. Crear el primer usuari administrador

Com que `POST /professors` requereix ja estar autenticat com a `admin` (problema de l'ou i la gallina), el primer usuari s'ha de crear manualment, per exemple amb el script `altaProfessor.js`:

```
cd KASBA-backend
node -e "
import('./src/scripts/altaProfessor.js').then(({ altaProfessor }) =>
  altaProfessor({
    email: 'admin@exemple.cat',
    nom: 'Administrador',
    rol: 'admin',
    password: 'Canvia-Aquesta-Contrasenya!'
  }).then(console.log).catch(console.error)
)
"
```

*Nota: `altaProfessor.js` usa sintaxi ESM (`import/export`), igual que `index.js` (el `package.json` del backend té `"type": "module"`). En canvi, `crearProfessor.js` i `testConnection.js` i `src/config/supabase.js` usen `require/module.exports` (CommonJS), el qual **no és compatible directament** amb `"type": "module"` sense canviar l'extensió a `.cjs` o adaptar la sintaxi. Caldria unificar l'estil (recomanat: tot a ESM, com `index.js` i `altaProfessor.js`).*

6.5. Compilació per a producció (frontend)

```
cd KASBA-frontend
npm run build      # genera KASBA-frontend/dist/
npm run preview    # previsualitza el build localment
```

El directori `dist/` és el que es desplegaria a Vercel (o un altre hosting estàtic).

7. Codi font complet

A continuació es presenta el codi font complet de tots els fitxers rellevants del projecte (s'exclouen `node_modules`, `dist` i fitxers `.bak`).

⚠ **Recordatori de seguretat:** el fitxer `KASBA-backend/src/scripts/crearProfessor.js` conté una clau `service_role` de Supabase hardcoded al codi. S'ha mantingut tal

*qual en aquest manual per a referència, però **s'hauria de rotar aquesta clau i eliminar-la del codi** (veure secció 5.1).*

7.1. Configuració d'arrel del projecte

KASBA/.gitignore

```
node_modules/  
.env  
.env.local  
dist/  
build/  
.DS_Store  
*.log
```

7.2. Backend (KASBA-backend)

7.2.1. Configuració

KASBA-backend/package.json

```
{  
  "name": "kasba-backend",  
  "version": "1.0.0",  
  "type": "module",  
  "main": "index.js",  
  "scripts": {  
    "dev": "nodemon index.js",  
    "start": "node index.js"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "description": "",  
  "dependencies": {  
    "@supabase/supabase-js": "^2.107.0",  
    "cors": "^2.8.6",  
    "dotenv": "^17.4.2",  
    "express": "^4.18.2",  
    "ws": "^8.21.0"  
  },  
  "devDependencies": {  
    "nodemon": "^3.1.14"  
  }  
}
```

KASBA-backend/.gitignore

```
.env
.env.*
!.env.example
```

KASBA-backend/.env.example

```
NODE_ENV=
PORT=
SUPABASE_URL=
SUPABASE_ANON_KEY=
# SUPABASE_SERVICE_KEY= (opcional)
```

7.2.2. Servidor principal

KASBA-backend/index.js

```
import express from 'express'
import cors from 'cors'
import ws from 'ws'
import { createClient } from '@supabase/supabase-js'
import dotenv from 'dotenv'

dotenv.config()
console.log('ROL DE LA CLAU:',
JSON.parse(atob(process.env.SUPABASE_SERVICE_KEY.split('.')[1])).role)

// Funció per crear un client Supabase fresc per a cada petició
function getSupabaseAdmin() {
  return createClient(
    process.env.SUPABASE_URL,
    process.env.SUPABASE_SERVICE_KEY,
    {
      realtime: { transport: ws },
      auth: {
        persistSession: false // per assegurar que no es guarda
estat de sessió
      }
    }
  )
}

const app = express()

//app.use(cors({ origin: 'http://localhost:5173' }))

//Només per proves a la LAN
app.use(cors({ origin: true })) // Permet qualsevol origen

app.use(express.json())
```

```
// =====
// MIDDLEWARE D'AUTENTICACIÓ
// =====

async function autenticacio(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No autoritzat: falta token' });
  }

  const token = authHeader.split(' ')[1];
  const supabase = getSupabaseAdmin();
  const { data: { user }, error } = await
supabase.auth.getUser(token);

  if (error || !user) {
    return res.status(401).json({ error: 'Token invàlid o expirat' });
  }

  // Obtenir el rol i el grup (si és tutor) de l'usuari
  const { data: professor, error: errProf } = await supabase
    .from('professors')
    .select('id, nom, email, rol')
    .eq('id', user.id)
    .single();

  if (errProf || !professor) {
    return res.status(403).json({ error: 'Usuari no autoritzat com a professor' });
  }

  req.user = {
    id: user.id,
    email: user.email,
    nom: professor.nom,
    rol: professor.rol
  };

  // Si és tutor, obtenir el seu grup_id
  if (professor.rol === 'tutor') {
    const { data: grup, error: errGrup } = await supabase
      .from('grups')
      .select('id')
      .eq('professor_id', user.id)
      .single();
    if (!errGrup && grup) {
      req.user.grup_id = grup.id;
    }
  }
}
```

```

    next();
}

function adminOnly(req, res, next) {
  if (req.user.rol !== 'admin') {
    return res.status(403).json({ error: 'Accés denegat: necessites permisos d\'administrador' });
  }
  next();
}

// =====
// AUTENTICACIÓ: LOGIN (públic)
// =====

app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ error: 'Falten email o contrasenya' });
  }

  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase.auth.signInWithPassword({
    email,
    password
  });

  if (error) {
    return res.status(401).json({ error: 'Credencials incorrectes' });
  }

  // Obtener rol i grup del professor
  const { data: professor, error: errProf } = await supabase
    .from('professors')
    .select('id, nom, email, rol')
    .eq('id', data.user.id)
    .single();

  if (errProf || !professor) {
    return res.status(403).json({ error: 'Usuari no autoritzat com a professor' });
  }

  let grup_id = null;
  if (professor.rol === 'tutor') {
    const { data: grup } = await supabase
      .from('grups')
      .select('id')
      .eq('professor_id', professor.id)
      .single();
    if (grup) grup_id = grup.id;
  }

```

```

    }

    res.json({
      token: data.session.access_token,
      user: {
        id: professor.id,
        nom: professor.nom,
        email: professor.email,
        rol: professor.rol,
        grup_id
      }
    });
  });
});

// =====
// RUTES PER A PROFESSORS (només admin)
// =====

app.get('/professors', autenticacio, adminOnly, async (req, res) => {
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('professors').select('*').order('nom')
  if (error) return res.status(500).json({ error: error.message })
  res.json(data)
})

app.post('/professors', autenticacio, adminOnly, async (req, res) => {
  const { email, nom, rol, password } = req.body
  if (!email || !nom || !rol || !password)
    return res.status(400).json({ error: 'Falten camps obligatoris' })

  const supabase = getSupabaseAdmin();
  const { data: authData, error: authError } =
    await supabase.auth.admin.createUser({
      email, password, email_confirm: true
    })
  if (authError) return res.status(400).json({ error:
authError.message })

  const userId = authData.user.id

  const { error: dbError } = await supabase
    .from('professors')
    .insert([{ id: userId, email, nom, rol }])

  if (dbError) {
    await supabase.auth.admin.deleteUser(userId)
    return res.status(500).json({ error: dbError.message })
  }

  res.json({ id: userId, email, nom, rol })
})

```

```

app.put('/professors/:id', autenticacio, adminOnly, async (req, res)
=> {
  const { id } = req.params;
  const { nom, email, rol } = req.body;

  if (!nom && !email && !rol) {
    return res.status(400).json({ error: 'Cal proporcionar algun camp
per actualitzar' });
  }

  const updates = {};
  if (nom) updates.nom = nom;
  if (email) updates.email = email;
  if (rol) updates.rol = rol;

  const supabase = getSupabaseAdmin();

  // 1. Actualitzar taula professors
  const { error: dbError } = await supabase
    .from('professors')
    .update(updates)
    .eq('id', id);

  if (dbError) {
    return res.status(500).json({ error: dbError.message });
  }

  // 2. Si s'ha canviat l'email, actualitzar també a Auth (opcional,
no crític)
  if (email) {
    const { error: authError } = await
supabase.auth.admin.updateUserById(id, { email });
    if (authError) console.error('Error actualitzant email a Auth:',
authError);
    // No retornem error per evitar bloquejar l'operació principal
  }

  res.json({ ok: true, message: 'Professor actualitzat
correctament' });
});

app.delete('/professors/:id', autenticacio, adminOnly, async (req,
res) => {
  const { id } = req.params
  const supabase = getSupabaseAdmin();
  await supabase.auth.admin.deleteUser(id)
  const { error } = await supabase
    .from('professors').delete().eq('id', id)
  if (error) return res.status(500).json({ error: error.message })
  res.json({ ok: true })
})

```

```

// =====
// RUTES PER A GRUPS
// =====

app.get('/grups', autenticacio, async (req, res) => {
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('grups')
    .select('*', professors(nom)')
    .order('nom');
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.post('/grups', autenticacio, adminOnly, async (req, res) => {
  const { nom, curs, professor_id, llindar_assistencia } = req.body;
  if (!nom || !curs) {
    return res.status(400).json({ error: 'El nom i el curs són obligatoris' });
  }
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('grups')
    .insert([
      {
        nom,
        curs,
        professor_id: professor_id || null,
        llindar_assistencia: llindar_assistencia || 80
      }
    ])
    .select()
    .single();
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.put('/grups/:id', autenticacio, adminOnly, async (req, res) => {
  const { id } = req.params;
  const { nom, curs, professor_id, llindar_assistencia } = req.body;
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('grups')
    .update({ nom, curs, professor_id, llindar_assistencia })
    .eq('id', id)
    .select()
    .single();
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.delete('/grups/:id', autenticacio, adminOnly, async (req, res) => {
  {

```

```

const { id } = req.params;
const supabase = getSupabaseAdmin();
const { error } = await supabase
  .from('grups')
  .delete()
  .eq('id', id);
if (error) return res.status(500).json({ error: error.message });
res.json({ ok: true });
});

// =====
// RUTES PER A ALUMNES (tots autenticats, tutors només el seu grup)
// =====

app.get('/alumnes', autenticacio, async (req, res) => {
  const supabase = getSupabaseAdmin();
  let query = supabase.from('alumnes').select('*', groups (nom));
  if (req.user.rol === 'tutor' && req.user.grup_id) {
    query = query.eq('grup_id', req.user.grup_id);
  }
  const { data, error } = await query.order('nom');
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.post('/alumnes', autenticacio, async (req, res) => {
  const { nom, cognoms, email, dni, grup_id, actiu } = req.body;
  if (!nom || !cognoms) {
    return res.status(400).json({ error: 'Nom i cognoms són obligatoris' });
  }
  // Tutor només pot crear alumnes en el seu grup
  if (req.user.rol === 'tutor') {
    if (!req.user.grup_id || grup_id !== req.user.grup_id) {
      return res.status(403).json({ error: 'No pots crear alumnes en un altre grup' });
    }
  }
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('alumnes')
    .insert([
      { nom, cognoms, email, dni, grup_id, actiu: actiu ?? true }
    ])
    .select()
    .single();
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.put('/alumnes/:id', autenticacio, async (req, res) => {
  const { id } = req.params;
  const { nom, cognoms, email, dni, grup_id, actiu } = req.body;

```



```

// Per a tutors, comprovar que l'alumne pertany al seu grup
if (req.user.rol === 'tutor') {
  const supabase = getSupabaseAdmin();
  const { data: alumne, error: errAl } = await supabase
    .from('alumnes')
    .select('grup_id')
    .eq('id', id)
    .single();
  if (errAl || !alumne || alumne.grup_id !== req.user.grup_id) {
    return res.status(403).json({ error: 'No pots modificar alumnes
d\'un altre grup' });
  }
}

const supabase = getSupabaseAdmin();
const { data, error } = await supabase
  .from('alumnes')
  .update({ nom, cognoms, email, dni, grup_id, actiu })
  .eq('id', id)
  .select()
  .single();
if (error) return res.status(500).json({ error: error.message });
res.json(data);
});

app.delete('/alumnes/:id', autenticacio, async (req, res) => {
  const { id } = req.params;
  // Per a tutors, comprovar que l'alumne pertany al seu grup
  if (req.user.rol === 'tutor') {
    const supabase = getSupabaseAdmin();
    const { data: alumne, error: errAl } = await supabase
      .from('alumnes')
      .select('grup_id')
      .eq('id', id)
      .single();
    if (errAl || !alumne || alumne.grup_id !== req.user.grup_id) {
      return res.status(403).json({ error: 'No pots eliminar alumnes
d\'un altre grup' });
    }
  }
  const supabase = getSupabaseAdmin();
  const { error } = await supabase
    .from('alumnes')
    .delete()
    .eq('id', id);
  if (error) return res.status(500).json({ error: error.message });
  res.json({ ok: true });
});

// =====
// RUTES PER A MATERIES (només admin)

```

```
// =====

app.get('/materies', autenticacio, adminOnly, async (req, res) => {
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('materies')
    .select('*')
    .order('nom');
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.post('/materies', autenticacio, adminOnly, async (req, res) => {
  const { nom, descripcio } = req.body;
  if (!nom) return res.status(400).json({ error: 'El nom és obligatori' });
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('materies')
    .insert([ { nom, descripcio } ])
    .select()
    .single();
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.put('/materies/:id', autenticacio, adminOnly, async (req, res) => {
  const { id } = req.params;
  const { nom, descripcio } = req.body;
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('materies')
    .update({ nom, descripcio })
    .eq('id', id)
    .select()
    .single();
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.delete('/materies/:id', autenticacio, adminOnly, async (req, res) => {
  const { id } = req.params;
  const supabase = getSupabaseAdmin();
  const { error } = await supabase
    .from('materies')
    .delete()
    .eq('id', id);
  if (error) return res.status(500).json({ error: error.message });
  res.json({ ok: true });
});
```

```
// =====
// RUTES PER A HORARIS (consulta: tots autenticats, modificació: només
admin)
// =====

app.get('/horaris', autenticacio, async (req, res) => {
  const supabase = getSupabaseAdmin();
  let query = supabase.from('horaris').select('*', materies(nom),
grups(nom));
  if (req.query.grup_id) {
    query = query.eq('grup_id', req.query.grup_id);
  }
  const { data, error } = await
query.order('dia_setmana').order('hora_inici');
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.post('/horaris', autenticacio, adminOnly, async (req, res) => {
  const { grup_id, materia_id, dia_setmana, hora_inici, durada_min } =
req.body;
  if (!grup_id || !materia_id || !dia_setmana || !hora_inici
|| !durada_min) {
    return res.status(400).json({ error: 'Falten camps
obligatoris' });
  }
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('horaris')
    .insert([ { grup_id, materia_id, dia_setmana, hora_inici,
durada_min } ])
    .select()
    .single();
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.put('/horaris/:id', autenticacio, adminOnly, async (req, res) => {
  const { id } = req.params;
  const { grup_id, materia_id, dia_setmana, hora_inici, durada_min } =
req.body;
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('horaris')
    .update({ grup_id, materia_id, dia_setmana, hora_inici,
durada_min })
    .eq('id', id)
    .select()
    .single();
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});
```

```

});

app.delete('/horaris/:id', autenticacio, adminOnly, async (req, res)
=> {
  const { id } = req.params;
  const supabase = getSupabaseAdmin();
  const { error } = await supabase
    .from('horaris')
    .delete()
    .eq('id', id);
  if (error) return res.status(500).json({ error: error.message });
  res.json({ ok: true });
});

// =====
// RUTES PER A HORARIS PER DIA (gestió ràpida, només admin)
// =====

app.get('/horaris/grup-dia', autenticacio, async (req, res) => {
  const { grup_id, dia_setmana } = req.query;
  if (!grup_id || !dia_setmana) {
    return res.status(400).json({ error: 'Falten grup_id o
dia_setmana' });
  }
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('horaris')
    .select('*', materies(nom, id))
    .eq('grup_id', grup_id)
    .eq('dia_setmana', parseInt(dia_setmana))
    .order('hora_inici');
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.post('/horaris/grup-dia', autenticacio, adminOnly, async (req,
res) => {
  const { grup_id, dia_setmana, franges } = req.body;
  if (!grup_id || !dia_setmana || !Array.isArray(franges)) {
    return res.status(400).json({ error: 'Dades invàlides' });
  }

  const hores = franges.map(f => f.hora_inici);
  if (new Set(hores).size !== hores.length) {
    return res.status(400).json({ error: 'No pots tenir dues franges a
la mateixa hora per al mateix dia.' });
  }

  const supabase = getSupabaseAdmin();
  const { error: deleteError } = await supabase
    .from('horaris')
    .delete()

```

```

    .eq('grup_id', grup_id)
    .eq('dia_setmana', parseInt(dia_setmana));

    if (deleteError) return res.status(500).json({ error:
deleteError.message });

    if (franges.length === 0) {
        return res.json({ ok: true, missatge: 'Franges buides, s\'han
eliminat totes.' });
    }

    const novesFranges = franges.map(f => ({
        grup_id,
        dia_setmana: parseInt(dia_setmana),
        hora_inici: f.hora_inici,
        durada_min: f.durada_min,
        materia_id: f.materia_id
    }));

    const { error: insertError } = await supabase
        .from('horaris')
        .insert(novesFranges);

    if (insertError) return res.status(500).json({ error:
insertError.message });
    res.json({ ok: true, missatge: `Guardades ${franges.length}
franges.` });
});

// =====
// RUTES PER A ASSISTÈNCIA (tots autenticats)
// =====

function getDiaSetmana(data) {
    const date = new Date(data);
    let dia = date.getDay();
    return dia === 0 ? 7 : dia;
}

app.get('/assistencia/config', autenticacio, async (req, res) => {
    const { grup_id, data } = req.query;
    if (!grup_id || !data) {
        return res.status(400).json({ error: 'Falten grup_id o data' });
    }

    const diaSetmana = getDiaSetmana(data);
    const supabase = getSupabaseAdmin();

    const { data: sessions, error: errSessions } = await supabase
        .from('horaris')
        .select(`*, materies(nom)`)
        .eq('grup_id', grup_id)

```

```

    .eq('dia_setmana', diaSetmana)
    .order('hora_inici');

    if (errSessions) return res.status(500).json({ error:
errSessions.message });

    const { data: alumnes, error: errAlumnes } = await supabase
        .from('alumnes')
        .select('id, nom, cognoms')
        .eq('grup_id', grup_id)
        .eq('actiu', true)
        .order('nom');

    if (errAlumnes) return res.status(500).json({ error:
errAlumnes.message });

    const sessionIds = sessions.map(s => s.id);
    let registresExistents = [];
    if (sessionIds.length > 0) {
        const { data: registres, error: errReg } = await supabase
            .from('registres')
            .select('*', minuts_justificats')
            .in('horari_id', sessionIds)
            .eq('data', data);
        if (!errReg) registresExistents = registres;
    }

    res.json({ sessions, alumnes, registresExistents });
});

app.post('/assistencia/guardar', autenticacio, async (req, res) => {
    const { data, registres } = req.body;
    if (!data || !Array.isArray(registres)) {
        return res.status(400).json({ error: 'Falten dades o format
incorrecte' });
    }

    const supabase = getSupabaseAdmin();
    const registresPerUpsert = registres.map(r => ({
        alumne_id: r.alumne_id,
        horari_id: r.horari_id,
        data: data,
        minuts_assistits: r.minuts_assistits || 0,
        minuts_justificats: r.minuts_justificats || 0,
        observacions: r.observacions || null
    }));

    const { error } = await supabase
        .from('registres')
        .upsert(registresPerUpsert, { onConflict: 'alumne_id, horari_id,
data' });

```

```

    if (error) return res.status(500).json({ error: error.message });
    res.json({ ok: true });
  });

  // =====
  // RUTES PER A DIES NO LECTIUS (només admin)
  // =====

app.get('/dies_no_lectius', autenticacio, adminOnly, async (req, res)
=> {
  const { grup_id } = req.query;
  if (!grup_id) {
    return res.status(400).json({ error: 'Falta grup_id' });
  }
  const supabase = getSupabaseAdmin();
  const { data, error } = await supabase
    .from('dies_no_lectius')
    .select('*')
    .eq('grup_id', grup_id)
    .order('data', { ascending: true });
  if (error) return res.status(500).json({ error: error.message });
  res.json(data);
});

app.post('/dies_no_lectius', autenticacio, adminOnly, async (req, res)
=> {
  const { grup_id, data, motiu } = req.body;
  if (!grup_id || !data) {
    return res.status(400).json({ error: 'Falten grup_id o data' });
  }
  const supabase = getSupabaseAdmin();
  const { error } = await supabase
    .from('dies_no_lectius')
    .insert([ { grup_id, data, motiu } ]);
  if (error) return res.status(500).json({ error: error.message });
  res.json({ ok: true });
});

app.put('/dies_no_lectius/:id', autenticacio, adminOnly, async (req,
res) => {
  const { id } = req.params;
  const { data, motiu } = req.body;
  const supabase = getSupabaseAdmin();
  const { error } = await supabase
    .from('dies_no_lectius')
    .update({ data, motiu })
    .eq('id', id);
  if (error) return res.status(500).json({ error: error.message });
  res.json({ ok: true });
});

app.delete('/dies_no_lectius/:id', autenticacio, adminOnly, async

```

```

(req, res) => {
  const { id } = req.params;
  const supabase = getSupabaseAdmin();
  const { error } = await supabase
    .from('dies_no_lectius')
    .delete()
    .eq('id', id);
  if (error) return res.status(500).json({ error: error.message });
  res.json({ ok: true });
});

app.post('/dies_no_lectius/copiar', autenticacio, adminOnly, async
(req, res) => {
  const { origen_grup_id, desti_grup_id } = req.body;
  if (!origen_grup_id || !desti_grup_id) {
    return res.status(400).json({ error: 'Falten els IDs dels
grups' });
  }

  const supabase = getSupabaseAdmin();
  const { data: diesOrigen, error: errGet } = await supabase
    .from('dies_no_lectius')
    .select('data, motiu')
    .eq('grup_id', origen_grup_id);
  if (errGet) return res.status(500).json({ error: errGet.message });

  if (diesOrigen.length === 0) {
    return res.json({ ok: true, missatge: 'El grup origen no té dies
no lectius' });
  }

  const diesPerInsertar = diesOrigen.map(d => ({
    grup_id: desti_grup_id,
    data: d.data,
    motiu: d.motiu
  }));

  const { error: errInsert } = await supabase
    .from('dies_no_lectius')
    .insert(diesPerInsertar);

  if (errInsert) return res.status(500).json({ error:
errInsert.message });
  res.json({ ok: true, missatge: `Copiats ${diesOrigen.length} dies al
grup destí` });
});

// =====
// RUTES PER A INFORMES (tots autenticats)
// =====

app.get('/informes/assistencia', autenticacio, async (req, res) => {

```



```

const { grup_id, data_inici, data_fi } = req.query;
if (!grup_id || !data_inici || !data_fi) {
  return res.status(400).json({ error: 'Falten paràmetres: grup_id,
data_inici, data_fi' });
}

const supabase = getSupabaseAdmin();

try {
  const { data: horaris, error: errHoraris } = await supabase
    .from('horaris')
    .select('id, dia_setmana, hora_inici, durada_min, materia_id,
materies(nom)')
    .eq('grup_id', grup_id);

  if (errHoraris) throw new Error(errHoraris.message);

  const { data: diesNoLectius, error: errDies } = await supabase
    .from('dies_no_lectius')
    .select('data')
    .eq('grup_id', grup_id)
    .gte('data', data_inici)
    .lte('data', data_fi);

  if (errDies) throw new Error(errDies.message);

  const diesNoLectiusSet = new Set(diesNoLectius.map(d => d.data));

  const { data: alumnes, error: errAlumnes } = await supabase
    .from('alumnes')
    .select('id, nom, cognoms, actiu')
    .eq('grup_id', grup_id)
    .order('nom');

  if (errAlumnes) throw new Error(errAlumnes.message);

  const alumnesIds = alumnes.map(a => a.id);
  const { data: registres, error: errRegistres } = await supabase
    .from('registres')
    .select('alumne_id, horari_id, minuts_assistits,
minuts_justificats, data')
    .in('alumne_id', alumnesIds)
    .gte('data', data_inici)
    .lte('data', data_fi);

  if (errRegistres) throw new Error(errRegistres.message);

  const registresPerAlumne = {};
  registres.forEach(r => {
    if (!registresPerAlumne[r.alumne_id]) {
      registresPerAlumne[r.alumne_id] = [];
    }
  });
}

```

```

    registresPerAlumne[r.alumne_id].push(r);
  });

  const dataIniciDate = new Date(data_inici);
  const dataFiDate = new Date(data_fi);
  let minutsTeoricsPerAlumne = 0;

  for (let d = new Date(dataIniciDate); d <= dataFiDate;
d.setDate(d.getDate() + 1)) {
    const dataStr = d.toISOString().split('T')[0];
    if (diesNoLectiusSet.has(dataStr)) continue;
    const diaSetmana = d.getDay() === 0 ? 7 : d.getDay();
    horaris.forEach(h => {
      if (h.dia_setmana === diaSetmana) {
        minutsTeoricsPerAlumne += h.durada_min;
      }
    });
  }

  function formatHoresMinuts(minuts) {
    const hores = Math.floor(minuts / 60);
    const minutsRestants = minuts % 60;
    if (hores === 0) return `${minutsRestants} min`;
    if (minutsRestants === 0) return `${hores} h`;
    return `${hores} h ${minutsRestants} min`;
  }

  const resultat = alumnes.map(alumne => {
    let minutsAssistits = 0;
    let minutsJustificats = 0;
    const registresAlumne = registresPerAlumne[alumne.id] || [];
    registresAlumne.forEach(r => {
      const dataRegistre = r.data;
      if (!diesNoLectiusSet.has(dataRegistre)) {
        minutsAssistits += r.minuts_assistits || 0;
        minutsJustificats += r.minuts_justificats || 0;
      }
    });
    const percentAssistit = minutsTeoricsPerAlumne > 0 ?
(minutsAssistits / minutsTeoricsPerAlumne) * 100 : 0;
    const percentAssistitJustificat = minutsTeoricsPerAlumne > 0 ?
((minutsAssistits + minutsJustificats) / minutsTeoricsPerAlumne) *
100 : 0;
    return {
      id: alumne.id,
      nom: alumne.nom,
      cognoms: alumne.cognoms,
      actiu: alumne.actiu,
      minuts_teorics: minutsTeoricsPerAlumne,
      minuts_assistits: minutsAssistits,
      minuts_justificats: minutsJustificats,
      hores_teoric: formatHoresMinuts(minutsTeoricsPerAlumne),

```

```

        hores_assistit: formatHoresMinuts(minutsAssistits),
        hores_justificat: formatHoresMinuts(minutsJustificats),
        percent_assistit: Math.round(percentAssistit * 100) / 100,
        percent_assistit_justificat:
Math.round(percentAssistitJustificat * 100) / 100
    };
});

res.json({
    grup_id,
    data_inici,
    data_fi,
    minuts_teorics_totals: minutsTeoricsPerAlumne,
    hores_teoric_total: formatHoresMinuts(minutsTeoricsPerAlumne),
    alumnes: resultat
});
} catch (error) {
    console.error('Error generant informe:', error);
    res.status(500).json({ error: error.message });
}
});

app.get('/informes/assistencia_materia', autenticacio, async (req,
res) => {
    const { grup_id, materia_id, data_inici, data_fi } = req.query;
    if (!grup_id || !materia_id || !data_inici || !data_fi) {
        return res.status(400).json({ error: 'Falten paràmetres' });
    }

    const supabase = getSupabaseAdmin();

    try {
        const { data: sessionsMateria, error: errSessions } = await
supabase
            .from('horaris')
            .select('id, dia_setmana, durada_min')
            .eq('grup_id', grup_id)
            .eq('materia_id', materia_id);
        if (errSessions) throw new Error(errSessions.message);
        const sessionIds = sessionsMateria.map(s => s.id);

        const { data: diesNoLectius, error: errDies } = await supabase
            .from('dies_no_lectius')
            .select('data')
            .eq('grup_id', grup_id)
            .gte('data', data_inici)
            .lte('data', data_fi);
        if (errDies) throw new Error(errDies.message);
        const diesNoLectiusSet = new Set(diesNoLectius.map(d => d.data));

        const dataIniciDate = new Date(data_inici);
        const dataFiDate = new Date(data_fi);

```

```

    let minutsTeorics = 0;
    for (let d = new Date(dataIniciDate); d <= dataFiDate;
d.setDate(d.getDate() + 1)) {
        const dataStr = d.toISOString().split('T')[0];
        if (diesNoLectiusSet.has(dataStr)) continue;
        const diaSetmana = d.getDay() === 0 ? 7 : d.getDay();
        sessionsMateria.forEach(s => {
            if (s.dia_setmana === diaSetmana) minutsTeorics +=
s.durada_min;
        });
    }

    const { data: alumnes, error: errAlumnes } = await supabase
        .from('alumnes')
        .select('id, nom, cognoms, actiu')
        .eq('grup_id', grup_id)
        .order('nom');
    if (errAlumnes) throw new Error(errAlumnes.message);

    const alumnesIds = alumnes.map(a => a.id);
    const { data: registres, error: errRegistres } = await supabase
        .from('registres')
        .select('alumne_id, horari_id, minuts_assistits,
minuts_justificats, data')
        .in('alumne_id', alumnesIds)
        .in('horari_id', sessionIds)
        .gte('data', data_inici)
        .lte('data', data_fi);
    if (errRegistres) throw new Error(errRegistres.message);

    function formatHores(minuts) {
        const h = Math.floor(minuts / 60);
        const m = minuts % 60;
        if (h === 0) return `${m} min`;
        if (m === 0) return `${h} h`;
        return `${h} h ${m} min`;
    }

    const resultat = alumnes.map(alumne => {
        const registresAlumne = registres.filter(r => r.alumne_id ===
alumne.id);
        let minutsAssistits = 0, minutsJustificats = 0;
        registresAlumne.forEach(r => {
            if (!diesNoLectiusSet.has(r.data)) {
                minutsAssistits += r.minuts_assistits || 0;
                minutsJustificats += r.minuts_justificats || 0;
            }
        });
        const percentAssistit = minutsTeorics > 0 ? (minutsAssistits /
minutsTeorics) * 100 : 0;
        const percentAssistitJustificat = minutsTeorics > 0 ?
((minutsAssistits + minutsJustificats) / minutsTeorics) * 100 : 0;
    });

```

```

        return {
            id: alumne.id,
            nom: alumne.nom,
            cognoms: alumne.cognoms,
            actiu: alumne.actiu,
            minuts_teorics: minutsTeorics,
            minuts_assistits: minutsAssistits,
            minuts_justificats: minutsJustificats,
            hores_teoric: formatHores(minutsTeorics),
            hores_assistit: formatHores(minutsAssistits),
            hores_justificat: formatHores(minutsJustificats),
            percent_assistit: Math.round(percentAssistit * 100) / 100,
            percent_assistit_justificat:
Math.round(percentAssistitJustificat * 100) / 100
        };
    });

    res.json({
        grup_id,
        materia_id,
        data_inici,
        data_fi,
        minuts_teorics_totals: minutsTeorics,
        hores_teoric_total: formatHores(minutsTeorics),
        alumnes: resultat
    });
} catch (error) {
    console.error('Error:', error);
    res.status(500).json({ error: error.message });
}
});

app.get('/informes/assistencia_grup', autenticacio, async (req, res)
=> {
    const { grup_id, data_inici, data_fi } = req.query;
    if (!data_inici || !data_fi) {
        return res.status(400).json({ error: 'Falten les dates' });
    }

    const supabase = getSupabaseAdmin();

    try {
        let queryGrups = supabase.from('grups').select('id, nom');
        if (grup_id) queryGrups = queryGrups.eq('id', grup_id);
        const { data: grups, error: errGrups } = await queryGrups;
        if (errGrups) throw new Error(errGrups.message);

        const resultat = [];

        for (const grup of grups) {
            const { data: horaris, error: errHor } = await supabase
                .from('horaris')

```

```

        .select('dia_setmana, durada_min')
        .eq('grup_id', grup.id);
    if (errHor) throw new Error(errHor.message);

    const { data: diesNoLectius, error: errDies } = await supabase
        .from('dies_no_lectius')
        .select('data')
        .eq('grup_id', grup.id)
        .gte('data', data_inici)
        .lte('data', data_fi);
    if (errDies) throw new Error(errDies.message);
    const diesNoLectiusSet = new Set(diesNoLectius.map(d =>
d.data));

    const dataIniciDate = new Date(data_inici);
    const dataFiDate = new Date(data_fi);
    let minutsTeorics = 0;
    for (let d = new Date(dataIniciDate); d <= dataFiDate;
d.setDate(d.getDate() + 1)) {
        const dataStr = d.toISOString().split('T')[0];
        if (diesNoLectiusSet.has(dataStr)) continue;
        const diaSetmana = d.getDay() === 0 ? 7 : d.getDay();
        horaris.forEach(h => {
            if (h.dia_setmana === diaSetmana) minutsTeorics +=
h.durada_min;
        });
    }

    const { data: alumnes, error: errAl } = await supabase
        .from('alumnes')
        .select('id')
        .eq('grup_id', grup.id);
    if (errAl) throw new Error(errAl.message);
    const alumnesIds = alumnes.map(a => a.id);
    let minutsAssistitsGrup = 0, minutsJustificatsGrup = 0;
    if (alumnesIds.length > 0) {
        const { data: registres, error: errReg } = await supabase
            .from('registres')
            .select('minuts_assistits, minuts_justificats, data')
            .in('alumne_id', alumnesIds)
            .gte('data', data_inici)
            .lte('data', data_fi);
        if (errReg) throw new Error(errReg.message);
        registres.forEach(r => {
            if (!diesNoLectiusSet.has(r.data)) {
                minutsAssistitsGrup += r.minuts_assistits || 0;
                minutsJustificatsGrup += r.minuts_justificats || 0;
            }
        });
    }

    const percentAssistit = minutsTeorics > 0 ? (minutsAssistitsGrup

```

```

/ minutsTeorics) * 100 : 0;
    const percentAssistitJustificat = minutsTeorics > 0 ?
((minutsAssistitsGrup + minutsJustificatsGrup) / minutsTeorics) *
100 : 0;

    function formatHores(minuts) {
        const h = Math.floor(minuts / 60);
        const m = minuts % 60;
        if (h === 0) return `${m} min`;
        if (m === 0) return `${h} h`;
        return `${h} h ${m} min`;
    }

    resultat.push({
        grup_id: grup.id,
        grup_nom: grup.nom,
        hores_teoric: formatHores(minutsTeorics),
        hores_assistit: formatHores(minutsAssistitsGrup),
        hores_justificat: formatHores(minutsJustificatsGrup),
        percent_assistit: Math.round(percentAssistit * 100) / 100,
        percent_assistit_justificat:
Math.round(percentAssistitJustificat * 100) / 100,
    });
}

    res.json({ data_inici, data_fi, grups: resultat });
} catch (error) {
    console.error(error);
    res.status(500).json({ error: error.message });
}
});

app.get('/informes/alertes', autenticacio, async (req, res) => {
    const { grup_id, data_inici, data_fi, lllindar_personalitzat } =
req.query;
    if (!data_inici || !data_fi) {
        return res.status(400).json({ error: 'Falten les dates' });
    }

    const supabase = getSupabaseAdmin();

    try {
        let queryGrups = supabase.from('grups').select('id, nom,
llindar_assistencia');
        if (grup_id) queryGrups = queryGrups.eq('id', grup_id);
        const { data: grups, error: errGrups } = await queryGrups;
        if (errGrups) throw new Error(errGrups.message);

        const alertes = [];

        for (const grup of grups) {
            const { data: horaris, error: errHor } = await supabase

```

```

        .from('horaris')
        .select('dia_setmana, durada_min')
        .eq('grup_id', grup.id);
    if (errHor) throw new Error(errHor.message);

    const { data: diesNoLectius, error: errDies } = await supabase
        .from('dies_no_lectius')
        .select('data')
        .eq('grup_id', grup.id)
        .gte('data', data_inici)
        .lte('data', data_fi);
    if (errDies) throw new Error(errDies.message);
    const diesNoLectiusSet = new Set(diesNoLectius.map(d =>
d.data));

    const dataIniciDate = new Date(data_inici);
    const dataFiDate = new Date(data_fi);
    let minutsTeorics = 0;
    for (let d = new Date(dataIniciDate); d <= dataFiDate;
d.setDate(d.getDate() + 1)) {
        const dataStr = d.toISOString().split('T')[0];
        if (diesNoLectiusSet.has(dataStr)) continue;
        const diaSetmana = d.getDay() === 0 ? 7 : d.getDay();
        horaris.forEach(h => {
            if (h.dia_setmana === diaSetmana) minutsTeorics +=
h.durada_min;
        });
    }
    if (minutsTeorics === 0) continue;

    const { data: alumnes, error: errAl } = await supabase
        .from('alumnes')
        .select('id, nom, cognoms')
        .eq('grup_id', grup.id)
        .eq('actiu', true);
    if (errAl) throw new Error(errAl.message);
    if (alumnes.length === 0) continue;

    const alumnesIds = alumnes.map(a => a.id);
    const { data: registres, error: errReg } = await supabase
        .from('registres')
        .select('alumne_id, minuts_assistits, minuts_justificats,
data')
        .in('alumne_id', alumnesIds)
        .gte('data', data_inici)
        .lte('data', data_fi);
    if (errReg) throw new Error(errReg.message);

    const registresPerAlumne = {};
    registres.forEach(r => {
        if (!registresPerAlumne[r.alumne_id])
registresPerAlumne[r.alumne_id] = [];
    });

```



```

    registresPerAlumne[r.alumne_id].push(r);
  });

  function formatHores(minuts) {
    const h = Math.floor(minuts / 60);
    const m = minuts % 60;
    if (h === 0) return `${m} min`;
    if (m === 0) return `${h} h`;
    return `${h} h ${m} min`;
  }

  for (const alumne of alumnes) {
    const registresAlumne = registresPerAlumne[alumne.id] || [];
    let minutsAssistits = 0, minutsJustificats = 0;
    registresAlumne.forEach(r => {
      if (!diesNoLectiusSet.has(r.data)) {
        minutsAssistits += r.minuts_assistits || 0;
        minutsJustificats += r.minuts_justificats || 0;
      }
    });
    const percentAssistit = (minutsAssistits / minutsTeorics) *
100;
    const lllindar = lllindar_personalitzat ?
parseFloat(lllindar_personalitzat) : grup.lllindar_assistencia;
    if (percentAssistit < lllindar) {
      alertes.push({
        alumne_id: alumne.id,
        alumne_nom: alumne.nom,
        alumne_cognoms: alumne.cognoms,
        grup_id: grup.id,
        grup_nom: grup.nom,
        hores_teoric: formatHores(minutsTeorics),
        hores_assistit: formatHores(minutsAssistits),
        minuts_teoric: minutsTeorics,
        minuts_assistit: minutsAssistits,
        percent_assistit: Math.round(percentAssistit * 100) / 100,
        lllindar: lllindar,
        lllindar_grup: grup.lllindar_assistencia,
        lllindar_personalitzat_utilitzat: !!lllindar_personalitzat
      });
    }
  }
}

alertes.sort((a, b) => a.percent_assistit - b.percent_assistit);
res.json({ data_inici, data_fi, total_alertes: alertes.length,
alertes });
} catch (error) {
  console.error('Error generant alertes:', error);
  res.status(500).json({ error: error.message });
}
});

```

```
// =====
// INICI DEL SERVIDOR
// =====

app.listen(3000, '0.0.0.0', () => console.log('Backend corrent a
http://0.0.0.0:3000'))
```

7.2.3. Configuració de Supabase

KASBA-backend/src/config/supabase.js

```
const { createClient } = require('@supabase/supabase-js');
const WebSocket = require('ws');
require('dotenv').config();

const supabaseUrl = process.env.SUPABASE_URL;
const supabaseAnonKey = process.env.SUPABASE_ANON_KEY;
const supabaseServiceKey = process.env.SUPABASE_SERVICE_KEY;

// Opció per evitar errors de WebSocket a Node.js 20
const customFetch = async (url, options) => {
  return fetch(url, options);
};

const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  realtime: {
    transport: WebSocket,
  },
});

const supabaseAdmin = createClient(supabaseUrl, supabaseServiceKey, {
  realtime: {
    transport: WebSocket,
  },
});

module.exports = { supabase, supabaseAdmin };
```

7.2.4. Scripts auxiliars

KASBA-backend/src/scripts/altaProfessor.js

```
import { createClient } from '@supabase/supabase-js'
import dotenv from 'dotenv'
dotenv.config()

const supabaseAdmin = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_KEY
```

```

)

export async function altaProfessor({ email, nom, rol, password }) {
  // 1. Crear l'usuari a Auth
  const { data: authData, error: authError } = await
supabaseAdmin.auth.admin.createUser({
    email,
    password,
    email_confirm: true
  })

  if (authError) throw new Error('Error Auth: ' + authError.message)

  const userId = authData.user.id

  // 2. Inserir a la taula professors
  const { error: dbError } = await supabaseAdmin
    .from('professors')
    .insert([ { id: userId, email, nom, rol } ])

  if (dbError) {
    // Si falla la BD, esborrem l'usuari d'Auth per no deixar-lo
penjat
    await supabaseAdmin.auth.admin.deleteUser(userId)
    throw new Error('Error BD: ' + dbError.message)
  }

  return { id: userId, email, nom, rol }
}

```

KASBA-backend/src/scripts/crearProfessor.js

```

const { createClient } = require('@supabase/supabase-js');
const WebSocket = require('ws');

const supabaseUrl = 'https://aurzyqkucdbedqfrtu.supabase.co';
const supabaseServiceKey =
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImFhdXJ6enlxa2N1ZGJlZHFmcnR1Iiwicm9sZSI6InNlcjZpY2Vfcm9sZSIsImhhdCI6MTc4MDI4Njk2MSwiZXhwIjoyMDk1ODYyOTYxfQ._Vn1cZ0tfyhHfVLI8Ja1HPvN2l4MBIs4LVsdGPOahYo';

const supabaseAdmin = createClient(supabaseUrl, supabaseServiceKey, {
  realtime: { transport: WebSocket }
});

async function crearProfessor(email, nom, rol = 'professor') {
  try {
    console.log(` Creant professor: ${nom} (${email})...`);

```

```

    const { data: authUser, error: authError } = await
supabaseAdmin.auth.admin.createUser({
    email: email,
    email_confirm: true,
    password: 'Canviar123!',
    user_metadata: { nom, rol }
});

if (authError) {
    console.error('✖ Error creant usuari a auth:', authError);
    return;
}

console.log('✔ Usuari creat amb ID:', authUser.user.id);

const { data: professor, error: dbError } = await supabaseAdmin
    .from('professors')
    .insert([
        { id: authUser.user.id, email, nom, rol }
    ])
    .select();

if (dbError) {
    console.error('✖ Error inserint a professors:', dbError);
    await supabaseAdmin.auth.admin.deleteUser(authUser.user.id);
    return;
}

console.log('✔ Professor creat correctament!');
console.log(`    Email: ${email}`);
console.log(`    Password temporal: Canviar123!`);
console.log(`    Rol: ${rol}`);
} catch (error) {
    console.error('✖ Error inesperat:', error);
}
}

const email = process.argv[2];
const nom = process.argv[3];
const rol = process.argv[4] || 'professor';

if (!email || !nom) {
    console.log('Ús: node src/scripts/crearProfessor.js <email> <nom> [rol]');
    process.exit(1);
}

crearProfessor(email, nom, rol);

```

KASBA-backend/src/scripts/testConnection.js

```

const { supabase } = require('../config/supabase');

async function testConnection() {
  console.log(' Connectant a Supabase...');

  const { count, error } = await supabase
    .from('professors')
    .select('*', { count: 'exact', head: true });

  if (error) {
    console.error('✖ Error de connexió o permisos:', error.message);
    console.log('i Intentant accedir a la taula grups...');
    const { data, error: err2 } = await supabase
      .from('grups')
      .select('id')
      .limit(1);
    if (err2) {
      console.error('✖ No s\'ha pogut accedir a grups:',
err2.message);
    } else {
      console.log('✓ Connexió OK, taula grups accessible. Resultat:',
data);
    }
  } else {
    console.log(`✓ Connexió exitosa! Hi ha ${count} professor(s) a la
base de dades.`);
  }
}

testConnection();

```

7.2.5. Pàgines públiques de prova

KASBA-backend/public/index.html

```

<!DOCTYPE html>
<html>
<head>
  <title>KASBA - Prova de Connexió</title>
  <script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-
js@2"></script>
</head>
<body>
  <h1>KASBA - Prova de Connexió a Supabase</h1>
  <button onclick="carregarDades()">Carregar Professors</button>
  <pre id="resultat"></pre>

  <script>
    // Substitueix LA_TEVA_CLAU_ANON per la teva clau pública real
    const SUPABASE_URL =

```

```

'https://aaurzyqkucdbedqfrtu.supabase.co';
    const SUPABASE_ANON_KEY = 'LA_TEVA_CLAU_ANON';

    const supabase = window.supabase.createClient(SUPABASE_URL,
SUPABASE_ANON_KEY);

    async function carregarDades() {
        const resultatDiv = document.getElementById('resultat');
        resultatDiv.textContent = 'Carregant...';

        const { data, error } = await supabase
            .from('professors')
            .select('*');

        if (error) {
            console.error('Error en la consulta:', error);
            resultatDiv.textContent = `Error: ${error.message}`;
        } else {
            console.log('Dades rebudes:', data);
            resultatDiv.textContent = JSON.stringify(data, null,
2);
        }
    }
</script>
</body>
</html>

```

KASBA-backend/public/gestio_kasba.html

```

<!DOCTYPE html>
<html lang="ca">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Gestió KASBA - Professors</title>
    <!-- Estils mínims per claredat -->
    <style>
        body { font-family: Arial, sans-serif; margin: 2rem;
background: #f4f7fc; }
        h1, h2 { color: #2c3e66; }
        .container { max-width: 1200px; margin: auto; background:
white; padding: 2rem; border-radius: 10px; box-shadow: 0 2px 8px
rgba(0,0,0,0.1); }
        table { width: 100%; border-collapse: collapse; margin: 1rem
0; }
        th, td { border: 1px solid #ddd; padding: 8px; text-align:
left; }
        th { background-color: #2c3e66; color: white; }
        form { display: flex; gap: 10px; flex-wrap: wrap; margin-

```

```

bottom: 2rem; align-items: end; }
    input, select, button { padding: 8px 12px; border-radius: 4px;
border: 1px solid #ccc; }
    button { background: #2c3e66; color: white; cursor: pointer;
border: none; }
    button:hover { background: #1a2a4a; }
    .accions button { background: #e67e22; margin-right: 5px; }
    .accions button.edit { background: #f39c12; }
    .accions button.delete { background: #e74c3c; }
    .missatge { margin: 1rem 0; padding: 0.5rem; border-radius:
5px; }
    .error { background: #f8d7da; color: #721c24; border: 1px
solid #f5c6cb; }
    .exit { background: #d4edda; color: #155724; border: 1px solid
#c3e6cb; }
    </style>
</head>
<body>
<div class="container">
    <h1><img alt="KASBA logo" data-bbox="205 395 225 405"/> Gestió KASBA - Professors</h1>

    <!-- Formulari per afegir/editar professor -->
    <h2>+ Afegir / Editar Professor</h2>
    <form id="professorForm">
        <input type="hidden" id="professorId">
        <input type="text" id="nom" placeholder="Nom complet" required
style="min-width: 180px;">
        <input type="email" id="email" placeholder="Email">
        <input type="text" id="departament" placeholder="Departament">
        <button type="submit" id="submitBtn">Desar Professor</button>
        <button type="button" id="cancelBtn"
style="background:#95a5a6;">Cancel·lar</button>
    </form>

    <!-- Llistat de professors -->
    <h2><img alt="List icon" data-bbox="205 655 225 665"/> Llista de Professors</h2>
    <table id="professorsTable">
        <thead>

<tr><th>ID</th><th>Nom</th><th>Email</th><th>Departament</th><th>Accio
ns</th></tr>
        </thead>
        <tbody></tbody>
    </table>
    <div id="missatge"></div>
</div>

<!-- Incloure la llibreria Supabase -->
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-
js@2"></script>

```

```

<script>
  import { createClient } from
  'https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2/+esm';
  // CONFIGURACIÓ DE SUPABASE (posa aquí les teves credencials)
  const SUPABASE_URL = 'https://aaurzzyqkcudbedqfrtu.supabase.co';
  const SUPABASE_ANON_KEY =
  'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6
  ImFhdXJ6enlxa2N1ZGJlZHFmcnR1Iiwicm9sZSI6ImFub24iLCJpYXQiOiE3ODAyODY5Nj
  EsImV4cCI6MjA5NTg2Mjk2MX0.eGoksfgkRLjgSiMIscCdg_gynmNkcmJxKrKEvkzHN0k'
  ;

  const supabase = window.supabase.createClient(SUPABASE_URL,
  SUPABASE_ANON_KEY);

  let editMode = false;

  // Funció per mostrar missatges
  function mostrarMissatge(text, tipus) {
    const div = document.getElementById('missatge');
    div.innerHTML = `<div class="missatge
    ${tipus}">${text}</div>`;
    setTimeout(() => div.innerHTML = '', 3000);
  }

  // Carregar tots els professors i pintar la taula
  async function carregarProfessors() {
    const { data, error } = await
    supabase.from('professors').select('*').order('nom');
    if (error) {
      mostrarMissatge('Error carregant professors: ' +
      error.message, 'error');
      return;
    }
    const tbody = document.querySelector('#professorsTable
    tbody');
    tbody.innerHTML = '';
    data.forEach(prof => {
      const row = tbody.insertRow();
      row.insertCell(0).textContent = prof.id.substring(0,8); //
      mostrar part de l'ID
      row.insertCell(1).textContent = prof.nom;
      row.insertCell(2).textContent = prof.email || '';
      row.insertCell(3).textContent = prof.departament || '';
      const accionsCell = row.insertCell(4);
      accionsCell.className = 'accions';

      const btnEdit = document.createElement('button');
      btnEdit.textContent = '✎ Editar';
      btnEdit.className = 'edit';
      btnEdit.onclick = () => omplirFormulari(prof);
    });
  }

```



```

        const btnDelete = document.createElement('button');
        btnDelete.textContent = '🗑 Eliminar';
        btnDelete.className = 'delete';
        btnDelete.onclick = () => eliminarProfessor(prof.id);

        accionsCell.appendChild(btnEdit);
        accionsCell.appendChild(btnDelete);
    });
}

// Omplir formulari per editar
function omplirFormulari(professor) {
    document.getElementById('professorId').value = professor.id;
    document.getElementById('nom').value = professor.nom;
    document.getElementById('email').value = professor.email ||
'';
    document.getElementById('departament').value =
professor.departament || '';
    document.getElementById('submitBtn').textContent =
'Actualitzar Professor';
    editMode = true;
}

// Cancel·lar edició
document.getElementById('cancelBtn').onclick = () => {
    document.getElementById('professorForm').reset();
    document.getElementById('professorId').value = '';
    document.getElementById('submitBtn').textContent = 'Desar
Professor';
    editMode = false;
};

// Eliminar professor
async function eliminarProfessor(id) {
    if (!confirm('Segur que vols eliminar aquest professor?'))
return;
    const { error } = await
supabase.from('professors').delete().eq('id', id);
    if (error) {
        mostrarMissatge('Error eliminant: ' + error.message,
'error');
    } else {
        mostrarMissatge('Professor eliminat correctament',
'exit');
        carregarProfessors();
    }
}

// Enviar formulari (crear o actualitzar)

```

```

document.getElementById('professorForm').addEventListener('submit',
async (e) => {
    e.preventDefault();
    const id = document.getElementById('professorId').value;
    const nom = document.getElementById('nom').value.trim();
    const email = document.getElementById('email').value.trim();
    const departament =
document.getElementById('departament').value.trim();

    if (!nom) {
        mostrarMissatge('El nom és obligatori', 'error');
        return;
    }

    if (editMode && id) {
        // Actualitzar
        const { error } = await
supabase.from('professors').update({ nom, email,
departament }).eq('id', id);
        if (error) {
            mostrarMissatge('Error actualitzant: ' +
error.message, 'error');
        } else {
            mostrarMissatge('Professor actualitzat', 'exit');
            cancelForm();
            carregarProfessors();
        }
    } else {
        // Crear nou
        const { error } = await
supabase.from('professors').insert([[{ nom, email, departament }]]);
        if (error) {
            mostrarMissatge('Error creant: ' + error.message,
'error');
        } else {
            mostrarMissatge('Professor afegit correctament',
'exit');
            cancelForm();
            carregarProfessors();
        }
    }
});

function cancelForm() {
    document.getElementById('professorForm').reset();
    document.getElementById('professorId').value = '';
    document.getElementById('submitBtn').textContent = 'Desar
Professor';
    editMode = false;
}

// Carregar dades en iniciar

```

```
    carregarProfessors();  
</script>  
</body>  
</html>
```

7.3. Frontend (KASBA-frontend)

7.3.1. Configuració

KASBA-frontend/package.json

```
{  
  "name": "kasba-frontend",  
  "private": true,  
  "version": "0.0.0",  
  "type": "module",  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build",  
    "lint": "eslint .",  
    "preview": "vite preview"  
  },  
  "dependencies": {  
    "@supabase/supabase-js": "^2.107.0",  
    "react": "^19.2.6",  
    "react-dom": "^19.2.6"  
  },  
  "devDependencies": {  
    "@eslint/js": "^10.0.1",  
    "@types/react": "^19.2.14",  
    "@types/react-dom": "^19.2.3",  
    "@vitejs/plugin-react": "^6.0.1",  
    "eslint": "^10.3.0",  
    "eslint-plugin-react-hooks": "^7.1.1",  
    "eslint-plugin-react-refresh": "^0.5.2",  
    "globals": "^17.6.0",  
    "vite": "^8.0.12"  
  }  
}
```

KASBA-frontend/vite.config.js

```
import { defineConfig } from 'vite'  
import react from '@vitejs/plugin-react'  
  
// https://vite.dev/config/  
export default defineConfig({  
  plugins: [react()],  
})
```

KASBA-frontend/eslint.config.js

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true } },
    },
  },
])
```

KASBA-frontend/.gitignore

```
# Logs
logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*
lerna-debug.log*

node_modules
dist
dist-ssr
*.local

# Editor directories and files
.vscode/*
!.vscode/extensions.json
.idea
.DS_Store
*.suo
*.ntvs*
*.njsproj
```

```
*.sln  
*.sw?  
.env
```

KASBA-frontend/index.html

```
<!doctype html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8" />  
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />  
    <meta name="viewport" content="width=device-width, initial-  
scale=1.0" />  
    <title>kasba-frontend</title>  
  </head>  
  <body>  
    <div id="root"></div>  
    <script type="module" src="/src/main.jsx"></script>  
  </body>  
</html>
```

7.3.2. Punt d'entrada i configuració global

src/main.jsx

```
// main.jsx  
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import App from './App';  
import { AuthProvider } from './context/AuthContext';  
  
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <AuthProvider>  
      <App />  
    </AuthProvider>  
  </React.StrictMode>  
>);
```

src/App.jsx

```
import { useState } from 'react';  
import { useAuth } from './context/AuthContext';  
import Login from './components/Login';  
import AlumneManagement from './components/AlumneManagement';  
import AssistenciaManagement from  
'./components/AssistenciaManagement';  
import HorariGrup from './components/HorariGrup';
```

```

import InformesPrincipal from
'./components/informes/InformesPrincipal';
import ConfiguracioPrincipal from
'./components/ConfiguracioPrincipal';

function AppContent() {
  const { user, logout, loading } = useAuth();
  const [seccio, setSeccio] = useState('assistencia');

  if (loading) return <div style={{ textAlign: 'center', marginTop:
50 }}>Carregant...</div>;
  if (!user) return <Login />;

  const esAdmin = user.rol === 'admin';
  const esTutor = user.rol === 'tutor';
  const esProfessor = user.rol === 'professor';

  return (
    <div style={{ maxWidth: 1000, margin: '40px auto', padding: '0
20px' }}>
      <div style={{ display: 'flex', justifyContent: 'space-between',
alignItems: 'center', marginBottom: 16 }}>
        <h1 style={{ fontSize: 24 }}> KASBA - Gestió acadèmica</h1>
        <div>
          <span style={{ marginRight: 12, fontSize: 14 }}>{user.nom}
({user.rol})</span>
          <button onClick={logout} style={{ background: '#dc3545',
color: 'white', border: 'none', padding: '6px 12px', borderRadius: 6,
cursor: 'pointer' }}>
            Tancar sessió
          </button>
        </div>
      </div>
      <div>
        <nav style={{ marginBottom: 20, borderBottom: '1px solid #ccc',
paddingBottom: 8, display: 'flex', flexWrap: 'wrap', gap: 8 }}>
          <button onClick={() => setSeccio('assistencia')}
style={{ background: seccio === 'assistencia' ? '#2d5be3' : '#f0eee8',
color: seccio === 'assistencia' ? '#fff' : '#000', border: 'none',
padding: '8px 16px', borderRadius: 6, cursor: 'pointer' }}>Assistència</button>
          <button onClick={() => setSeccio('horari_grup')}
style={{ background: seccio === 'horari_grup' ? '#2d5be3' : '#f0eee8',
color: seccio === 'horari_grup' ? '#fff' : '#000', border: 'none',
padding: '8px 16px', borderRadius: 6, cursor: 'pointer' }}>Horaris</button>
          <button onClick={() => setSeccio('informes')}
style={{ background: seccio === 'informes' ? '#2d5be3' : '#f0eee8',
color: seccio === 'informes' ? '#fff' : '#000', border: 'none',
padding: '8px 16px', borderRadius: 6, cursor: 'pointer' }}>Informes</button>

```

```

        {(esAdmin || esTutor) && (
            <button onClick={() => setSeccio('alumnes')}
style={{ background: seccio === 'alumnes' ? '#2d5be3' : '#f0eee8',
color: seccio === 'alumnes' ? '#fff' : '#000', border: 'none',
padding: '8px 16px', borderRadius: 6, cursor: 'pointer' }}> 
Alumnes</button>
        )}

        {esAdmin && (
            <button onClick={() => setSeccio('configuracio')}
style={{ background: seccio === 'configuracio' ? '#2d5be3' :
'#f0eee8', color: seccio === 'configuracio' ? '#fff' : '#000', border:
'none', padding: '8px 16px', borderRadius: 6, cursor: 'pointer' }}>
Configuració</button>
        )}
    </nav>

    {seccio === 'assistencia' && <AssistenciaManagement />}
    {seccio === 'horari_grup' && <HorariGrup />}
    {seccio === 'informes' && <InformesPrincipal />}
    {(esAdmin || esTutor) && seccio === 'alumnes' &&
<AlumneManagement />}
    {esAdmin && seccio === 'configuracio' && <ConfiguracioPrincipal
/>}
  </div>
);
}

export default AppContent;

```

src/App.css

```

.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

```

```

}

.hero {
  position: relative;

  .base,
  .framework,
  .vite {
    inset-inline: 0;
    margin: 0 auto;
  }

  .base {
    width: 170px;
    position: relative;
    z-index: 0;
  }

  .framework,
  .vite {
    position: absolute;

    .framework {
      z-index: 1;
      top: 34px;
      height: 28px;
      transform: perspective(2000px) rotateZ(300deg) rotateX(44deg)
rotateY(39deg)
        scale(1.4);
    }

    .vite {
      z-index: 0;
      top: 107px;
      height: 26px;
      width: auto;
      transform: perspective(2000px) rotateZ(300deg) rotateX(40deg)
rotateY(39deg)
        scale(0.8);
    }
  }
}

#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;

  @media (max-width: 1024px) {

```



```

        padding: 32px 20px 24px;
        gap: 18px;
    }
}

#next-steps {
    display: flex;
    border-top: 1px solid var(--border);
    text-align: left;

    & > div {
        flex: 1 1 0;
        padding: 32px;
        @media (max-width: 1024px) {
            padding: 24px 20px;
        }
    }

    .icon {
        margin-bottom: 16px;
        width: 22px;
        height: 22px;
    }

    @media (max-width: 1024px) {
        flex-direction: column;
        text-align: center;
    }
}

#docs {
    border-right: 1px solid var(--border);

    @media (max-width: 1024px) {
        border-right: none;
        border-bottom: 1px solid var(--border);
    }
}

#next-steps ul {
    list-style: none;
    padding: 0;
    display: flex;
    gap: 8px;
    margin: 32px 0 0;

    .logo {
        height: 18px;
    }

    a {
        color: var(--text-h);
    }
}

```

```

    font-size: 16px;
    border-radius: 6px;
    background: var(--social-bg);
    display: flex;
    padding: 6px 12px;
    align-items: center;
    gap: 8px;
    text-decoration: none;
    transition: box-shadow 0.3s;

    &:hover {
        box-shadow: var(--shadow);
    }
    .button-icon {
        height: 18px;
        width: 18px;
    }
}

@media (max-width: 1024px) {
    margin-top: 20px;
    flex-wrap: wrap;
    justify-content: center;

    li {
        flex: 1 1 calc(50% - 8px);
    }

    a {
        width: 100%;
        justify-content: center;
        box-sizing: border-box;
    }
}

}

#spacer {
    height: 88px;
    border-top: 1px solid var(--border);
    @media (max-width: 1024px) {
        height: 48px;
    }
}

.ticks {
    position: relative;
    width: 100%;

    &::before,
    &::after {
        content: '';
        position: absolute;

```

```

    top: -4.5px;
    border: 5px solid transparent;
  }

  &::before {
    left: 0;
    border-left-color: var(--border);
  }
  &::after {
    right: 0;
    border-right-color: var(--border);
  }
}

```

src/index.css

```

:root {
  --text: #6b6375;
  --text-h: #08060d;
  --bg: #fff;
  --border: #e5e4e7;
  --code-bg: #f4f3ec;
  --accent: #aa3bff;
  --accent-bg: rgba(170, 59, 255, 0.1);
  --accent-border: rgba(170, 59, 255, 0.5);
  --social-bg: rgba(244, 243, 236, 0.5);
  --shadow:
    rgba(0, 0, 0, 0.1) 0 10px 15px -3px, rgba(0, 0, 0, 0.05) 0 4px 6px
    -2px;

  --sans: system-ui, 'Segoe UI', Roboto, sans-serif;
  --heading: system-ui, 'Segoe UI', Roboto, sans-serif;
  --mono: ui-monospace, Consolas, monospace;

  font: 18px/145% var(--sans);
  letter-spacing: 0.18px;
  color-scheme: light dark;
  color: var(--text);
  background: var(--bg);
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;

  @media (max-width: 1024px) {
    font-size: 16px;
  }
}

@media (prefers-color-scheme: dark) {

```

```

:root {
  --text: #9ca3af;
  --text-h: #f3f4f6;
  --bg: #16171d;
  --border: #2e303a;
  --code-bg: #1f2028;
  --accent: #c084fc;
  --accent-bg: rgba(192, 132, 252, 0.15);
  --accent-border: rgba(192, 132, 252, 0.5);
  --social-bg: rgba(47, 48, 58, 0.5);
  --shadow:
    rgba(0, 0, 0, 0.4) 0 10px 15px -3px, rgba(0, 0, 0, 0.25) 0 4px
    6px -2px;
}

#social .button-icon {
  filter: invert(1) brightness(2);
}

body {
  margin: 0;
}

#root {
  width: 1126px;
  max-width: 100%;
  margin: 0 auto;
  text-align: center;
  border-inline: 1px solid var(--border);
  min-height: 100svh;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
}

h1,
h2 {
  font-family: var(--heading);
  font-weight: 500;
  color: var(--text-h);
}

h1 {
  font-size: 56px;
  letter-spacing: -1.68px;
  margin: 32px 0;
  @media (max-width: 1024px) {
    font-size: 36px;
    margin: 20px 0;
  }
}

```

```

h2 {
  font-size: 24px;
  line-height: 118%;
  letter-spacing: -0.24px;
  margin: 0 0 8px;
  @media (max-width: 1024px) {
    font-size: 20px;
  }
}
p {
  margin: 0;
}

code,
.counter {
  font-family: var(--mono);
  display: inline-flex;
  border-radius: 4px;
  color: var(--text-h);
}

code {
  font-size: 15px;
  line-height: 135%;
  padding: 4px 8px;
  background: var(--code-bg);
}

```

7.3.3. API i Supabase

src/api.js

```

const API_URL = 'http://192.168.3.10:3000';

export function apiFetch(endpoint, options = {}) {
  const token = localStorage.getItem('token');
  const headers = {
    'Content-Type': 'application/json',
    ...options.headers,
  };
  if (token) {
    headers['Authorization'] = `Bearer ${token}`;
  }
  return fetch(`${API_URL}${endpoint}`, { ...options, headers });
}

```

src/lib/supabase.js

```

import { createClient } from '@supabase/supabase-js'

```

```
export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
)
```

7.3.4. Context d'autenticació

src/context/AuthContext.jsx

```
import { createContext, useContext, useState, useEffect } from
'react';
import { apiFetch } from '../api';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const storedToken = localStorage.getItem('token');
    const storedUser = localStorage.getItem('user');
    if (storedToken && storedUser) {
      setToken(storedToken);
      setUser(JSON.parse(storedUser));
    }
    setLoading(false);
  }, []);

  const login = async (email, password) => {
    const res = await apiFetch('/login', {
      method: 'POST',
      body: JSON.stringify({ email, password })
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error);
    setToken(data.token);
    setUser(data.user);
    localStorage.setItem('token', data.token);
    localStorage.setItem('user', JSON.stringify(data.user));
    return data;
  };

  const logout = () => {
    setToken(null);
    setUser(null);
    localStorage.removeItem('token');
    localStorage.removeItem('user');
  };
}
```

```

    return (
      <AuthContext.Provider value={{ user, token, login, logout,
loading }}>
        {children}
      </AuthContext.Provider>
    );
  }

export function useAuth() {
  return useContext(AuthContext);
}

```

7.3.5. Components principals

src/components/Login.jsx

```

import { useState } from 'react';
import { useAuth } from '../context/AuthContext';

export default function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setLoading(true);
    try {
      await login(email, password);
      window.location.reload(); // Recarregar per aplicar canvis
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div style={{ maxWidth: 400, margin: '100px auto', padding: 20,
border: '1px solid #ccc', borderRadius: 10, background: '#fff' }}>
      <h2 style={{ textAlign: 'center' }}>Inici de sessió</h2>
      <form onSubmit={handleSubmit}>
        <div style={{ marginBottom: 12 }}>
          <label>Email</label>
          <input
            type="email"
            value={email}
            onChange={e => setEmail(e.target.value)}

```

```

        required
        style={{ width: '100%', padding: 8, border: '1px solid
#ccc', borderRadius: 4 }}
      />
    </div>
    <div style={{ marginBottom: 12 }}>
      <label>Contrasenya</label>
      <input
        type="password"
        value={password}
        onChange={e => setPassword(e.target.value)}
        required
        style={{ width: '100%', padding: 8, border: '1px solid
#ccc', borderRadius: 4 }}
      />
    </div>
    {error && <div style={{ color: 'red', marginBottom:
12 }}>{error}</div>}
    <button
      type="submit"
      disabled={loading}
      style={{ background: '#2d5be3', color: 'white', padding:
'8px 16px', border: 'none', borderRadius: 6, width: '100%', cursor:
'pointer' }}
    >
      {loading ? 'Carregant...' : 'Iniciar sessió'}
    </button>
  </form>
</div>
);
}

```

src/components/ProfessorManagement.jsx

```

import { useState, useEffect } from 'react';
import { apiFetch } from '../api';

export default function ProfessorManagement() {
  const [professors, setProfessors] = useState([]);
  const [loading, setLoading] = useState(true);
  const [form, setForm] = useState({ email: '', nom: '', rol:
'professor', password: '' });
  const [editingId, setEditingId] = useState(null);
  const [saving, setSaving] = useState(false);
  const [missatge, setMissatge] = useState(null);

  useEffect(() => {
    carregarProfessors();
  }, []);

```



```

async function carregarProfessors() {
  setLoading(true);
  try {
    const res = await apiFetch('/professors');
    if (!res.ok) {
      let errorMsg = await res.text();
      try {
        const data = JSON.parse(errorMsg);
        errorMsg = data.error || errorMsg;
      } catch { /* no fer res */ }
      throw new Error(errorMsg);
    }
    const data = await res.json();
    setProfessors(Array.isArray(data) ? data : []);
  } catch (err) {
    setMissatge({ tipus: 'error', text: err.message });
    console.error(err);
  } finally {
    setLoading(false);
  }
}

async function handleSubmit(e) {
  e.preventDefault();
  setSaving(true);
  setMissatge(null);

  const formData = { ...form };
  if (editingId && !formData.password) delete formData.password;

  const url = editingId ? `/professors/${editingId}` :
  '/professors';
  const method = editingId ? 'PUT' : 'POST';

  try {
    const res = await apiFetch(url, { method, body:
JSON.stringify(formData) });
    if (!res.ok) {
      let errorMsg = await res.text();
      try {
        const data = JSON.parse(errorMsg);
        errorMsg = data.error || errorMsg;
      } catch { /* no fer res */ }
      throw new Error(errorMsg);
    }
    setMissatge({ tipus: 'ok', text: editingId ? 'Professor
actualitzat' : 'Professor creat' });
    setForm({ email: '', nom: '', rol: 'professor', password: '' });
    setEditingId(null);
    carregarProfessors();
  } catch (err) {
    setMissatge({ tipus: 'error', text: err.message });
  }
}

```

```

    } finally {
      setSaving(false);
    }
  }

function editar(prof) {
  setForm({
    email: prof.email,
    nom: prof.nom,
    rol: prof.rol,
    password: ''
  });
  setEditingId(prof.id);
}

async function eliminar(id, nom) {
  if (!confirm(`Eliminar "${nom}"?`)) return;
  try {
    const res = await apiFetch(`/professors/${id}`, { method:
'DELETE' });
    if (!res.ok) throw new Error(await res.text());
    setMissatge({ tipus: 'ok', text: `Professor "${nom}"
eliminat` });
    carregarProfessors();
  } catch (err) {
    setMissatge({ tipus: 'error', text: err.message });
  }
}

function cancelarEdicio() {
  setForm({ email: '', nom: '', rol: 'professor', password: '' });
  setEditingId(null);
}

async function handleCsvUpload(e) {
  const file = e.target.files[0];
  if (!file) return;

  const reader = new FileReader();
  reader.onload = async (ev) => {
    const text = ev.target.result;
    const lines = text.split('\n');
    if (lines.length < 2) return setMissatge({ tipus: 'error', text:
'CSV buit' });

    const headers = lines[0].split(',').map(h =>
h.trim().toLowerCase().replace(/"/g, ''));
    const idxEmail = headers.findIndex(h => h === 'email');
    const idxNom = headers.findIndex(h => h === 'nom');
    const idxRol = headers.findIndex(h => h === 'rol');
    if (idxEmail === -1 || idxNom === -1) {
      return setMissatge({ tipus: 'error', text: 'CSV ha de tenir

```

```

columnes "email" i "nom" }));
    }

    const professorsPerImportar = [];
    for (let i = 1; i < lines.length; i++) {
        if (!lines[i].trim()) continue;
        const vals = lines[i].split(',').map(v =>
v.trim().replace(/^\|"$/g, ''));
        const email = vals[idxEmail];
        const nom = vals[idxNom];
        const rol = vals[idxRol] || 'professor';
        if (email && nom) professorsPerImportar.push({ email, nom,
rol, password: 'temp123456' });
    }

    if (!professorsPerImportar.length) return setMissatge({ tipus:
'error', text: 'No hi ha dades vàlides' });

    setSaving(true);
    let creats = 0, errors = 0;
    const errorsList = [];
    for (const prof of professorsPerImportar) {
        try {
            const res = await apiFetch('/professors', { method: 'POST',
body: JSON.stringify(prof) });
            if (!res.ok) throw new Error(await res.text());
            creats++;
        } catch (err) {
            errors++;
            errorsList.push(`${prof.email}: ${err.message}`);
        }
    }
    setMissatge({ tipus: errors ? 'error' : 'ok', text: `Importats
${creats}. Errors: ${errors}${errorsList.length ? ' ' +
errorsList.slice(0,2).join('; ') : ''}` });
    carregatProfessors();
    setSaving(false);
};
reader.readAsText(file, 'UTF-8');
e.target.value = '';
}

function exportarCSV() {
    if (!professors.length) return setMissatge({ tipus: 'error', text:
'No hi ha professors per exportar' });
    const rows = [['email', 'nom', 'rol'].join(',')];
    professors.forEach(p =>
rows.push(`${p.email},${p.nom},${p.rol}`));
    const blob = new Blob([rows.join('\n')], { type: 'text/csv' });
    const a = document.createElement('a');
    a.href = URL.createObjectURL(blob);
    a.download = `professors_${new

```

```

Date().toISOString().slice(0,19).replace(/:/g, '-')).csv`;
    a.click();
    URL.revokeObjectURL(a.href);
    setMissatge({ tipus: 'ok', text: `Exportats ${professors.length}
professors` });
  }

  return (
    <div>
      <h2>Gestió de professors</h2>

      <div style={{ marginBottom: 16, display: 'flex', justifyContent:
'flex-end', gap: 8 }}>
        <button onClick={exportarCSV} style={{ background: '#17a2b8',
color: 'white', padding: '8px 16px', borderRadius: 6, border: 'none',
cursor: 'pointer' }}>📄 Exportar CSV</button>
        <label style={{ background: '#28a745', color: 'white',
padding: '8px 16px', borderRadius: 6, cursor: 'pointer' }}>
          Importar CSV
          <input type="file" accept=".csv" onChange={handleCsvUpload}
style={{ display: 'none' }} />
        </label>
      </div>

      <form onSubmit={handleSubmit} style={{ marginBottom: 20,
padding: 16, border: '1px solid #ccc', borderRadius: 8, background:
'#fff' }}>
        <h3>{editingId ? 'Editar professor' : 'Nou professor'}</h3>
        <div style={{ display: 'grid', gap: 12, gridTemplateColumns:
'repeat(2,1fr)' }}>
          <div>
            <label>Nom complet *</label>
            <input type="text" value={form.nom} onChange={e =>
setForm({...form, nom: e.target.value})} required style={{ width:
'100%', padding: '8px', border: '1px solid #ccc', borderRadius: 4 }}
/>
          </div>
          <div>
            <label>Correu electrònic *</label>
            <input type="email" value={form.email} onChange={e =>
setForm({...form, email: e.target.value})} required style={{ width:
'100%', padding: '8px', border: '1px solid #ccc', borderRadius: 4 }}
/>
          </div>
          <div>
            <label>Contrasenya {!editingId && '*'}</label>
            <input type="password" value={form.password} onChange={e
=> setForm({...form, password: e.target.value})} required={!editingId}
minLength={6} style={{ width: '100%', padding: '8px', border: '1px
solid #ccc', borderRadius: 4 }} />
            {editingId && <span style={{ fontSize: 11, color:

```

```

'#888' }}>(deixa buit per mantenir-la)</span>
</div>
<div>
  <label>Rol *</label>
  <select value={form.rol} onChange={e => setForm({...form,
rol: e.target.value})} style={{ width: '100%', padding: '8px', border:
'1px solid #ccc', borderRadius: 4 }}>
    <option value="professor">Professor</option>
    <option value="tutor">Tutor</option>
    <option value="admin">Administrador</option>
  </select>
</div>
</div>
<div style={{ marginTop: 16, display: 'flex', gap: 8 }}>
  <button type="submit" disabled={saving} style={{ background:
'#2d5be3', color: '#fff', border: 'none', padding: '8px 16px',
borderRadius: 4, cursor: 'pointer' }}>
    {saving ? 'Guardant...' : editingId ? 'Actualitzar
professor' : '+ Afegir professor'}
  </button>
  {editingId && <button type="button" onClick={cancelarEdicio}
style={{ background: '#f0eee8', border: 'none', padding: '8px 16px',
borderRadius: 4, cursor: 'pointer' }}>Cancel·lar</button>}
</div>
{missatge && <div style={{ marginTop: 12, padding: 10,
borderRadius: 4, background: missatge.tipus === 'ok' ? '#e8f5ee' :
'#fceaea', color: missatge.tipus === 'ok' ? '#1a7a4a' :
'#b83232' }}>{missatge.text}</div>}
</form>

<div style={{ background: '#fff', border: '1px solid #ddd',
borderRadius: 8 }}>
  <div style={{ padding: '12px 16px', borderBottom: '1px solid
#ddd' }}>
    <span>{loading ? 'Carregant...' : `${professors.length}
professors`}</span>
  </div>
  {!loading && professors.length === 0 && <p style={{ padding:
24, textAlign: 'center' }}>No hi ha professors.</p>}
  {professors.length > 0 && (
    <table style={{ width: '100%', borderCollapse:
'collapse' }}>
      <thead>
        <tr style={{ background: '#f5f4f0' }}>
          <th style={{ textAlign: 'left', padding:
12 }}>Nom</th>
          <th style={{ textAlign: 'left', padding:
12 }}>Email</th>
          <th style={{ textAlign: 'left', padding:
12 }}>Rol</th>
          <th style={{ textAlign: 'center', padding:
12 }}>Accions</th>

```

```

        </tr>
      </thead>
      <tbody>
        {professors.map(p => (
          <tr key={p.id} style={{ borderTop: '1px solid
#eee' }}>
            <td style={{ padding:
12 }}><strong>{p.nom}</strong></td>
            <td style={{ padding: 12 }}>{p.email}</td>
            <td style={{ padding: 12 }}>
              <span style={{ padding: '4px 8px', borderRadius:
20, fontSize: 12, background: p.rol === 'tutor' ? '#e0f0ff' : p.rol
=== 'admin' ? '#f0e0ff' : '#eee' }}>
                {p.rol === 'professor' ? 'Professor' : p.rol ===
'tutor' ? 'Tutor' : 'Administrador'}
              </span>
            </td>
            <td style={{ padding: 12, textAlign: 'center' }}>
              <button onClick={() => editar(p)}
style={{ background: 'none', border: 'none', color: '#2d5be3', cursor:
'pointer', marginRight: 12 }}>✎</button>
              <button onClick={() => eliminar(p.id, p.nom)}
style={{ background: 'none', border: 'none', color: '#b83232', cursor:
'pointer' }}>🗑️</button>
            </td>
          </tr>
        )))
      </tbody>
    </table>
  )}
</div>
</div>
);
}

```

src/components/GrupManagement.jsx

```

import { useState, useEffect } from 'react';
import { apiFetch } from '../api';

export default function GrupManagement() {
  const [grups, setGrups] = useState([]);
  const [professors, setProfessors] = useState([]);
  const [loading, setLoading] = useState(true);
  const [form, setForm] = useState({ nom: '', curs: '', professor_id:
'', llinar_assistencia: 80 });
  const [editingId, setEditingId] = useState(null);
  const [saving, setSaving] = useState(false);
  const [missatge, setMissatge] = useState(null);

```

```

useEffect(() => {
  carregarGrups();
  carregarProfessors();
}, []);

async function carregarGrups() {
  setLoading(true);
  const res = await apiFetch('/grups');
  const data = await res.json();
  if (res.ok) {
    setGrups(Array.isArray(data) ? data : []);
  } else {
    setMissatge({ tipus: 'error', text: data.error || 'Error
carregant grups' });
  }
  setLoading(false);
}

async function carregarProfessors() {
  const res = await apiFetch('/professors');
  const data = await res.json();
  if (res.ok) {
    setProfessors(Array.isArray(data) ? data : []);
  } else {
    console.error('Error carregant professors:', data.error);
  }
}

async function handleSubmit(e) {
  e.preventDefault();
  setSaving(true);
  setMissatge(null);

  const url = editingId ? `/grups/${editingId}` : '/grups';
  const method = editingId ? 'PUT' : 'POST';

  const res = await apiFetch(url, {
    method,
    body: JSON.stringify(form)
  });
  const data = await res.json();

  if (!res.ok) {
    setMissatge({ tipus: 'error', text: data.error });
  } else {
    setMissatge({ tipus: 'ok', text: editingId ? 'Grup actualitzat
correctament' : 'Grup creat correctament' });
    setForm({ nom: '', curs: '', professor_id: '',
lindar_assistencia: 80 });
    setEditingId(null);
    carregarGrups();
  }
}

```

```

    setSaving(false);
  }

  function editar(grup) {
    setForm({
      nom: grup.nom,
      curs: grup.curs,
      professor_id: grup.professor_id || '',
      llindar_assistencia: grup.llindar_assistencia
    });
    setEditingId(grup.id);
  }

  async function eliminar(id, nom) {
    if (!confirm(`Segur que vols eliminar el grup "${nom}"?`)) return;
    const res = await apiFetch(`/grups/${id}`, { method: 'DELETE' });
    if (res.ok) {
      setMissatge({ tipus: 'ok', text: `Grup "${nom}" eliminat` });
      carregarGrups();
    } else {
      const data = await res.json();
      setMissatge({ tipus: 'error', text: data.error || 'Error en
eliminar el grup' });
    }
  }

  function cancelarEdicio() {
    setForm({ nom: '', curs: '', professor_id: '',
llindar_assistencia: 80 });
    setEditingId(null);
    setMissatge(null);
  }

  return (
    <div>
      <h2>Gestió de grups</h2>

      <form onSubmit={handleSubmit} style={{ marginBottom: 20,
padding: 16, border: '1px solid #ccc', borderRadius: 8, background:
'#fff' }}>
        <h3 style={{ marginBottom: 16, fontSize: 16, color:
'#1a1a18' }}>{editingId ? 'Editar grup' : 'Nou grup'}</h3>
        <div style={{ display: 'grid', gap: 12, gridTemplateColumns:
'repeat(2, 1fr)' }}>
          <div>
            <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Nom del grup *</label>
            <input
              type="text"
              placeholder="1r ESO A"
              value={form.nom}
              onChange={e => setForm({ ...form, nom:

```



```

e.target.value }}}
      required
      style={{ width: '100%', border: '1px solid #e0ddd5',
borderRadius: 6, padding: '7px 10px', fontSize: 13, background:
'#f5f4f0', color: '#1a1a18', boxSizing: 'border-box' }}
    />
  </div>
  <div>
    <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Curs *</label>
    <input
      type="text"
      placeholder="1r ES0"
      value={form.curs}
      onChange={e => setForm({ ...form, curs:
e.target.value }}}
      required
      style={{ width: '100%', border: '1px solid #e0ddd5',
borderRadius: 6, padding: '7px 10px', fontSize: 13, background:
'#f5f4f0', color: '#1a1a18', boxSizing: 'border-box' }}
    />
  </div>
  <div>
    <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Tutor del grup</label>
    <select
      value={form.professor_id}
      onChange={e => setForm({ ...form, professor_id:
e.target.value }}}
      style={{ width: '100%', border: '1px solid #e0ddd5',
borderRadius: 6, padding: '7px 10px', fontSize: 13, background:
'#f5f4f0', color: '#1a1a18', boxSizing: 'border-box' }}
    >
      <option value="">Sense tutor assignat</option>
      {professors.map(p => <option key={p.id}
value={p.id}>{p.nom}</option>)}
    </select>
  </div>
  <div>
    <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Llindar d'assistència
(%)</label>
    <input
      type="number"
      placeholder="80"
      value={form.llindar_assistencia}
      onChange={e => setForm({ ...form, llindar_assistencia:
parseInt(e.target.value) || 0 }}}
      min="0"
      max="100"
      style={{ width: '100%', border: '1px solid #e0ddd5',
borderRadius: 6, padding: '7px 10px', fontSize: 13, background:

```

```

'#f5f4f0', color: '#1a1a18', boxSizing: 'border-box' }}
    />
  </div>
</div>
<div style={{ marginTop: 16, display: 'flex', gap: 8 }}>
  <button
    type="submit"
    disabled={saving}
    style={{ background: '#2d5be3', color: '#fff', border:
'none', borderRadius: 6, padding: '8px 18px', fontSize: 13,
fontWeight: 500, cursor: 'pointer' }}
  >
    {saving ? 'Guardant...' : editingId ? 'Actualitzar grup' :
'+ Afegir grup'}
  </button>
  {editingId && (
    <button
      type="button"
      onClick={cancelarEdicio}
      style={{ background: '#f0eee8', border: 'none',
borderRadius: 6, padding: '8px 18px', fontSize: 13, cursor: 'pointer',
color: '#1a1a18' }}
    >
      Cancel·lar
    </button>
  )}
</div>
{missatge && (
  <div style={{
    marginTop: 12,
    padding: 10,
    borderRadius: 6,
    background: missatge.tipus === 'ok' ? '#e8f5ee' :
'#fceaea',
    color: missatge.tipus === 'ok' ? '#1a7a4a' : '#b83232',
    border: `1px solid ${missatge.tipus === 'ok' ? '#9fe1cb' :
'#e8a0a0'}`,
    fontSize: 13
  }}>
    {missatge.text}
  </div>
)}
</form>

<div style={{ background: '#fff', border: '1px solid #e0ddd5',
borderRadius: 10, overflow: 'hidden' }}>
  <div style={{ padding: '12px 16px', borderBottom: '1px solid
#e0ddd5' }}>
    <span style={{ fontSize: 13, fontWeight: 500, color:
'#6b6a64' }}>
      {loading ? 'Carregant...' : `${grups.length}
${grups.length === 1 ? 'grup' : 'grups'}}`
    </span>
  </div>
</div>

```

```

    </span>
  </div>
  {!loading && grups.length === 0 && (
    <p style={{ padding: 24, textAlign: 'center', color:
'#a8a79f', fontSize: 13 }}>
      Encara no hi ha grups. Utilitza el formulari per crear-ne
un.
    </p>
  )}
  {grups.length > 0 && (
    <table style={{ width: '100%', borderCollapse: 'collapse',
fontSize: 13 }}>
      <thead>
        <tr style={{ background: '#f5f4f0' }}>
          <th style={{ textAlign: 'left', padding: '12px 12px',
color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Nom</th>
          <th style={{ textAlign: 'left', padding: '12px 12px',
color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Curs</th>
          <th style={{ textAlign: 'left', padding: '12px 12px',
color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Tutor</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Llindar</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Accions</th>
        </tr>
      </thead>
      <tbody>
        {grups.map(g => {
          let llindarColor = '#e8f5ee';
          let llindarTextColor = '#1a7a4a';
          if (g.llindar_assistencia < 80 &&
g.llindar_assistencia >= 50) {
            llindarColor = '#fff3e0';
            llindarTextColor = '#b86c00';
          } else if (g.llindar_assistencia < 50) {
            llindarColor = '#fceaea';
            llindarTextColor = '#b83232';
          }
          return (
            <tr key={g.id} style={{ borderTop: '1px solid
#e0ddd5' }}>
              <td style={{ padding: '12px 12px', color:
'#1a1a18' }}><strong>{g.nom}</strong></td>
              <td style={{ padding: '12px 12px', color:
'#1a1a18' }}>{g.curs}</td>
              <td style={{ padding: '12px 12px', color:
'#1a1a18' }}>{g.professors?.nom || '-'}</td>

```

```

        <td style={{ padding: '12px 12px', textAlign:
'center' }}>
            <span style={{
                padding: '4px 10px',
                borderRadius: 20,
                fontSize: 11,
                fontWeight: 500,
                background: lllindarColor,
                color: lllindarTextColor
            }}>
                {g.lllindar_assistencia}%
            </span>
        </td>
        <td style={{ padding: '12px 12px', textAlign:
'center' }}>
            <button onClick={() => editar(g)}
style={{ background: 'none', border: 'none', color: '#2d5be3', cursor:
'pointer', fontSize: 16, marginRight: 12 }} title="Editar">✎</button>
            <button onClick={() => eliminar(g.id, g.nom)}
style={{ background: 'none', border: 'none', color: '#b83232', cursor:
'pointer', fontSize: 16 }} title="Eliminar">🗑️</button>
        </td>
    </tr>
</tbody>
</table>
    </div>
</div>
);
}

```

src/components/MateriaManagement.jsx

```

import { useState, useEffect } from 'react';
import { apiFetch } from '../api';

export default function MateriaManagement() {
    const [materies, setMateries] = useState([]);
    const [loading, setLoading] = useState(true);
    const [saving, setSaving] = useState(false);
    const [form, setForm] = useState({ nom: '', descripcio: '' });
    const [editingId, setEditingId] = useState(null);
    const [missatge, setMissatge] = useState(null);

    useEffect(() => {
        carregarMateries();
    }, []);
}

```

```

async function carregarMateries() {
  setLoading(true);
  const res = await apiFetch('/materies');
  const data = await res.json();
  if (res.ok) {
    setMateries(Array.isArray(data) ? data : []);
  } else {
    setMissatge({ tipus: 'error', text: data.error || 'Error
carregant matèries' });
  }
  setLoading(false);
}

async function handleSubmit(e) {
  e.preventDefault();
  setSaving(true);
  setMissatge(null);

  const url = editingId ? `/materies/${editingId}` : '/materies';
  const method = editingId ? 'PUT' : 'POST';

  const res = await apiFetch(url, {
    method,
    body: JSON.stringify(form),
  });

  const data = await res.json();
  if (!res.ok) {
    setMissatge({ tipus: 'error', text: data.error });
  } else {
    setMissatge({ tipus: 'ok', text: editingId ? 'Matèria
actualitzada correctament' : 'Matèria creada correctament' });
    setForm({ nom: '', descripcio: '' });
    setEditingId(null);
    carregarMateries();
  }
  setSaving(false);
}

function editar(materia) {
  setForm({
    nom: materia.nom,
    descripcio: materia.descripcio || '',
  });
  setEditingId(materia.id);
}

async function eliminar(id, nom) {
  if (!confirm(`Segur que vols eliminar la matèria "${nom}"?`))
return;
  const res = await apiFetch(`/materies/${id}`, { method:
'DELETE' });

```

```

    if (res.ok) {
      setMissatge({ tipus: 'ok', text: `Matèria "${nom}"
eliminada` });
      carregarMateries();
    } else {
      const data = await res.json();
      setMissatge({ tipus: 'error', text: data.error || 'Error en
eliminar la matèria' });
    }
  }

function cancel·larEdicio() {
  setForm({ nom: '', descripcio: '' });
  setEditingId(null);
  setMissatge(null);
}

return (
  <div>
    <h2>Gestió de matèries</h2>

    <form onSubmit={handleSubmit} style={{ marginBottom: 20,
padding: 16, border: '1px solid #ccc', borderRadius: 8, background:
'#fff' }}>
      <h3 style={{ marginBottom: 16, fontSize: 16, color:
'#1a1a18' }}>{editingId ? 'Editar matèria' : 'Nova matèria'}</h3>
      <div style={{ display: 'grid', gap: 12, gridTemplateColumns:
'repeat(2, 1fr)' }}>
        <div>
          <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Nom de la matèria
*</label>
          <input
            type="text"
            placeholder="Ex: Matemàtiques"
            value={form.nom}
            onChange={e => setForm({ ...form, nom:
e.target.value })}
            required
            style={{ width: '100%', border: '1px solid #e0ddd5',
borderRadius: 6, padding: '7px 10px', fontSize: 13, background:
'#f5f4f0', color: '#1a1a18', boxSizing: 'border-box' }}
          />
        </div>
        <div>
          <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Descripció
(opcional)</label>
          <input
            type="text"
            placeholder="Descripció breu"
            value={form.descripcio}

```

```

        onChange={e => setForm({ ...form, descripcio:
e.target.value })}
        style={{ width: '100%', border: '1px solid #e0ddd5',
borderRadius: 6, padding: '7px 10px', fontSize: 13, background:
'#f5f4f0', color: '#1a1a18', boxSizing: 'border-box' }}
      />
    </div>
  </div>
  <div style={{ marginTop: 16, display: 'flex', gap: 8 }}>
    <button
      type="submit"
      disabled={saving}
      style={{ background: '#2d5be3', color: '#fff', border:
'none', borderRadius: 6, padding: '8px 18px', fontSize: 13,
fontWeight: 500, cursor: 'pointer' }}
    >
      {saving ? 'Guardant...' : editingId ? 'Actualitzar
matèria' : '+ Afegir matèria'}
    </button>
    {editingId && (
      <button
        type="button"
        onClick={cancelarEdicio}
        style={{ background: '#f0eee8', border: 'none',
borderRadius: 6, padding: '8px 18px', fontSize: 13, cursor: 'pointer',
color: '#1a1a18' }}
      >
        Cancel·lar
      </button>
    )}
  </div>
  {missatge && (
    <div style={{
      marginTop: 12,
      padding: 10,
      borderRadius: 6,
      background: missatge.tipus === 'ok' ? '#e8f5ee' :
'#fceaea',
      color: missatge.tipus === 'ok' ? '#1a7a4a' : '#b83232',
      border: `1px solid ${missatge.tipus === 'ok' ? '#9fe1cb' :
'#e8a0a0'}`,
      fontSize: 13
    }}>
      {missatge.text}
    </div>
  )}
</form>

  <div style={{ background: '#fff', border: '1px solid #e0ddd5',
borderRadius: 10, overflow: 'hidden' }}>
    <div style={{ padding: '12px 16px', borderBottom: '1px solid
#e0ddd5' }}>

```

```

        <span style={{ fontSize: 13, fontWeight: 500, color:
'#6b6a64' }}>
            {loading ? 'Carregant...' : `${materies.length}
${materies.length === 1 ? 'matèria' : 'matèries'}}`
        </span>
    </div>
    {!!loading && materies.length === 0 && (
        <p style={{ padding: 24, textAlign: 'center', color:
'#a8a79f', fontSize: 13 }}>
            Encara no hi ha matèries. Utilitza el formulari per crear-
ne una.
        </p>
    )}
    {materies.length > 0 && (
        <div style={{ overflowX: 'auto' }}>
            <table style={{ width: '100%', borderCollapse: 'collapse',
fontSize: 13 }}>
                <thead>
                    <tr style={{ background: '#f5f4f0' }}>
                        <th style={{ textAlign: 'left', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Nom</th>
                        <th style={{ textAlign: 'left', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Descripció</th>
                        <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Accions</th>
                    </tr>
                </thead>
                <tbody>
                    {materies.map(m => (
                        <tr key={m.id} style={{ borderTop: '1px solid
#e0ddd5' }}>
                            <td style={{ padding: '12px 12px', color:
'#1a1a18' }}><strong>{m.nom}</strong></td>
                            <td style={{ padding: '12px 12px', color:
'#1a1a18' }}>{m.descripcio || '-'}</td>
                            <td style={{ padding: '12px 12px', textAlign:
'center' }}>
                                <button onClick={() => editar(m)}
style={{ background: 'none', border: 'none', color: '#2d5be3', cursor:
'pointer', fontSize: 16, marginRight: 12 }} title="Editar">✎</button>
                                <button onClick={() => eliminar(m.id, m.nom)}
style={{ background: 'none', border: 'none', color: '#b83232', cursor:
'pointer', fontSize: 16 }} title="Eliminar">🗑️</button>
                            </td>
                        </tr>
                    ))}
                </tbody>
            </table>

```



```

        </div>
      )}
    </div>
  </div>
);
}

```

src/components/AlumneManagement.jsx

```

import { useState, useEffect } from 'react';
import { apiFetch } from '../api';

export default function AlumneManagement() {
  const [alumnes, setAlumnes] = useState([]);
  const [grups, setGrups] = useState([]);
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [form, setForm] = useState({ nom: '', cognoms: '', email: '',
  grup_id: '', actiu: true });
  const [editingId, setEditingId] = useState(null);
  const [missatge, setMissatge] = useState(null);

  useEffect(() => {
    carregarAlumnes();
    carregarGrups();
  }, []);

  async function carregarAlumnes() {
    setLoading(true);
    const res = await apiFetch('/alumnes');
    const data = await res.json();
    if (res.ok) {
      setAlumnes(Array.isArray(data) ? data : []);
    } else {
      setMissatge({ tipus: 'error', text: data.error || 'Error
carregant alumnes' });
    }
    setLoading(false);
  }

  async function carregarGrups() {
    const res = await apiFetch('/grups');
    const data = await res.json();
    if (res.ok) {
      setGrups(Array.isArray(data) ? data : []);
    }
  }

  async function handleSubmit(e) {
    e.preventDefault();
  }

```

```

    setSaving(true);
    setMissatge(null);

    const url = editingId ? `/alumnes/${editingId}` : '/alumnes';
    const method = editingId ? 'PUT' : 'POST';

    const res = await apiFetch(url, {
      method,
      body: JSON.stringify(form),
    });

    const data = await res.json();
    if (!res.ok) {
      setMissatge({ tipus: 'error', text: data.error });
    } else {
      setMissatge({ tipus: 'ok', text: editingId ? 'Alumne actualitzat correctament' : 'Alumne creat correctament' });
      setForm({ nom: '', cognoms: '', email: '', grup_id: '', actiu: true });
      setEditingId(null);
      carregarAlumnes();
    }
    setSaving(false);
  }

  function editar(alumne) {
    setForm({
      nom: alumne.nom,
      cognoms: alumne.cognoms,
      email: alumne.email || '',
      grup_id: alumne.grup_id || '',
      actiu: alumne.actiu,
    });
    setEditingId(alumne.id);
  }

  async function eliminar(id, nomCompleto) {
    if (!confirm(`Segur que vols eliminar l'alumne "${nomCompleto}"?`))
      return;
    const res = await apiFetch(`/alumnes/${id}`, { method: 'DELETE' });
    if (res.ok) {
      setMissatge({ tipus: 'ok', text: `Alumne "${nomCompleto}" eliminat` });
      carregarAlumnes();
    } else {
      const data = await res.json();
      setMissatge({ tipus: 'error', text: data.error || 'Error en eliminar l\'alumne' });
    }
  }

```

```

function cancelarEdicio() {
    setForm({ nom: '', cognoms: '', email: '', grup_id: '', actiu:
true });
    setEditingId(null);
    setMissatge(null);
}

async function handleCsvUpload(e) {
    const file = e.target.files[0];
    if (!file) return;

    const reader = new FileReader();
    reader.onload = async (event) => {
        const text = event.target.result;
        const lines = text.split('\n');
        if (lines.length === 0) {
            setMissatge({ tipus: 'error', text: 'El fitxer està buit' });
            return;
        }

        const headers = lines[0].split(',').map(h =>
h.trim().toLowerCase().replace(/"/g, ''));

        const nomIdx = headers.findIndex(h => h === 'nom');
        const cognomsIdx = headers.findIndex(h => h === 'cognoms');
        const emailIdx = headers.findIndex(h => h === 'email');

        if (nomIdx === -1 || cognomsIdx === -1) {
            setMissatge({ tipus: 'error', text: 'El CSV ha de tenir
columnes "nom" i "cognoms"' });
            return;
        }

        const alumnesPerImportar = [];
        for (let i = 1; i < lines.length; i++) {
            if (!lines[i].trim()) continue;

            const values = lines[i].split(',').map(v =>
v.trim().replace(/^"|"$/g, ''));
            const nom = values[nomIdx];
            const cognoms = values[cognomsIdx];
            const email = emailIdx !== -1 ? values[emailIdx] : '';

            if (nom && cognoms) {
                alumnesPerImportar.push({
                    nom,
                    cognoms,
                    email: email || '',
                    grup_id: null,
                    actiu: true
                });
            }
        }
    }
}

```

```

    }

    if (alumnesPerImportar.length === 0) {
        setMissatge({ tipus: 'error', text: 'No s\'han trobat dades
vàlides al CSV' });
        return;
    }

    setSaving(true);
    let creats = 0;
    let errors = 0;
    const errorsList = [];

    for (const alumne of alumnesPerImportar) {
        const res = await apiFetch('/alumnes', {
            method: 'POST',
            body: JSON.stringify(alumne)
        });
        const data = await res.json();
        if (res.ok) {
            creats++;
        } else {
            errors++;
            errorsList.push(`${alumne.nom} ${alumne.cognoms}:
${data.error}`);
        }
    }

    if (errors > 0) {
        setMissatge({ tipus: 'error', text: `Importats ${creats}
alumnes. Errors: ${errors}. ${errorsList.slice(0, 3).join('; ')} ` });
    } else {
        setMissatge({ tipus: 'ok', text: `Importats ${creats} alumnes
correctament!` });
    }
    carregarAlumnes();
    setSaving(false);
};
reader.readAsText(file, 'UTF-8');
e.target.value = '';
}

function exportarCSV() {
    if (alumnes.length === 0) {
        setMissatge({ tipus: 'error', text: 'No hi ha alumnes per
exportar.' });
        return;
    }

    const headers = ['nom', 'cognoms', 'email'];
    const csvRows = [headers.join(',')];

```

```

    alumnes.forEach(alumne => {
      const row = [
        `${alumne.nom}`,
        `${alumne.cognoms}`,
        `${alumne.email || ''}`
      ];
      csvRows.push(row.join(','));
    });

    const blob = new Blob([csvRows.join('\n')], { type:
'text/csv;charset=utf-8;' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `alumnes_${new Date().toISOString().slice(0,
19).replace(/:/g, '-')}.csv`;
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);
    URL.revokeObjectURL(url);

    setMissatge({ tipus: 'ok', text: `Exportats ${alumnes.length}
alumnes.` });
  }

  return (
    <div>
      <h2>Gestió d'alumnes</h2>

      <div style={{ marginBottom: 16, display: 'flex', justifyContent:
'flex-end', gap: 8 }}>
        <button
          onClick={exportarCSV}
          style={{ background: '#17a2b8', color: 'white', padding:
'8px 16px', borderRadius: 6, cursor: 'pointer', fontSize: 13, border:
'none' }}
          title="Exportar tots els alumnes a CSV"
        >
          📄 Exportar CSV
        </button>
        <label style={{ background: '#28a745', color: 'white',
padding: '8px 16px', borderRadius: 6, cursor: 'pointer', fontSize:
13 }}>
          Importar CSV
          <input type="file" accept=".csv" onChange={handleCsvUpload}
style={{ display: 'none' }} />
        </label>
      </div>

      <form onSubmit={handleSubmit} style={{ marginBottom: 20,
padding: 16, border: '1px solid #ccc', borderRadius: 8, background:

```

```

'#fff' }}>
    <h3 style={{ marginBottom: 16, fontSize: 16, color:
'#1a1a18' }}>{editingId ? 'Editar alumne' : 'Nou alumne'}</h3>
    <div style={{ display: 'grid', gap: 12, gridTemplateColumns:
'repeat(2, 1fr)' }}>
        <div>
            <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Nom *</label>
            <input
                type="text"
                placeholder="Nom"
                value={form.nom}
                onChange={e => setForm({ ...form, nom:
e.target.value })}
                required
                style={{ width: '100%', border: '1px solid #e0ddd5',
borderRadius: 6, padding: '7px 10px', fontSize: 13, background:
'#f5f4f0', color: '#1a1a18', boxSizing: 'border-box' }}
            />
        </div>
        <div>
            <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Cognoms *</label>
            <input
                type="text"
                placeholder="Cognoms"
                value={form.cognoms}
                onChange={e => setForm({ ...form, cognoms:
e.target.value })}
                required
                style={{ width: '100%', border: '1px solid #e0ddd5',
borderRadius: 6, padding: '7px 10px', fontSize: 13, background:
'#f5f4f0', color: '#1a1a18', boxSizing: 'border-box' }}
            />
        </div>
        <div>
            <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Email</label>
            <input
                type="email"
                placeholder="alumne@exemple.cat"
                value={form.email}
                onChange={e => setForm({ ...form, email:
e.target.value })}
                style={{ width: '100%', border: '1px solid #e0ddd5',
borderRadius: 6, padding: '7px 10px', fontSize: 13, background:
'#f5f4f0', color: '#1a1a18', boxSizing: 'border-box' }}
            />
        </div>
        <div>
            <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Grup</label>

```

```

        <select
          value={form.grup_id}
          onChange={e => setForm({ ...form, grup_id:
e.target.value })}
          style={{ width: '100%', border: '1px solid #e0ddd5',
borderRadius: 6, padding: '7px 10px', fontSize: 13, background:
'#f5f4f0', color: '#1a1a18', boxSizing: 'border-box' }}
        >
          <option value="">Sense grup</option>
          {groups.map(g => <option key={g.id}
value={g.id}>{g.nom}</option>)}
        </select>
      </div>
      <div style={{ display: 'flex', alignItems: 'center' }}>
        <label style={{ display: 'flex', alignItems: 'center',
gap: 8, fontSize: 13, color: '#1a1a18' }}>
          <input
            type="checkbox"
            checked={form.actiu}
            onChange={e => setForm({ ...form, actiu:
e.target.checked })}
            style={{ width: 18, height: 18 }}
          />
          Alumne actiu
        </label>
      </div>
    </div>
    <div style={{ marginTop: 16, display: 'flex', gap: 8 }}>
      <button
        type="submit"
        disabled={saving}
        style={{ background: '#2d5be3', color: '#fff', border:
'none', borderRadius: 6, padding: '8px 18px', fontSize: 13,
fontWeight: 500, cursor: 'pointer' }}
      >
        {saving ? 'Guardant...' : editingId ? 'Actualitzar
alumne' : '+ Afegir alumne'}
      </button>
      {editingId && (
        <button
          type="button"
          onClick={cancelarEdicio}
          style={{ background: '#f0eee8', border: 'none',
borderRadius: 6, padding: '8px 18px', fontSize: 13, cursor: 'pointer',
color: '#1a1a18' }}
        >
          Cancel·lar
        </button>
      )}
    </div>
    {missatge && (
      <div style={{

```

```

        marginTop: 12,
        padding: 10,
        borderRadius: 6,
        background: missatge.tipus === 'ok' ? '#e8f5ee' :
'#fceaea',
        color: missatge.tipus === 'ok' ? '#1a7a4a' : '#b83232',
        border: `1px solid ${missatge.tipus === 'ok' ? '#9fe1cb' :
'#e8a0a0'}`,
        fontSize: 13
    }}>
    {missatge.text}
</div>
  )}
</form>

<div style={{ background: '#fff', border: '1px solid #e0ddd5',
borderRadius: 10, overflow: 'hidden' }}>
  <div style={{ padding: '12px 16px', borderBottom: '1px solid
#e0ddd5' }}>
    <span style={{ fontSize: 13, fontWeight: 500, color:
'#6b6a64' }}>
      {loading ? 'Carregant...' : `${alumnes.length}
${alumnes.length === 1 ? 'alumne' : 'alumnes'}`}
    </span>
  </div>
  {!loading && alumnes.length === 0 && (
    <p style={{ padding: 24, textAlign: 'center', color:
'#a8a79f', fontSize: 13 }}>
      Encara no hi ha alumnes. Utilitza el formulari per crear-
ne un.
    </p>
  )}
  {alumnes.length > 0 && (
    <table style={{ width: '100%', borderCollapse: 'collapse',
fontSize: 13 }}>
      <thead>
        <tr style={{ background: '#f5f4f0' }}>
          <th style={{ textAlign: 'left', padding: '12px 12px',
color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Nom complet</th>
          <th style={{ textAlign: 'left', padding: '12px 12px',
color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Email</th>
          <th style={{ textAlign: 'left', padding: '12px 12px',
color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Grup</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Actiu</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, textTransform: 'uppercase',
letterSpacing: '.5px', fontSize: 11 }}>Accions</th>

```



```

    </tr>
  </thead>
  <tbody>
    {alumnos.map(a => {
      let estatColor = '#e8f5ee';
      let estatTextColor = '#1a7a4a';
      let estatText = '✓ Actiu';
      if (!a.actiu) {
        estatColor = '#fceaea';
        estatTextColor = '#b83232';
        estatText = '✗ Inactiu';
      }
      return (
        <tr key={a.id} style={{ borderTop: '1px solid
#e0ddd5' }}>
          <td style={{ padding: '12px 12px', color:
'#1a1a18' }}><strong>{a.nom} {a.cognoms}</strong></td>
          <td style={{ padding: '12px 12px', color:
'#1a1a18' }}>{a.email || '-'}</td>
          <td style={{ padding: '12px 12px', color:
'#1a1a18' }}>{a.grups?.nom || '-'}</td>
          <td style={{ padding: '12px 12px', textAlign:
'center' }}>
            <span style={{
              padding: '4px 10px',
              borderRadius: 20,
              fontSize: 11,
              fontWeight: 500,
              background: estatColor,
              color: estatTextColor
            }}>
              {estatText}
            </span>
          </td>
          <td style={{ padding: '12px 12px', textAlign:
'center' }}>
            <button onClick={() => editar(a)}
style={{ background: 'none', border: 'none', color: '#2d5be3', cursor:
'pointer', fontSize: 16, marginRight: 12 }} title="Editar">⚙️</button>
            <button onClick={() => eliminar(a.id, `${a.nom}
${a.cognoms}`)} style={{ background: 'none', border: 'none', color:
'#b83232', cursor: 'pointer', fontSize: 16 }}
title="Eliminar">🗑️</button>
          </td>
        </tr>
      );
    })}
  </tbody>
</table>
)}
</div>

```

```

    </div>
  );
}

```

src/components/ConfiguracioPrincipal.jsx

```

// src/components/ConfiguracioPrincipal.jsx
import { useState } from 'react';
import ProfessorManagement from '../ProfessorManagement';
import GrupManagement from '../GrupManagement';
import MateriaManagement from '../MateriaManagement';
import HorariPerDia from '../HorariPerDia';
import DiesNoLectiusManagement from '../DiesNoLectiusManagement';

export default function ConfiguracioPrincipal() {
  const [tab, setTab] = useState('professors');

  const tabs = [
    { id: 'professors', nom: 'Professors', component:
<ProfessorManagement /> },
    { id: 'grups', nom: 'Grups', component: <GrupManagement /> },
    { id: 'materies', nom: '📖 Matèries', component: <MateriaManagement
/> },
    { id: 'horaris', nom: 'Horari (Disseny)', component:
<HorariPerDia /> },
    { id: 'dies_no_lectius', nom: 'Dies no lectius', component:
<DiesNoLectiusManagement /> },
  ];

  const activeTab = tabs.find(t => t.id === tab);

  return (
    <div>
      <h2> Configuració</h2>
      <div style={{
        display: 'flex',
        flexWrap: 'wrap',
        gap: 4,
        borderBottom: '1px solid #e0ddd5',
        marginBottom: 20,
        background: '#fff',
        borderRadius: '10px 10px 0 0',
        padding: '0 4px'
      }}>
        {tabs.map(t => (
          <button
            key={t.id}
            onClick={() => setTab(t.id)}
            style={{
              padding: '10px 20px',

```

```

        fontSize: 13,
        fontWeight: tab === t.id ? 600 : 400,
        background: tab === t.id ? '#2d5be3' : 'transparent',
        color: tab === t.id ? '#fff' : '#6b6a64',
        border: 'none',
        borderRadius: '8px 8px 0 0',
        cursor: 'pointer',
        transition: 'all 0.2s'
      }}
    >
      {t.nom}
    </button>
  )}}
</div>
<div style={{
  background: '#fff',
  border: '1px solid #e0ddd5',
  borderRadius: 10,
  padding: 20,
  borderTopLeftRadius: 0
}}>
  {activeTab.component}
</div>
</div>
);
}

```

src/components/HorariPerDia.jsx

```

import { useState, useEffect } from 'react';
import { apiFetch } from '../api';

const dies = {
  1: 'Dilluns', 2: 'Dimarts', 3: 'Dimecres', 4: 'Dijous', 5:
'Divendres', 6: 'Dissabte', 7: 'Diumenge'
};

export default function HorariPerDia() {
  const [grups, setGrups] = useState([]);
  const [materies, setMateries] = useState([]);
  const [grupId, setGrupId] = useState('');
  const [diaSetmana, setDiaSetmana] = useState(1);
  const [franges, setFranges] = useState([]);
  const [loading, setLoading] = useState(false);
  const [saving, setSaving] = useState(false);
  const [missatge, setMissatge] = useState(null);

  useEffect(() => {
    carregarGrups();
    carregarMateries();
  }, []);

```

```

}, []);

async function carregarGrups() {
  const res = await apiFetch('/grups');
  const data = await res.json();
  if (res.ok) {
    setGrups(Array.isArray(data) ? data : []);
  } else {
    setMissatge({ tipus: 'error', text: data.error || 'Error
carregant grups' });
  }
}

async function carregarMateries() {
  const res = await apiFetch('/materies');
  const data = await res.json();
  if (res.ok) {
    setMateries(Array.isArray(data) ? data : []);
  } else {
    setMissatge({ tipus: 'error', text: data.error || 'Error
carregant matèries' });
  }
}

async function carregarFranges() {
  if (!grupId || !diaSetmana) return;
  setLoading(true);
  setMissatge(null);
  try {
    const res = await apiFetch(`/horaris/grup-
dia?grup_id=${grupId}&dia_setmana=${diaSetmana}`);
    const data = await res.json();
    if (!res.ok) throw new Error(data.error);
    const frangesForm = data.map(f => ({
      id: f.id,
      hora_inici: f.hora_inici.slice(0,5),
      durada_min: f.durada_min,
      materia_id: f.materia_id
    }));
    setFranges(frangesForm);
  } catch (err) {
    setMissatge({ tipus: 'error', text: err.message });
  } finally {
    setLoading(false);
  }
}

function afegirFranja() {
  setFranges([...franges, { hora_inici: '09:00', durada_min: 60,
materia_id: '' }]);
}

```

```

function eliminarFranja(index) {
  const noves = [...franges];
  noves.splice(index, 1);
  setFranges(noves);
}

function updateFranja(index, camp, valor) {
  const noves = [...franges];
  noves[index][camp] = valor;
  setFranges(noves);
}

async function guardarFranges() {
  for (let i = 0; i < franges.length; i++) {
    const f = franges[i];
    if (!f.materia_id) {
      setMissatge({ tipus: 'error', text: `Franja ${i+1}: has de
seleccionar una matèria.` });
      return;
    }
    if (!f.hora_inici || f.durada_min < 1) {
      setMissatge({ tipus: 'error', text: `Franja ${i+1}: hora o
durada invàlida.` });
      return;
    }
  }

  setSaving(true);
  setMissatge(null);
  try {
    const res = await apiFetch('/horaris/grup-dia', {
      method: 'POST',
      body: JSON.stringify({
        grup_id: grupId,
        dia_setmana: diaSetmana,
        franges: franges.map(({ hora_inici, durada_min,
materia_id }) => ({
          hora_inici,
          durada_min,
          materia_id
        })))
      });
    const result = await res.json();
    if (!res.ok) throw new Error(result.error);
    setMissatge({ tipus: 'ok', text: 'Horari desat
correctament!' });
    carregarFranges();
  } catch (err) {
    setMissatge({ tipus: 'error', text: err.message });
  } finally {
    setSaving(false);
  }
}

```

```

    }
  }

  return (
    <div>
      <h2>Gestió ràpida d'horaris per dia</h2>

      <div style={{ marginBottom: 20, padding: 16, border: '1px solid
#ccc', borderRadius: 8, display: 'flex', gap: 12, alignItems: 'flex-
end', flexWrap: 'wrap' }}>
        <div>
          <label>Grup</label>
          <select value={grupId} onChange={e =>
setGrupId(e.target.value)} style={{ padding: 8, marginLeft: 8 }}>
            <option value="">-- Selecciona --</option>
            {grups.map(g => <option key={g.id}
value={g.id}>{g.nom}</option>)}
          </select>
        </div>
        <div>
          <label>Dia de la setmana</label>
          <select value={diaSetmana} onChange={e =>
setDiaSetmana(parseInt(e.target.value))} style={{ padding: 8,
marginLeft: 8 }}>
            {Object.entries(dies).map(([num, nom]) => (
              <option key={num} value={num}>{nom}</option>
            ))}
          </select>
        </div>
        <button onClick={carregarFranges} disabled={!grupId}>Carregar
franges actuals</button>
      </div>

      {loading && <p>Carregant...</p>}
      {missatge && <div style={{ marginBottom: 12, padding: 8,
borderRadius: 4, background: missatge.tipus === 'ok' ? '#d4edda' :
'#f8d7da', color: missatge.tipus === 'ok' ? '#155724' :
'#721c24' }}>{missatge.text}</div>}

      <div style={{ marginBottom: 16 }}>
        <button onClick={afegirFranja} style={{ background: '#28a745',
color: 'white' }}>+ Afegir franja horària</button>
      </div>

      {franges.map((franja, idx) => (
        <div key={idx} style={{ marginBottom: 12, padding: 12, border:
'1px solid #ddd', borderRadius: 6, background: '#fafafa', display:
'flex', gap: 8, alignItems: 'center', flexWrap: 'wrap' }}>
          <input
            type="time"
            value={franja.hora_inici}
            onChange={e => updateFranja(idx, 'hora_inici',

```

```

e.target.value))
    style={{ padding: 6 }}
  />
  <input
    type="number"
    placeholder="Durada (min)"
    value={franja.durada_min}
    onChange={e => updateFanja(idx, 'durada_min',
parseInt(e.target.value) || 0)}
    style={{ width: 100, padding: 6 }}
    min="1"
  />
  <select value={franja.materia_id} onChange={e =>
updateFanja(idx, 'materia_id', e.target.value)} style={{ padding: 6,
minWidth: 150 }}>
    <option value="">-- Matèria --</option>
    {materies.map(m => <option key={m.id}
value={m.id}>{m.nom}</option>)}
  </select>
  <button onClick={() => eliminarFanja(idx)}
style={{ background: '#dc3545', color: 'white' }}>Eliminar</button>
</div>
)))

{franges.length > 0 && (
  <div style={{ marginTop: 20 }}>
    <button onClick={guardarFranges} disabled={saving}
style={{ background: '#2d5be3', color: 'white', padding: '8px
20px' }}>
      {saving ? 'Guardant...' : 'Desar totes les franges'}
    </button>
  </div>
)}
</div>
);
}

```

src/components/HorariGrup.jsx

```

import { useState, useEffect } from 'react';
import { apiFetch } from '../api';

const dies = {
  1: 'Dilluns',
  2: 'Dimarts',
  3: 'Dimecres',
  4: 'Dijous',
  5: 'Divendres',
  6: 'Dissabte',
  7: 'Diumenge'
}

```

```

};

export default function HorariGrup() {
  const [grups, setGrups] = useState([]);
  const [grupId, setGrupId] = useState('');
  const [horari, setHorari] = useState([]);
  const [loading, setLoading] = useState(false);
  const [missatge, setMissatge] = useState(null);

  useEffect(() => {
    carregarGrups();
  }, []);

  async function carregarGrups() {
    const res = await apiFetch('/grups');
    const data = await res.json();
    if (res.ok) {
      setGrups(Array.isArray(data) ? data : []);
    } else {
      setMissatge({ tipus: 'error', text: data.error || 'Error carregant grups' });
    }
  }

  async function carregarHorari() {
    if (!grupId) return;
    setLoading(true);
    setMissatge(null);
    try {
      // Carregar tots els horaris del grup seleccionat
      const res = await apiFetch(`/horaris?grup_id=${grupId}`);
      const data = await res.json();
      if (!res.ok) throw new Error(data.error);

      // Filtrar per assegurar-nos que només tenim horaris del grup seleccionat
      const horarisFiltrats = Array.isArray(data) ? data.filter(h => h.grup_id === grupId) : [];
      setHorari(horarisFiltrats);

      if (horarisFiltrats.length === 0) {
        setMissatge({ tipus: 'info', text: 'No hi ha horari definit per a aquest grup.' });
      }
    } catch (err) {
      setMissatge({ tipus: 'error', text: err.message });
    } finally {
      setLoading(false);
    }
  }

  // Obtener el nom del grup seleccionat

```



```

const nomGrup = grups.find(g => g.id === grupId)?.nom || '';

return (
  <div>
    <h2>Horari del grup (visor)</h2>

    <div style={{ marginBottom: 20, padding: 16, border: '1px solid
#e0ddd5', borderRadius: 10, background: '#fff', display: 'flex', gap:
12, alignItems: 'flex-end', flexWrap: 'wrap' }}>
      <div style={{ flex: 1, minWidth: 200 }}>
        <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Grup</label>
        <select
          value={grupId}
          onChange={e => setGrupId(e.target.value)}
          style={{ width: '100%', padding: '8px 12px', border: '1px
solid #e0ddd5', borderRadius: 6, background: '#f5f4f0', color:
'#1a1a18', fontSize: 13 }}
        >
          <option value="">-- Selecciona un grup --</option>
          {grups.map(g => <option key={g.id}
value={g.id}>{g.nom}</option>)}
        </select>
      </div>
      <button
        onClick={carregarHorari}
        disabled={!grupId}
        style={{ background: '#2d5be3', color: '#fff', border:
'none', borderRadius: 6, padding: '8px 20px', fontSize: 13,
fontWeight: 500, cursor: 'pointer', height: 38 }}
      >
        Carregar horari
      </button>
    </div>

    {loading && <p style={{ textAlign: 'center', padding: 40, color:
'#6b6a64' }}>Carregant horari...</p>}

    {missatge && missatge.tipus === 'info' && (
      <div style={{ padding: 40, textAlign: 'center', background:
'#fff', border: '1px solid #e0ddd5', borderRadius: 10, color:
'#a8a79f', fontSize: 13 }}>
        {missatge.text}
      </div>
    )}

    {missatge && missatge.tipus === 'error' && (
      <div style={{ marginBottom: 12, padding: 10, borderRadius: 6,
background: '#fceaea', color: '#b83232', border: '1px solid #e8a0a0',
fontSize: 13 }}>
        {missatge.text}
      </div>
    )}
  )

```

```

    })

    {!loading && horari.length > 0 && (
      <div style={{ background: '#fff', border: '1px solid #e0ddd5',
borderRadius: 10, overflow: 'hidden' }}>
        <div style={{ padding: '12px 16px', borderBottom: '1px solid
#e0ddd5', background: '#f5f4f0' }}>
          <span style={{ fontSize: 13, fontWeight: 600, color:
'#1a1a18' }}>
            Horari del grup: {nomGrup}
          </span>
        </div>

        {Object.entries(dies).map(([num, nomDia]) => {
          const franges = horari.filter(f => f.dia_setmana ===
parseInt(num)).sort((a, b) =>
a.hora_inici.localeCompare(b.hora_inici));
          if (franges.length === 0) return null;

          return (
            <div key={num} style={{ marginBottom: 0 }}>
              <div style={{ padding: '10px 16px', background:
'#fafaf8', borderBottom: '1px solid #e0ddd5', borderTop: '1px solid
#e0ddd5' }}>
                <h3 style={{ margin: 0, fontSize: 14, fontWeight:
600, color: '#1a1a18' }}>{nomDia}</h3>
              </div>
              <table style={{ width: '100%', borderCollapse:
'collapse', fontSize: 13 }}>
                <thead>
                  <tr style={{ background: '#fff' }}>
                    <th style={{ textAlign: 'left', padding: '10px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11, textTransform:
'uppercase', letterSpacing: '.5px', borderBottom: '1px solid
#e0ddd5' }}>Hora inici</th>
                    <th style={{ textAlign: 'left', padding: '10px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11, textTransform:
'uppercase', letterSpacing: '.5px', borderBottom: '1px solid
#e0ddd5' }}>Durada</th>
                    <th style={{ textAlign: 'left', padding: '10px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11, textTransform:
'uppercase', letterSpacing: '.5px', borderBottom: '1px solid
#e0ddd5' }}>Matèria</th>
                  </tr>
                </thead>
                <tbody>
                  {franges.map((f, idx) => (
                    <tr key={f.id} style={{ borderBottom: idx ===
franges.length - 1 ? 'none' : '1px solid #f0eee8' }}>
                      <td style={{ padding: '10px 12px', color:
'#1a1a18' }}>{f.hora_inici.slice(0, 5)}</td>
                      <td style={{ padding: '10px 12px', color:

```

```

'#1a1a18' }}>{f.durada_min} min</td>
      <td style={{ padding: '10px 12px' }}>
        <span style={{
          padding: '4px 10px',
          borderRadius: 20,
          fontSize: 11,
          fontWeight: 500,
          background: '#ebf0fd',
          color: '#1a3a9e',
          display: 'inline-block'
        }}>
          {f.materies?.nom || '-'}
        </span>
      </td>
    </tr>
  </tbody>
</table>
</div>
);
}}
</div>
)}
</div>
);
}

```

src/components/HorariManagement.jsx

```

import { useState, useEffect } from 'react';

const API = 'http://localhost:3000';

const dies = {
  1: 'Dilluns', 2: 'Dimarts', 3: 'Dimecres', 4: 'Dijous', 5:
'Divendres', 6: 'Dissabte', 7: 'Diumenge'
};

export default function HorariManagement() {
  const [horaris, setHoraris] = useState([]);
  const [grups, setGrups] = useState([]);
  const [materies, setMateries] = useState([]);
  const [loading, setLoading] = useState(true);
  const [form, setForm] = useState({ grup_id: '', materia_id: '',
dia_setmana: 1, hora_inici: '09:00', durada_min: 60 });
  const [editingId, setEditingId] = useState(null);
  const [missatge, setMissatge] = useState(null);

  useEffect(() => {
    carregarHoraris();
  });
}

```

```

    carregarGrups();
    carregarMateries();
  }, []);

  async function carregarHoraris() {
    setLoading(true);
    const res = await fetch(`${API}/horaris`);
    const data = await res.json();
    setHoraris(Array.isArray(data) ? data : []);
    setLoading(false);
  }

  async function carregarGrups() {
    const res = await fetch(`${API}/grups`);
    const data = await res.json();
    setGrups(Array.isArray(data) ? data : []);
  }

  async function carregarMateries() {
    const res = await fetch(`${API}/materies`);
    const data = await res.json();
    setMateries(Array.isArray(data) ? data : []);
  }

  async function handleSubmit(e) {
    e.preventDefault();
    setMissatge(null);

    const url = editingId ? `${API}/horaris/${editingId}` :
`${API}/horaris`;
    const method = editingId ? 'PUT' : 'POST';

    const res = await fetch(url, {
      method,
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(form),
    });

    const data = await res.json();
    if (!res.ok) {
      setMissatge({ tipus: 'error', text: data.error });
    } else {
      setMissatge({ tipus: 'ok', text: editingId ? 'Horari
actualitzat' : 'Horari creat' });
      setForm({ grup_id: '', materia_id: '', dia_setmana: 1,
hora_inici: '09:00', durada_min: 60 });
      setEditingId(null);
      carregarHoraris();
    }
  }

  function editar(horari) {

```

```

    setForm({
      grup_id: horari.grup_id,
      materia_id: horari.materia_id,
      dia_setmana: horari.dia_setmana,
      hora_inici: horari.hora_inici.slice(0, 5),
      durada_min: horari.durada_min,
    });
    setEditingId(horari.id);
  }

  async function eliminar(id, materiaNom) {
    if (!confirm(`Eliminar l'horari de ${materiaNom}?`)) return;
    const res = await fetch(`${API}/horaris/${id}`, { method:
'DELETE' });
    if (res.ok) carregarHoraris();
  }

  function cancelarEdicio() {
    setForm({ grup_id: '', materia_id: '', dia_setmana: 1, hora_inici:
'09:00', durada_min: 60 });
    setEditingId(null);
  }

  return (
    <div>
      <h2>Gestió d'horaris setmanals</h2>

      <form onSubmit={handleSubmit} style={{ marginBottom: 20,
padding: 16, border: '1px solid #ccc', borderRadius: 8 }}>
        <h3>{editingId ? 'Editar horari' : 'Nou horari'}</h3>
        <div style={{ display: 'grid', gap: 8, gridTemplateColumns:
'1fr 1fr 1fr' }}>
          <select value={form.grup_id} onChange={e =>
setForm({ ...form, grup_id: e.target.value })} required>
            <option value="">Selecciona un grup</option>
            {groups.map(g => <option key={g.id}
value={g.id}>{g.nom}</option>)}
          </select>

          <select value={form.materia_id} onChange={e =>
setForm({ ...form, materia_id: e.target.value })} required>
            <option value="">Selecciona una matèria</option>
            {materies.map(m => <option key={m.id}
value={m.id}>{m.nom}</option>)}
          </select>

          <select value={form.dia_setmana} onChange={e =>
setForm({ ...form, dia_setmana: parseInt(e.target.value) })} required>
            {Object.entries(dies).map(([num, nom]) => (
              <option key={num} value={num}>{nom}</option>
            ))}
          </select>

```

```

        <input type="time" value={form.hora_inici} onChange={e =>
setForm({ ...form, hora_inici: e.target.value })} required />

        <input type="number" placeholder="Durada (minuts)"
value={form.durada_min} onChange={e => setForm({ ...form, durada_min:
parseInt(e.target.value) })} required min="1" step="1" />
    </div>
    <div style={{ marginTop: 12 }}>
        <button type="submit">{editingId ? 'Actualitzar' : 'Crear
horari'}</button>
        {editingId && <button type="button"
onClick={cancelarEdicio}>Cancel·lar</button>}
    </div>
    {missatge && <div style={{ marginTop: 8, color: missatge.tipus
=== 'ok' ? 'green' : 'red' }}>{missatge.text}</div>}
</form>

    {loading && <p>Carregant horaris...</p>}
    {!loading && horaris.length === 0 && <p>No hi ha cap horari
definit.</p>}
    {horaris.length > 0 && (
        <table border="1" cellPadding="8" style={{ width: '100%',
borderCollapse: 'collapse' }}>
            <thead>
                <tr><th>Grup</th><th>Matèria</th><th>Dia</th><th>Hora
inici</th><th>Durada</th><th>Accions</th></tr>
            </thead>
            <tbody>
                {horaris.map(h => (
                    <tr key={h.id}>
                        <td>{h.grups?.nom || '-'}</td>
                        <td>{h.materies?.nom || '-'}</td>
                        <td>{dies[h.dia_setmana]}</td>
                        <td>{h.hora_inici.slice(0,5)}</td>
                        <td>{h.durada_min} min</td>
                        <td>
                            <button onClick={() => editar(h)}></button>
                            <button onClick={() => eliminar(h.id,
h.materies?.nom)}></button>
                        </td>
                    </tr>
                ))}
            </tbody>
        </table>
    )}
</div>
);
}

```

src/components/DiesNoLectiusManagement.jsx

```
import { useState, useEffect } from 'react';
import { apiFetch } from '../api';

export default function DiesNoLectiusManagement() {
  const [grups, setGrups] = useState([]);
  const [grupId, setGrupId] = useState('');
  const [data, setData] = useState('');
  const [motiu, setMotiu] = useState('');
  const [dies, setDies] = useState([]);
  const [loading, setLoading] = useState(false);
  const [saving, setSaving] = useState(false);
  const [editingId, setEditingId] = useState(null);
  const [missatge, setMissatge] = useState(null);
  const [copiant, setCopiant] = useState(false);

  useEffect(() => {
    carregarGrups();
  }, []);

  useEffect(() => {
    if (grupId) {
      carregarDies();
    } else {
      setDies([]);
    }
  }, [grupId]);

  async function carregarGrups() {
    const res = await apiFetch('/grups');
    const data = await res.json();
    if (res.ok) {
      setGrups(Array.isArray(data) ? data : []);
    } else {
      setMissatge({ tipus: 'error', text: data.error || 'Error carregant grups' });
    }
  }

  async function carregarDies() {
    if (!grupId) return;
    setLoading(true);
    try {
      const res = await
apiFetch(`/dies_no_lectius?grup_id=${grupId}`);
      const data = await res.json();
      if (!res.ok) throw new Error(data.error);
      setDies(Array.isArray(data) ? data : []);
    } catch (err) {
      setMissatge({ tipus: 'error', text: err.message });
    } finally {

```

```

        setLoading(false);
    }
}

async function handleSubmit(e) {
    e.preventDefault();
    if (!grupId || !data) {
        setMissatge({ tipus: 'error', text: 'Selecciona un grup i una
data' });
        return;
    }

    setSaving(true);
    setMissatge(null);

    const url = editingId ? `/dies_no_lectius/${editingId}` :
'/dies_no_lectius';
    const method = editingId ? 'PUT' : 'POST';

    try {
        const res = await apiFetch(url, {
            method,
            body: JSON.stringify({ grup_id: grupId, data, motiu }),
        });
        const result = await res.json();
        if (!res.ok) throw new Error(result.error);

        setMissatge({ tipus: 'ok', text: editingId ? 'Dia actualitzat' :
'Dia afegit' });
        setData('');
        setMotiu('');
        setEditingId(null);
        carregarDies();
    } catch (err) {
        setMissatge({ tipus: 'error', text: err.message });
    } finally {
        setSaving(false);
    }
}

function editar(dia) {
    setData(dia.data);
    setMotiu(dia.motiu || '');
    setEditingId(dia.id);
}

async function eliminar(id, dataDia) {
    if (!confirm(`Eliminar el dia no lectiu ${dataDia}?`)) return;
    try {
        const res = await apiFetch(`/dies_no_lectius/${id}`, { method:
'DELETE' });
        if (!res.ok) throw new Error('Error en eliminar');
    }
}

```



```

        setMissatge({ tipus: 'ok', text: 'Dia eliminat' });
        carregaraDies();
    } catch (err) {
        setMissatge({ tipus: 'error', text: err.message });
    }
}

function cancelarEdicio() {
    setData('');
    setMotiu('');
    setEditingId(null);
}

async function copiarDies(origenId) {
    if (!origenId) return;
    if (!confirm(`Vols copiar els dies no lectius del grup seleccionat
al grup actual?`)) return;

    setCopiant(true);
    try {
        const res = await apiFetch('/dies_no_lectius/copiar', {
            method: 'POST',
            body: JSON.stringify({ origen_grup_id: origenId,
desti_grup_id: grupId })
        });
        const data = await res.json();
        if (!res.ok) throw new Error(data.error);
        setMissatge({ tipus: 'ok', text: data.missatge });
        carregaraDies();
    } catch (err) {
        setMissatge({ tipus: 'error', text: err.message });
    } finally {
        setCopiant(false);
    }
}

const nomGrup = grups.find(g => g.id === grupId)?.nom || '';
const grupsSenseActual = grups.filter(g => g.id !== grupId);

return (
    <div>
        <h2>Gestió de dies no lectius</h2>
        <p style={{ fontSize: 13, color: '#6b6a64', marginBottom: 16 }}>
            Defineix els dies en què no hi ha classe (festius, excursions,
etc.) per a cada grup.
            Aquests dies no es comptabilitzaran en els informes
d'assistència.
        </p>

        <div style={{
            marginBottom: 20,
            padding: 16,

```

```

border: '1px solid #e0ddd5',
borderRadius: 10,
background: '#fff',
display: 'flex',
gap: 16,
alignItems: 'flex-end',
flexWrap: 'wrap'
}}>
<div style={{ minWidth: 200 }}>
  <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Grup</label>
  <select
    value={grupId}
    onChange={e => setGrupId(e.target.value)}
    style={{ width: '100%', padding: '10px 12px', border: '1px
solid #c0bdb5', borderRadius: 6, background: '#fff', color: '#1a1a18',
fontSize: 14, cursor: 'pointer' }}
  >
    <option value="">-- Selecciona un grup --</option>
    {grups.map(g => <option key={g.id}
value={g.id}>{g.nom}</option>)}
  </select>
</div>
</div>

{grupId && (
  <>
    <form onSubmit={handleSubmit} style={{
      marginBottom: 20,
      padding: 16,
      border: '1px solid #e0ddd5',
      borderRadius: 10,
      background: '#fff'
    }}>
      <h3 style={{ marginBottom: 16, fontSize: 16, color:
'#1a1a18' }}>
        {editingId ? 'Editar dia no lectiu' : `Afegir dia no
lectiu - ${nomGrup}`}
      </h3>
      <div style={{ display: 'grid', gap: 12,
gridTemplateColumns: 'repeat(2, 1fr)' }}>
        <div>
          <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Data *</label>
          <input
            type="date"
            value={data}
            onChange={e => setData(e.target.value)}
            required
            style={{ width: '100%', padding: '10px 12px',
border: '2px solid #2d5be3', borderRadius: 6, background: '#fff',
color: '#1a1a18', fontSize: 14 }}

```

```

        />
      </div>
      <div>
        <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Motiu
(opcional)</label>
        <input
          type="text"
          placeholder="Ex: Festa local, Excursió, Dia de
lliure disposició"
          value={motiu}
          onChange={e => setMotiu(e.target.value)}
          style={{ width: '100%', padding: '10px 12px',
border: '1px solid #c0bdb5', borderRadius: 6, background: '#fff',
color: '#1a1a18', fontSize: 14 }}
        />
      </div>
    </div>
    <div style={{ marginTop: 16, display: 'flex', gap: 8 }}>
      <button
        type="submit"
        disabled={saving || !data}
        style={{ background: '#2d5be3', color: '#fff', border:
'none', borderRadius: 6, padding: '10px 24px', fontSize: 13,
fontWeight: 500, cursor: 'pointer' }}
      >
        {saving ? 'Guardant...' : editingId ? 'Actualitzar
dia' : '+ Afegir dia'}
      </button>
      {editingId && (
        <button
          type="button"
          onClick={cancelarEdicio}
          style={{ background: '#f0eee8', border: 'none',
borderRadius: 6, padding: '10px 24px', fontSize: 13, cursor:
'pointer' }}
        >
          Cancel·lar
        </button>
      )}
    </div>
    {missatge && (
      <div style={{
        marginTop: 12,
        padding: 10,
        borderRadius: 6,
        background: missatge.tipus === 'ok' ? '#e8f5ee' :
'#fceaea',
        color: missatge.tipus === 'ok' ? '#1a7a4a' :
'#b83232',
        border: `1px solid ${missatge.tipus === 'ok' ?
'#9fe1cb' : '#e8a0a0'}`

```

```

    }}>
    {missatge.text}
  </div>
)}
</form>

{/* Secció per copiar dies */}
{grupsSenseActual.length > 0 && (
  <div style={{
    marginBottom: 20,
    padding: 16,
    border: '1px solid #e0ddd5',
    borderRadius: 10,
    background: '#fff'
  }}>
    <h3 style={{ marginBottom: 16, fontSize: 16, color:
'#1a1a18' }}>Copiar dies no lectius</h3>
    <div style={{ display: 'flex', gap: 16, alignItems:
'flex-end', flexWrap: 'wrap' }}>
      <div style={{ minWidth: 200 }}>
        <label style={{ display: 'block', fontSize: 12,
color: '#6b6a64', marginBottom: 5, fontWeight: 500 }}>Copiar des del
grup</label>
        <select
          id="grupOrigen"
          style={{ width: '100%', padding: '10px 12px',
border: '1px solid #c0bdb5', borderRadius: 6, background: '#fff',
fontSize: 14 }}
          >
            <option value="">-- Selecciona un grup --</option>
            {grupsSenseActual.map(g => <option key={g.id}
value={g.id}>{g.nom}</option>)}
          </select>
        </div>
        <button
          onClick={() => {
            const origenId =
document.getElementById('grupOrigen').value;
            if (origenId) copiarDies(origenId);
          }}
          disabled={copiant}
          style={{ background: '#17a2b8', color: '#fff',
border: 'none', borderRadius: 6, padding: '10px 24px', fontSize: 13,
cursor: 'pointer', height: 42 }}
          >
          {copiant ? 'Copiant...' : '📋 Copiar dies'}
        </button>
      </div>
      <p style={{ fontSize: 12, color: '#6b6a64', marginTop:
12 }}>
        Els dies copiats se sumaran als existents. Després

```

pots editar o eliminar dies individuals.

```
    </p>
  </div>
)}

  {/* Llista de dies existents */}
  <div style={{ background: '#fff', border: '1px solid
#e0ddd5', borderRadius: 10, overflow: 'hidden' }}>
    <div style={{ padding: '12px 16px', background: '#f5f4f0',
borderBottom: '1px solid #e0ddd5' }}>
      <strong>Dies no lectius - {nomGrup}</strong>
      <span style={{ marginLeft: 16, fontSize: 12, color:
'#6b6a64' }}>
        {dies.length} {dies.length === 1 ? 'dia' : 'dies'}
      </span>
    </div>
    {!loading && dies.length === 0 && (
      <p style={{ padding: 40, textAlign: 'center', color:
'#a8a79f' }}>
        No hi ha dies no lectius definits per a aquest grup.
      </p>
    )}
    {dies.length > 0 && (
      <table style={{ width: '100%', borderCollapse:
'collapse', fontSize: 13 }}>
        <thead>
          <tr style={{ background: '#fff', borderBottom: '1px
solid #e0ddd5' }}>
            <th style={{ textAlign: 'left', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Data</th>
            <th style={{ textAlign: 'left', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Motiu</th>
            <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Accions</th>
          </tr>
        </thead>
        <tbody>
          {dies.map(d => (
            <tr key={d.id} style={{ borderBottom: '1px solid
#f0eee8' }}>
              <td style={{ padding: '10px 12px', color:
'#1a1a18' }}>
                {new Date(d.data).toLocaleDateString('ca-ES')}
              </td>
              <td style={{ padding: '10px 12px', color:
'#1a1a18' }}>{d.motiu || '-'}</td>
              <td style={{ padding: '10px 12px', textAlign:
'center' }}>
                <button onClick={() => editar(d)}
style={{ background: 'none', border: 'none', color: '#2d5be3', cursor:
'pointer', fontSize: 16, marginRight: 12 }} title="Editar">✎</button>
              </td>
            </tr>
          )}
        </tbody>
      </table>
    )}
  </div>
}
```



```

carregant grups' });
    }
}

function obtenirDiaSetmana(dataStr) {
    const date = new Date(dataStr);
    let dia = date.getDay();
    return dia === 0 ? 7 : dia;
}

async function carregarSessions() {
    if (!grupId || !data) return;
    setLoading(true);
    setMissatge(null);
    setSessioSeleccionada(null);
    setAlumnes([]);
    setValorsPerSessio({});

    try {
        const diaNum = obtenirDiaSetmana(data);
        setDiaSetmana(diaNum);

        const res = await apiFetch(`/horaris?grup_id=${grupId}`);
        const totesSessions = await res.json();
        if (!res.ok) throw new Error(totesSessions.error);

        const sessionsDia = totesSessions.filter(s => s.dia_setmana ===
diaNum);
        setSessions(sessionsDia);

        if (sessionsDia.length === 0) {
            setMissatge({ tipus: 'info', text: 'No hi ha cap sessió
programada per a aquest grup en aquest dia.' });
        } else if (sessionsDia.length === 1) {
            setSessioSeleccionada(sessionsDia[0]);
        }
    } catch (err) {
        setMissatge({ tipus: 'error', text: err.message });
    } finally {
        setLoading(false);
    }
}

async function carregarAssistenciaPerSessio() {
    if (!sessioSeleccionada) return;
    setLoading(true);
    setMissatge(null);

    try {
        const resAlumnes = await apiFetch('/alumnes');
        const totsAlumnes = await resAlumnes.json();
        if (!resAlumnes.ok) throw new Error(totsAlumnes.error || 'Error

```

```

carregant alumnes');
    const alumnesGrup = (Array.isArray(totsAlumnes) ? totsAlumnes :
[]).filter(a => a.grup_id === grupId && a.actiu === true);
    setAlumnes(alumnesGrup);

    const resRegistres = await
apiFetch(`/assistencia/config?grup_id=${grupId}&data=${data}`);
    const config = await resRegistres.json();
    if (!resRegistres.ok) throw new Error(config.error);

    const registresSessio = config.registresExistents.filter(r =>
r.horari_id === sessioSeleccionada.id);

    const nousValors = {};
    alumnesGrup.forEach(alumne => {
        const registreExistent = registresSessio.find(r => r.alumne_id
=== alumne.id);
        nousValors[alumne.id] = {
            minuts: registreExistent ?
registreExistent.minuts_assistits : sessioSeleccionada.durada_min,
            justificats: registreExistent ?
registreExistent.minuts_justificats || 0 : 0,
            observacions: registreExistent ?
registreExistent.observacions || '' : ''
        };
    });
    setValorsPerSessio(nousValors);

    } catch (err) {
        setMissatge({ tipus: 'error', text: err.message });
    } finally {
        setLoading(false);
    }
}

useEffect(() => {
    if (sessioSeleccionada) {
        carregarAssistenciaPerSessio();
    }
}, [sessioSeleccionada]);

function handleCanviMinuts(alumneId, minuts) {
    const valorsActuals = valorsPerSessio[alumneId];
    const justificats = valorsActuals?.justificats || 0;

    if (minuts + justificats > sessioSeleccionada.durada_min) {
        setMissatge({ tipus: 'error', text: `Els minuts assistits +
justificats no poden superar els ${sessioSeleccionada.durada_min}
minuts totals.` });
        return;
    }
}

```



```

    setValorsPerSessio(prev => ({
      ...prev,
      [alumneId]: { ...prev[alumneId], minuts }
    }));
  }

  function handleCanviJustificats(alumneId, justificats) {
    const valorsActuals = valorsPerSessio[alumneId];
    const minuts = valorsActuals?.minuts || 0;

    if (minuts + justificats > sessioSeleccionada.durada_min) {
      setMissatge({ tipus: 'error', text: `Els minuts assistits + justificats no poden superar els ${sessioSeleccionada.durada_min} minuts totals.` });
      return;
    }

    setValorsPerSessio(prev => ({
      ...prev,
      [alumneId]: { ...prev[alumneId], justificats }
    }));
  }

  function handleCanviObservacions(alumneId, observacions) {
    setValorsPerSessio(prev => ({
      ...prev,
      [alumneId]: { ...prev[alumneId], observacions }
    }));
  }

  async function guardarAssistencia() {
    if (!sessioSeleccionada) return;

    let teError = false;
    Object.keys(valorsPerSessio).forEach(alumneId => {
      const valors = valorsPerSessio[alumneId];
      if (valors.minuts + valors.justificats > sessioSeleccionada.durada_min) {
        teError = true;
      }
    });

    if (teError) {
      setMissatge({ tipus: 'error', text: 'Corregeix els errors abans de guardar.' });
      return;
    }

    const registresAGuardar =
    Object.keys(valorsPerSessio).map(alumneId => ({
      alumne_id: alumneId,
      horari_id: sessioSeleccionada.id,

```

```

        minuts_assistits: valorsPerSessio[alumneId].minuts,
        minuts_justificats: valorsPerSessio[alumneId].justificats,
        observacions: valorsPerSessio[alumneId].observacions || null
    }));

    setSaving(true);
    try {
        const res = await apiFetch('/assistencia/guardar', {
            method: 'POST',
            body: JSON.stringify({ data, registres: registresAGuardar }),
        });
        const result = await res.json();
        if (!res.ok) throw new Error(result.error);
        setMissatge({ tipus: 'ok', text: 'Assistència guardada correctament' });
        carregarAssistenciaPerSessio();
    } catch (err) {
        setMissatge({ tipus: 'error', text: err.message });
    } finally {
        setSaving(false);
    }
}

const nomGrup = grups.find(g => g.id === grupId)?.nom || '';
const dataFormatejada = data ? new
Date(data).toLocaleDateString('ca-ES') : '';
const nomDia = diaSetmana ? dies[diaSetmana] : '';

return (
    <div>
        <h2>Registre d'assistència</h2>

        {/* Formulari de selecció - Millorat amb més separació */}
        <div style={{
            marginBottom: 20,
            padding: 16,
            border: '1px solid #e0ddd5',
            borderRadius: 10,
            background: '#fff',
            display: 'flex',
            gap: 20,
            alignItems: 'flex-end',
            flexWrap: 'wrap'
        }}>
            <div style={{ flex: 1, minWidth: 200 }}>
                <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Grup</label>
                <select
                    value={grupId}
                    onChange={e => setGrupId(e.target.value)}
                    style={{ width: '100%', padding: '10px 12px', border: '1px
solid #c0bdb5', borderRadius: 6, background: '#fff', color: '#1a1a18',

```

```

fontSize: 14, cursor: 'pointer' }}
    >
      <option value="">-- Selecciona --</option>
      {grups.map(g => <option key={g.id}
value={g.id}>{g.nom}</option>)}
    </select>
  </div>
  <div style={{ minWidth: 240 }}>
    <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Data</label>
    <input
      type="date"
      value={data}
      onChange={e => setData(e.target.value)}
      style={{
        width: '100%',
        padding: '10px 12px',
        border: '2px solid #2d5be3',
        borderRadius: 6,
        background: '#fff',
        color: '#1a1a18',
        fontSize: 14,
        cursor: 'pointer',
        fontFamily: 'system-ui, sans-serif',
        boxShadow: '0 1px 3px rgba(0,0,0,0.1)'
      }}
    />
  </div>
  <div style={{ minWidth: 160 }}>
    <button
      onClick={carregarSessions}
      disabled={!grupId || !data}
      style={{ background: '#2d5be3', color: '#fff', border:
'none', borderRadius: 6, padding: '10px 24px', fontSize: 13,
fontWeight: 500, cursor: 'pointer', height: 42, whiteSpace: 'nowrap',
width: '100%' }}
    >
      Carregar sessions
    </button>
  </div>
</div>

{diaSetmana && data && (
  <div style={{ marginBottom: 16, padding: '10px 16px',
background: '#f5f4f0', borderRadius: 8, border: '1px solid
#e0ddd5' }}>
    <span style={{ fontSize: 14, color: '#1a1a18' }}>
      <strong>{nomDia}</strong>, {dataFormatejada} - Grup:
<strong>{nomGrup}</strong>
    </span>
  </div>
)}

```

```

    {sessions.length > 0 && (
      <div style={{ marginBottom: 20, padding: 16, border: '1px
solid #e0ddd5', borderRadius: 10, background: '#fff' }}>
        <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Selecciona una
sessió</label>
        <select
          value={sessioSeleccionada?.id || ''}
          onChange={e => {
            const sessio = sessions.find(s => s.id ===
e.target.value);
            setSessioSeleccionada(sessio);
          }}
          style={{ width: '100%', padding: '10px 12px', border: '1px
solid #c0bdb5', borderRadius: 6, background: '#fff', color: '#1a1a18',
fontSize: 14, cursor: 'pointer' }}
        >
          <option value="">-- Tria una sessió --</option>
          {sessions.map(s => (
            <option key={s.id} value={s.id}>
              {s.materies?.nom} - {s.hora_inici.slice(0,5)}
              ({s.durada_min} min)
            </option>
          ))}
        </select>
      </div>
    )}

    {missatge && missatge.tipus === 'info' && (
      <div style={{ padding: 40, textAlign: 'center', background:
'#fff', border: '1px solid #e0ddd5', borderRadius: 10, color:
'#a8a79f' }}>
        {missatge.text}
      </div>
    )}

    {missatge && missatge.tipus === 'error' && (
      <div style={{ marginBottom: 12, padding: 10, borderRadius: 6,
background: '#fceaea', color: '#b83232', border: '1px solid
#e8a0a0' }}>
        {missatge.text}
      </div>
    )}

    {missatge && missatge.tipus === 'ok' && (
      <div style={{ marginBottom: 12, padding: 10, borderRadius: 6,
background: '#e8f5ee', color: '#1a7a4a', border: '1px solid
#9fe1cb' }}>
        {missatge.text}
      </div>
    )}

```

```

    {sessioSeleccionada && !loading && alumnes.length > 0 && (
      <div style={{ background: '#fff', border: '1px solid #e0ddd5',
borderRadius: 10, overflow: 'hidden' }}>
        <div style={{ padding: '12px 16px', background: '#f5f4f0',
borderBottom: '1px solid #e0ddd5' }}>
          <strong>{sessioSeleccionada.materies?.nom}</strong> -
{sessioSeleccionada.hora_inici.slice(0,5)} (durada:
{sessioSeleccionada.durada_min} min)
        </div>
        <table style={{ width: '100%', borderCollapse: 'collapse',
fontSize: 13 }}>
          <thead>
            <tr style={{ background: '#fff', borderBottom: '1px
solid #e0ddd5' }}>
              <th style={{ textAlign: 'left', padding: '12px 12px',
color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Alumne</th>
              <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Minuts
assistits</th>
              <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Minuts
justificats</th>
              <th style={{ textAlign: 'left', padding: '12px 12px',
color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Observacions</th>
            </tr>
          </thead>
          <tbody>
            {alumnes.map(alumne => {
              const valors = valorsPerSessio[alumne.id] || { minuts:
sessioSeleccionada.durada_min, justificats: 0, observacions: '' };
              const quart = Math.floor(sessioSeleccionada.durada_min
/ 4);
              const meitat =
Math.floor(sessioSeleccionada.durada_min / 2);
              const total = sessioSeleccionada.durada_min;
              return (
                <tr key={alumne.id} style={{ borderBottom: '1px
solid #f0eee8' }}>
                  <td style={{ padding: '10px 12px', color:
'#1a1a18' }}>{alumne.nom} {alumne.cognoms}</td>
                  <td style={{ textAlign: 'center', padding: '10px
12px' }}>
                    <div style={{ display: 'flex', gap: '4px',
alignItems: 'center', justifyContent: 'center', flexWrap: 'wrap' }}>
                      <button onClick={() =>
handleCanviMinuts(alumne.id, 0)} style={{ background: '#dc3545',
color: 'white', border: 'none', borderRadius: 4, padding: '4px 8px',
cursor: 'pointer', fontSize: 11 }} title="0 minuts">0</button>
                      <button onClick={() =>
handleCanviMinuts(alumne.id, quart)} style={{ background: '#ffc107',
color: '#333', border: 'none', borderRadius: 4, padding: '4px 8px',

```

```

cursor: 'pointer', fontSize: 11 }} title="25%">¼</button>
      <button onClick={() =>
handleCanviMinuts(alumne.id, meitat)} style={{ background: '#17a2b8',
color: 'white', border: 'none', borderRadius: 4, padding: '4px 8px',
cursor: 'pointer', fontSize: 11 }} title="50%">½</button>
      <button onClick={() =>
handleCanviMinuts(alumne.id, total)} style={{ background: '#28a745',
color: 'white', border: 'none', borderRadius: 4, padding: '4px 8px',
cursor: 'pointer', fontSize: 11 }} title="100%">✓</button>
      <input type="number" min="0"
max={sessioSeleccionada.durada_min - valors.justificats}
value={valors.minuts} onChange={e => handleCanviMinuts(alumne.id,
parseInt(e.target.value) || 0)} style={{ width: 60, padding: 4,
fontSize: 12, borderRadius: 4, border: '1px solid #e0ddd5' }} />
    </div>
  </td>
  <td style={{ textAlign: 'center', padding: '10px
12px' }}>
    <div style={{ display: 'flex', gap: '4px',
alignItems: 'center', justifyContent: 'center', flexWrap: 'wrap' }}>
      <button onClick={() =>
handleCanviJustificats(alumne.id, 0)} style={{ background: '#dc3545',
color: 'white', border: 'none', borderRadius: 4, padding: '4px 8px',
cursor: 'pointer', fontSize: 11 }} title="0 minuts">0</button>
      <button onClick={() =>
handleCanviJustificats(alumne.id, quart)} style={{ background:
'#ffc107', color: '#333', border: 'none', borderRadius: 4, padding:
'4px 8px', cursor: 'pointer', fontSize: 11 }} title="25%">¼</button>
      <button onClick={() =>
handleCanviJustificats(alumne.id, meitat)} style={{ background:
'#17a2b8', color: 'white', border: 'none', borderRadius: 4, padding:
'4px 8px', cursor: 'pointer', fontSize: 11 }} title="50%">½</button>
      <button onClick={() =>
handleCanviJustificats(alumne.id, total)} style={{ background:
'#28a745', color: 'white', border: 'none', borderRadius: 4, padding:
'4px 8px', cursor: 'pointer', fontSize: 11 }} title="100%">✓</button>
      <input type="number" min="0"
max={sessioSeleccionada.durada_min - valors.minuts}
value={valors.justificats} onChange={e =>
handleCanviJustificats(alumne.id, parseInt(e.target.value) || 0)}
style={{ width: 60, padding: 4, fontSize: 12, borderRadius: 4, border:
'1px solid #e0ddd5' }} />
    </div>
  </td>
  <td style={{ padding: '10px 12px' }}>
    <input type="text" placeholder="Opcional"
value={valors.observacions} onChange={e =>
handleCanviObservacions(alumne.id, e.target.value)} style={{ width:
'100%', padding: 6, fontSize: 12, border: '1px solid #e0ddd5',
borderRadius: 4 }} />
  </td>

```

```

        </tr>
      );
    })}
  </tbody>
</table>
<div style={{ padding: '16px', textAlign: 'center',
borderTop: '1px solid #e0ddd5' }}>
  <button onClick={guardarAssistencia} disabled={saving}
style={{ background: '#28a745', color: 'white', border: 'none',
borderRadius: 6, padding: '8px 24px', fontSize: 13, fontWeight: 500,
cursor: 'pointer' }}>
    {saving ? 'Guardant...' : 'Guardar assistència'}
  </button>
</div>
</div>
  )}
</div>
);
}

```

7.3.6. Components d'informes

src/components/informes/InformesPrincipal.jsx

```

import { useState } from 'react';
import AssistenciaAlumne from './InformeAssistencia';
import AssistenciaMateria from './AssistenciaMateria';
import AssistenciaGrup from './AssistenciaGrup';
import AlertesAssistencia from './AlertesAssistencia';

export default function InformesPrincipal() {
  const [tab, setTab] = useState('alumne');

  const tabs = [
    { id: 'alumne', nom: '📄 Assistència per alumne', component:
<AssistenciaAlumne /> },
    { id: 'materia', nom: '📖 Assistència per matèria', component:
<AssistenciaMateria /> },
    { id: 'grup', nom: '👥 Assistència per grup', component:
<AssistenciaGrup /> },
    { id: 'alertes', nom: '⚠️ Alertes', component: <AlertesAssistencia
/> },
  ];

  const activeTab = tabs.find(t => t.id === tab);

  return (
    <div>
      <h2>📊 Informes</h2>
    </div>
  );
}

```

```

    {/* Pestanyes */}
    <div style={{
      display: 'flex',
      flexWrap: 'wrap',
      gap: 4,
      borderBottom: '1px solid #e0ddd5',
      marginBottom: 20,
      background: '#fff',
      borderRadius: '10px 10px 0 0',
      padding: '0 4px'
    }}>
      {tabs.map(t => (
        <button
          key={t.id}
          onClick={() => setTab(t.id)}
          style={{
            padding: '10px 20px',
            fontSize: 13,
            fontWeight: tab === t.id ? 600 : 400,
            background: tab === t.id ? '#2d5be3' : 'transparent',
            color: tab === t.id ? '#fff' : '#6b6a64',
            border: 'none',
            borderRadius: '8px 8px 0 0',
            cursor: 'pointer',
            transition: 'all 0.2s'
          }}
        >
          {t.nom}
        </button>
      ))}
    </div>

    {/* Contingut de la pestanya activa */}
    <div style={{
      background: '#fff',
      border: '1px solid #e0ddd5',
      borderRadius: 10,
      padding: 20,
      borderTopLeftRadius: 0
    }}>
      {activeTab.component}
    </div>
  </div>
);
}

```

src/components/informes/InformeAssistencia.jsx


```

import { useState, useEffect } from 'react';
import { apiFetch } from '../api';

export default function InformeAssistencia() {
  const [grups, setGrups] = useState([]);
  const [grupId, setGrupId] = useState('');
  const [dataInici, setDataInici] = useState('');
  const [dataFi, setDataFi] = useState('');
  const [loading, setLoading] = useState(false);
  const [informe, setInforme] = useState(null);
  const [missatge, setMissatge] = useState(null);

  useEffect(() => {
    carregarGrups();
    const avui = new Date();
    const fa30Dies = new Date();
    fa30Dies.setDate(avui.getDate() - 30);
    setDataFi(avui.toISOString().split('T')[0]);
    setDataInici(fa30Dies.toISOString().split('T')[0]);
  }, []);

  async function carregarGrups() {
    try {
      const res = await apiFetch('/grups');
      const data = await res.json();
      if (res.ok) {
        setGrups(Array.isArray(data) ? data : []);
      } else {
        setMissatge({ tipus: 'error', text: data.error || 'Error carregant grups' });
      }
    } catch (err) {
      setMissatge({ tipus: 'error', text: err.message });
    }
  }

  async function generarInforme() {
    if (!grupId || !dataInici || !dataFi) {
      setMissatge({ tipus: 'error', text: 'Selecciona un grup i un període' });
      return;
    }

    setLoading(true);
    setMissatge(null);
    setInforme(null);

    try {
      const url =
        `/informes/assistencia?grup_id=${grupId}&data_inici=${dataInici}&data_
fi=${dataFi}`;

```

```

    const res = await apiFetch(url);
    const data = await res.json();
    if (!res.ok) throw new Error(data.error);
    setInforme(data);
  } catch (err) {
    setMissatge({ tipus: 'error', text: err.message });
  } finally {
    setLoading(false);
  }
}

const nomGrup = grups.find(g => g.id === grupId)?.nom || '';

return (
  <div>
    <h2>Informe d'assistència</h2>

    <div style={{
      marginBottom: 20,
      padding: 16,
      border: '1px solid #e0ddd5',
      borderRadius: 10,
      background: '#fff',
      display: 'flex',
      gap: 16,
      alignItems: 'flex-end',
      flexWrap: 'wrap'
    }}>
      <div style={{ flex: 1, minWidth: 180 }}>
        <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Grup</label>
        <select
          value={grupId}
          onChange={e => setGrupId(e.target.value)}
          style={{ width: '100%', padding: '10px 12px', border: '1px
solid #c0bdb5', borderRadius: 6, background: '#fff', color: '#1a1a18',
fontSize: 14, cursor: 'pointer' }}
          >
            <option value="">-- Selecciona --</option>
            {grups.map(g => <option key={g.id}
value={g.id}>{g.nom}</option>)}
          </select>
        </div>
        <div style={{ minWidth: 200 }}>
          <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Data inicial</label>
          <input
            type="date"
            value={dataInici}
            onChange={e => setDataInici(e.target.value)}
            style={{ width: '100%', padding: '10px 12px', border: '2px
solid #2d5be3', borderRadius: 6, background: '#fff', color: '#1a1a18',

```

```

fontSize: 14 }}
  />
</div>
<div style={{ minWidth: 200 }}>
  <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Data final</label>
  <input
    type="date"
    value={dataFi}
    onChange={e => setDataFi(e.target.value)}
    style={{ width: '100%', padding: '10px 12px', border: '2px
solid #2d5be3', borderRadius: 6, background: '#fff', color: '#1a1a18',
fontSize: 14 }}
  />
</div>
<div>
  <button
    onClick={generarInforme}
    disabled={!grupId || !dataInici || !dataFi || loading}
    style={{ background: '#2d5be3', color: '#fff', border:
'none', borderRadius: 6, padding: '10px 24px', fontSize: 13,
fontWeight: 500, cursor: 'pointer', height: 42, whiteSpace:
'nowrap' }}
  >
    {loading ? 'Generant...' : 'Generar informe'}
  </button>
</div>
</div>

{missatge && (
  <div style={{ marginBottom: 12, padding: 10, borderRadius: 6,
background: '#fceaea', color: '#b83232', border: '1px solid
#e8a0a0' }}>
    {missatge.text}
  </div>
)}

{informe && (
  <div style={{ background: '#fff', border: '1px solid #e0ddd5',
borderRadius: 10, overflow: 'hidden' }}>
    <div style={{ padding: '12px 16px', background: '#f5f4f0',
borderBottom: '1px solid #e0ddd5' }}>
      <strong>Informe del grup {nomGrup}</strong>
      <span style={{ marginLeft: 16, fontSize: 12, color:
'#6b6a64' }}>
        {new Date(informe.data_inici).toLocaleDateString('ca-
ES')} - {new Date(informe.data_fi).toLocaleDateString('ca-ES')}
      </span>
      <span style={{ marginLeft: 16, fontSize: 12, color:
'#6b6a64' }}>
        Total teòric: {informe.hores_teoric_total}
      </span>
    </div>
  </div>
)}

```

```

    </div>
    <table style={{ width: '100%', borderCollapse: 'collapse',
fontSize: 13 }}>
      <thead>
        <tr style={{ background: '#fff', borderBottom: '1px
solid #e0ddd5' }}>
          <th style={{ textAlign: 'left', padding: '12px 12px',
color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Alumne</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Hores
teòriques</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Hores
assistides</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Hores
justificades</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>%
Assistit</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>%
Assistit+Justif.</th>
        </tr>
      </thead>
      <tbody>
        {informe.alumnes.map(alumne => {
          const percentClass = alumne.percent_assistit < 80 ?
'#fceaee' : '#e8f5ee';
          const percentColor = alumne.percent_assistit < 80 ?
'#b83232' : '#1a7a4a';
          return (
            <tr key={alumne.id} style={{ borderBottom: '1px
solid #f0eee8' }}>
              <td style={{ padding: '10px 12px', color:
'#1a1a18' }}>
                {alumne.nom} {alumne.cognoms}
                {!alumne.actiu && <span style={{ marginLeft: 8,
fontSize: 11, color: '#a8a79f' }}>(inactiu)</span>}
              </td>
              <td style={{ textAlign: 'center', padding: '10px
12px', color: '#1a1a18' }}>{alumne.hores_teoric}</td>
              <td style={{ textAlign: 'center', padding: '10px
12px', color: '#1a1a18' }}>{alumne.hores_assistit}</td>
              <td style={{ textAlign: 'center', padding: '10px
12px', color: '#1a1a18' }}>{alumne.hores_justificat}</td>
              <td style={{ textAlign: 'center', padding: '10px
12px' }}>
                <span style={{
                  padding: '4px 10px',
                  borderRadius: 20,
                  fontSize: 11,

```

```

        fontWeight: 500,
        background: percentClass,
        color: percentColor
    }}>
        {alumne.percent_assistit}%
    </span>
</td>
<td style={{ textAlign: 'center', padding: '10px
12px' }}>
        <span style={{
            padding: '4px 10px',
            borderRadius: 20,
            fontSize: 11,
            fontWeight: 500,
            background: '#ebf0fd',
            color: '#1a3a9e'
        }}>
            {alumne.percent_assistit_justificat}%
        </span>
    </td>
</tr>
    );
    })}
</tbody>
</table>
</div>
    )}
</div>
    );
}

```

src/components/informes/AssistenciaMateria.jsx

```

import { useState, useEffect } from 'react';
import { apiFetch } from '../../api';

export default function AssistenciaMateria() {
    const [grups, setGrups] = useState([]);
    const [materies, setMateries] = useState([]);
    const [grupId, setGrupId] = useState('');
    const [materiaId, setMateriaId] = useState('');
    const [dataInici, setDataInici] = useState('');
    const [dataFi, setDataFi] = useState('');
    const [loading, setLoading] = useState(false);
    const [informe, setInforme] = useState(null);
    const [missatge, setMissatge] = useState(null);

    useEffect(() => {
        carregarGrups();
        carregarMateries();
    }, []);
}

```

```

    const avui = new Date();
    const fa30Dies = new Date();
    fa30Dies.setDate(avui.getDate() - 30);
    setDataFi(avui.toISOString().split('T')[0]);
    setDataInici(fa30Dies.toISOString().split('T')[0]);
  }, []);

  async function carregarGrups() {
    const res = await apiFetch('/grups');
    const data = await res.json();
    if (res.ok) {
      setGrups(Array.isArray(data) ? data : []);
    } else {
      setMissatge({ tipus: 'error', text: data.error || 'Error carregant grups' });
    }
  }

  async function carregarMateries() {
    const res = await apiFetch('/materies');
    const data = await res.json();
    if (res.ok) {
      setMateries(Array.isArray(data) ? data : []);
    } else {
      setMissatge({ tipus: 'error', text: data.error || 'Error carregant matèries' });
    }
  }

  async function generarInforme() {
    if (!grupId || !materiaId || !dataInici || !dataFi) {
      setMissatge({ tipus: 'error', text: 'Selecciona grup, matèria i període' });
      return;
    }

    setLoading(true);
    setMissatge(null);
    setInforme(null);

    try {
      const url =
        `/informes/assistencia_materia?grup_id=${grupId}&materia_id=${materiaId}&data_inici=${dataInici}&data_fi=${dataFi}`;
      const res = await apiFetch(url);
      const data = await res.json();
      if (!res.ok) throw new Error(data.error);
      setInforme(data);
    } catch (err) {
      setMissatge({ tipus: 'error', text: err.message });
    } finally {
      setLoading(false);
    }
  }

```

```

    }
  }

  const nomGrup = grups.find(g => g.id === grupId)?.nom || '';
  const nomMateria = materies.find(m => m.id === materiaId)?.nom ||
  '';

  return (
    <div>
      <h3>Assistència per matèria</h3>
      <p style={{ fontSize: 13, color: '#6b6a64', marginBottom: 16 }}>
        Percentatges d'assistència d'un grup en una matèria específica
        durant un període.
      </p>

      <div style={{
        marginBottom: 20,
        padding: 16,
        border: '1px solid #e0ddd5',
        borderRadius: 10,
        background: '#fff',
        display: 'flex',
        gap: 16,
        alignItems: 'flex-end',
        flexWrap: 'wrap'
      }}>
        <div style={{ flex: 1, minWidth: 180 }}>
          <label style={{ display: 'block', fontSize: 12, color:
            '#6b6a64', marginBottom: 5, fontWeight: 500 }}>Grup</label>
          <select
            value={grupId}
            onChange={e => setGrupId(e.target.value)}
            style={{ width: '100%', padding: '10px 12px', border: '1px
            solid #c0bdb5', borderRadius: 6, background: '#fff', color: '#1a1a18',
            fontSize: 14, cursor: 'pointer' }}
          >
            <option value="">-- Selecciona --</option>
            {grups.map(g => <option key={g.id}
            value={g.id}>{g.nom}</option>)}
          </select>
        </div>
        <div style={{ flex: 1, minWidth: 180 }}>
          <label style={{ display: 'block', fontSize: 12, color:
            '#6b6a64', marginBottom: 5, fontWeight: 500 }}>Matèria</label>
          <select
            value={materiaId}
            onChange={e => setMateriaId(e.target.value)}
            style={{ width: '100%', padding: '10px 12px', border: '1px
            solid #c0bdb5', borderRadius: 6, background: '#fff', color: '#1a1a18',
            fontSize: 14, cursor: 'pointer' }}
          >
            <option value="">-- Selecciona --</option>

```

```

        {materies.map(m => <option key={m.id}
value={m.id}>{m.nom}</option>)}
      </select>
    </div>
    <div style={{ minWidth: 200 }}>
      <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Data inicial</label>
      <input
        type="date"
        value={dataInici}
        onChange={e => setDataInici(e.target.value)}
        style={{ width: '100%', padding: '10px 12px', border: '2px
solid #2d5be3', borderRadius: 6, background: '#fff', color: '#1a1a18',
fontSize: 14 }}
      />
    </div>
    <div style={{ minWidth: 200 }}>
      <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Data final</label>
      <input
        type="date"
        value={dataFi}
        onChange={e => setDataFi(e.target.value)}
        style={{ width: '100%', padding: '10px 12px', border: '2px
solid #2d5be3', borderRadius: 6, background: '#fff', color: '#1a1a18',
fontSize: 14 }}
      />
    </div>
    <div>
      <button
        onClick={generarInforme}
        disabled={!grupId || !materiaId || !dataInici || !dataFi
|| loading}
        style={{ background: '#2d5be3', color: '#fff', border:
'none', borderRadius: 6, padding: '10px 24px', fontSize: 13,
fontWeight: 500, cursor: 'pointer', height: 42, whiteSpace:
'nowrap' }}
      >
        {loading ? 'Generant...' : 'Generar informe'}
      </button>
    </div>
  </div>

  {missatge && (
    <div style={{ marginBottom: 12, padding: 10, borderRadius: 6,
background: '#fceaea', color: '#b83232', border: '1px solid
#e8a0a0' }}>
      {missatge.text}
    </div>
  )}

  {informe && (

```



```

<div style={{ background: '#fff', border: '1px solid #e0ddd5',
borderRadius: 10, overflow: 'hidden' }}>
  <div style={{ padding: '12px 16px', background: '#f5f4f0',
borderBottom: '1px solid #e0ddd5' }}>
    <strong>Informe de {nomMateria} - {nomGrup}</strong>
    <span style={{ marginLeft: 16, fontSize: 12, color:
'#6b6a64' }}>
      {new Date(informe.data_inici).toLocaleDateString('ca-
ES')} - {new Date(informe.data_fi).toLocaleDateString('ca-ES')}
    </span>
    <span style={{ marginLeft: 16, fontSize: 12, color:
'#6b6a64' }}>
      Total teòric: {informe.hores_teoric_total}
    </span>
  </div>
  <table style={{ width: '100%', borderCollapse: 'collapse',
fontSize: 13 }}>
    <thead>
      <tr style={{ background: '#fff', borderBottom: '1px
solid #e0ddd5' }}>
        <th style={{ textAlign: 'left', padding: '12px 12px',
color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Alumne</th>
        <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Hores
teòriques</th>
        <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Hores
assistides</th>
        <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Hores
justificades</th>
        <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>%
Assistit</th>
        <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>%
Assistit+Justif.</th>
      </tr>
    </thead>
    <tbody>
      {informe.alumnes.map(alumne => {
        const percentClass = alumne.percent_assistit < 80 ?
'#fcea6a' : '#e8f5ee';
        const percentColor = alumne.percent_assistit < 80 ?
'#b83232' : '#1a7a4a';
        return (
          <tr key={alumne.id} style={{ borderBottom: '1px
solid #f0eee8' }}>
            <td style={{ padding: '10px 12px', color:
'#1a1a18' }}>
              {alumne.nom} {alumne.cognoms}
              {!alumne.actiu && <span style={{ marginLeft: 8,

```

```

fontSize: 11, color: '#a8a79f' }}>(inactiu)</span>}
      </td>
      <td style={{ textAlign: 'center', padding: '10px
12px', color: '#1a1a18' }}>{alumne.hores_teoric}</td>
      <td style={{ textAlign: 'center', padding: '10px
12px', color: '#1a1a18' }}>{alumne.hores_assistit}</td>
      <td style={{ textAlign: 'center', padding: '10px
12px', color: '#1a1a18' }}>{alumne.hores_justificat}</td>
      <td style={{ textAlign: 'center', padding: '10px
12px' }}>
        <span style={{
          padding: '4px 10px',
          borderRadius: 20,
          fontSize: 11,
          fontWeight: 500,
          background: percentClass,
          color: percentColor
        }}>
          {alumne.percent_assistit}%
        </span>
      </td>
      <td style={{ textAlign: 'center', padding: '10px
12px' }}>
        <span style={{
          padding: '4px 10px',
          borderRadius: 20,
          fontSize: 11,
          fontWeight: 500,
          background: '#ebf0fd',
          color: '#1a3a9e'
        }}>
          {alumne.percent_assistit_justificat}%
        </span>
      </td>
    </tr>
  );
  })}
</tbody>
</table>
</div>
  )}
</div>
);
}

```

src/components/informes/AssistenciaGrup.jsx

```

import { useState, useEffect } from 'react';
import { apiFetch } from '../../api';

```

```

export default function AssistenciaGrup() {
  const [grups, setGrups] = useState([]);
  const [grupId, setGrupId] = useState('');
  const [dataInici, setDataInici] = useState('');
  const [dataFi, setDataFi] = useState('');
  const [loading, setLoading] = useState(false);
  const [informe, setInforme] = useState(null);
  const [missatge, setMissatge] = useState(null);

  useEffect(() => {
    carregarGrups();
    const avui = new Date();
    const fa30Dies = new Date();
    fa30Dies.setDate(avui.getDate() - 30);
    setDataFi(avui.toISOString().split('T')[0]);
    setDataInici(fa30Dies.toISOString().split('T')[0]);
  }, []);

  async function carregarGrups() {
    const res = await apiFetch('/grups');
    const data = await res.json();
    if (res.ok) {
      setGrups(Array.isArray(data) ? data : []);
    } else {
      setMissatge({ tipus: 'error', text: data.error || 'Error
carregant grups' });
    }
  }

  async function generarInforme() {
    if (!dataInici || !dataFi) {
      setMissatge({ tipus: 'error', text: 'Selecciona un període' });
      return;
    }

    setLoading(true);
    setMissatge(null);
    setInforme(null);

    try {
      let url =
`/informes/assistencia_grup?data_inici=${dataInici}&data_fi=${dataFi}`
      ;
      if (grupId) {
        url += `&grup_id=${grupId}`;
      }
      const res = await apiFetch(url);
      const data = await res.json();
      if (!res.ok) throw new Error(data.error);
      setInforme(data);
    } catch (err) {
      setMissatge({ tipus: 'error', text: err.message });
    }
  }
}

```

```

    } finally {
      setLoading(false);
    }
  }

  return (
    <div>
      <h3>Assistència per grup</h3>
      <p style={{ fontSize: 13, color: '#6b6a64', marginBottom: 16 }}>
        Resum d'assistència agregat per grup en el període
        seleccionat.
      </p>

      <div style={{
        marginBottom: 20,
        padding: 16,
        border: '1px solid #e0ddd5',
        borderRadius: 10,
        background: '#fff',
        display: 'flex',
        gap: 16,
        alignItems: 'flex-end',
        flexWrap: 'wrap'
      }}>
        <div style={{ minWidth: 180 }}>
          <label style={{ display: 'block', fontSize: 12, color:
            '#6b6a64', marginBottom: 5, fontWeight: 500 }}>Grup (opcional)</label>
          <select
            value={grupId}
            onChange={e => setGrupId(e.target.value)}
            style={{ width: '100%', padding: '10px 12px', border: '1px
              solid #c0bdb5', borderRadius: 6, background: '#fff', color: '#1a1a18',
              fontSize: 14 }}
            >
            <option value="">-- Tots els grups --</option>
            {groups.map(g => <option key={g.id}
              value={g.id}>{g.nom}</option>)}
          </select>
        </div>
        <div style={{ minWidth: 200 }}>
          <label style={{ display: 'block', fontSize: 12, color:
            '#6b6a64', marginBottom: 5, fontWeight: 500 }}>Data inicial</label>
          <input
            type="date"
            value={dataInici}
            onChange={e => setDataInici(e.target.value)}
            style={{ width: '100%', padding: '10px 12px', border: '2px
              solid #2d5be3', borderRadius: 6, background: '#fff', color: '#1a1a18',
              fontSize: 14 }}
            />
        </div>
        <div style={{ minWidth: 200 }}>

```

```

        <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Data final</label>
        <input
            type="date"
            value={dataFi}
            onChange={e => setDataFi(e.target.value)}
            style={{ width: '100%', padding: '10px 12px', border: '2px
solid #2d5be3', borderRadius: 6, background: '#fff', color: '#1a1a18',
fontSize: 14 }}
        />
    </div>
    <div>
        <button
            onClick={generarInforme}
            disabled={!dataInici || !dataFi || loading}
            style={{ background: '#2d5be3', color: '#fff', border:
'none', borderRadius: 6, padding: '10px 24px', fontSize: 13,
fontWeight: 500, cursor: 'pointer', height: 42 }}
        >
            {loading ? 'Generant...' : 'Generar informe'}
        </button>
    </div>
</div>

    {missatge && (
        <div style={{ marginBottom: 12, padding: 10, borderRadius: 6,
background: '#fceaea', color: '#b83232', border: '1px solid
#e8a0a0' }}>
            {missatge.text}
        </div>
    )}

    {informe && (
        <div style={{ background: '#fff', border: '1px solid #e0ddd5',
borderRadius: 10, overflow: 'hidden' }}>
            <div style={{ padding: '12px 16px', background: '#f5f4f0',
borderBottom: '1px solid #e0ddd5' }}>
                <strong>Resum per grup</strong>
                <span style={{ marginLeft: 16, fontSize: 12, color:
'#6b6a64' }}>
                    {new Date(informe.data_inici).toLocaleDateString('ca-
ES')} - {new Date(informe.data_fi).toLocaleDateString('ca-ES')}
                </span>
            </div>
            {informe.grups.length === 0 && (
                <p style={{ padding: 40, textAlign: 'center', color:
'#a8a79f' }}>
                    No hi ha dades per als paràmetres seleccionats.
                </p>
            )}
            {informe.grups.length > 0 && (
                <table style={{ width: '100%', borderCollapse: 'collapse',

```

```

fontSize: 13 }}>
    <thead>
        <tr style={{ background: '#fff', borderBottom: '1px
solid #e0ddd5' }}>
            <th style={{ textAlign: 'left', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Grup</th>
            <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Hores
teòriques</th>
            <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Hores
assistides</th>
            <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Hores
justificades</th>
            <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>%
Assistit</th>
            <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>%
Assistit+Justif.</th>
        </tr>
    </thead>
    <tbody>
        {informe.grups.map(g => {
            const percentClass = g.percent_assistit < 80 ?
'#fcea6a' : '#e8f5e9';
            const percentColor = g.percent_assistit < 80 ?
'#b83232' : '#1a7a4a';
            return (
                <tr key={g.grup_id} style={{ borderBottom: '1px
solid #f0eee8' }}>
                    <td style={{ padding: '10px 12px', color:
'#1a1a18' }}><strong>{g.grup_nom}</strong></td>
                    <td style={{ textAlign: 'center', padding: '10px
12px' }}>{g.hores_teoric}</td>
                    <td style={{ textAlign: 'center', padding: '10px
12px' }}>{g.hores_assistit}</td>
                    <td style={{ textAlign: 'center', padding: '10px
12px' }}>{g.hores_justificat}</td>
                    <td style={{ textAlign: 'center', padding: '10px
12px' }}>
                        <span style={{
                            padding: '4px 10px',
                            borderRadius: 20,
                            fontSize: 11,
                            fontWeight: 500,
                            background: percentClass,
                            color: percentColor
                        }}>
                            {g.percent_assistit}%
                        </span>
                    </td>
                </tr>
            )
        })}
    </tbody>

```

```

        </td>
        <td style={{ textAlign: 'center', padding: '10px
12px' }}>
            <span style={{
                padding: '4px 10px',
                borderRadius: 20,
                fontSize: 11,
                fontWeight: 500,
                background: '#ebf0fd',
                color: '#1a3a9e'
            }}>
                {g.percent_assistit_justificat}%
            </span>
        </td>
    </tr>
    );
    }}}
</tbody>
</table>
    )}
</div>
    )}
</div>
    );
}

```

src/components/informes/AlertesAssistencia.jsx

```

import { useState, useEffect } from 'react';
import { apiFetch } from '../../api';

export default function AlertesAssistencia() {
    const [grups, setGrups] = useState([]);
    const [grupId, setGrupId] = useState('');
    const [dataInici, setDataInici] = useState('');
    const [dataFi, setDataFi] = useState('');
    const [llindarPersonalitzat, setLlindarPersonalitzat] =
    useState('');
    const [loading, setLoading] = useState(false);
    const [alertes, setAlertes] = useState(null);
    const [missatge, setMissatge] = useState(null);

    useEffect(() => {
        carregarGrups();
        const avui = new Date();
        const fa30Dies = new Date();
        fa30Dies.setDate(avui.getDate() - 30);
        setDataFi(avui.toISOString().split('T')[0]);
        setDataInici(fa30Dies.toISOString().split('T')[0]);
    }, []);
}

```

```

async function carregarGrups() {
  const res = await apiFetch('/grups');
  const data = await res.json();
  if (res.ok) {
    setGrups(Array.isArray(data) ? data : []);
  } else {
    setMissatge({ tipus: 'error', text: data.error || 'Error
carregant grups' });
  }
}

async function generarAlertes() {
  if (!dataInici || !dataFi) {
    setMissatge({ tipus: 'error', text: 'Selecciona un període' });
    return;
  }

  setLoading(true);
  setMissatge(null);
  setAlertes(null);

  try {
    let url =
`/informes/alertes?data_inici=${dataInici}&data_fi=${dataFi}`;
    if (grupId) {
      url += `&grup_id=${grupId}`;
    }
    if (llindarPersonalitzat && parseFloat(llindarPersonalitzat) >
0) {
      url += `&llindar_personalitzat=${llindarPersonalitzat}`;
    }
    const res = await apiFetch(url);
    const data = await res.json();
    if (!res.ok) throw new Error(data.error);
    setAlertes(data);
  } catch (err) {
    setMissatge({ tipus: 'error', text: err.message });
  } finally {
    setLoading(false);
  }
}

return (
  <div>
    <h3>⚠ Alertes d'assistència</h3>
    <p style={{ fontSize: 13, color: '#6b6a64', marginBottom: 16 }}>
      Alumnes que han baixat del llindar mínim d'assistència en el
període seleccionat.
    </p>
  </div>
)

```



```

<div style={{
  marginBottom: 20,
  padding: 16,
  border: '1px solid #e0ddd5',
  borderRadius: 10,
  background: '#fff',
  display: 'flex',
  gap: 16,
  alignItems: 'flex-end',
  flexWrap: 'wrap'
}}>
  <div style={{ minWidth: 180 }}>
    <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Grup (opcional)</label>
    <select
      value={grupId}
      onChange={e => setGrupId(e.target.value)}
      style={{ width: '100%', padding: '10px 12px', border: '1px
solid #c0bdb5', borderRadius: 6, background: '#fff', color: '#1a1a18',
fontSize: 14 }}
    >
      <option value="">-- Tots els grups --</option>
      {grups.map(g => <option key={g.id}
value={g.id}>{g.nom}</option>)}
    </select>
  </div>
  <div style={{ minWidth: 200 }}>
    <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Data inicial</label>
    <input
      type="date"
      value={dataInici}
      onChange={e => setDataInici(e.target.value)}
      style={{ width: '100%', padding: '10px 12px', border: '2px
solid #2d5be3', borderRadius: 6, background: '#fff', color: '#1a1a18',
fontSize: 14 }}
    />
  </div>
  <div style={{ minWidth: 200 }}>
    <label style={{ display: 'block', fontSize: 12, color:
'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Data final</label>
    <input
      type="date"
      value={dataFi}
      onChange={e => setDataFi(e.target.value)}
      style={{ width: '100%', padding: '10px 12px', border: '2px
solid #2d5be3', borderRadius: 6, background: '#fff', color: '#1a1a18',
fontSize: 14 }}
    />
  </div>
  <div style={{ minWidth: 150 }}>
    <label style={{ display: 'block', fontSize: 12, color:

```

```

'#6b6a64', marginBottom: 5, fontWeight: 500 }}>Llindar (%)</label>
  <input
    type="number"
    placeholder="Per defecte: del grup"
    value={llindarPersonalitzat}
    onChange={e => setLlindarPersonalitzat(e.target.value)}
    min="0"
    max="100"
    style={{ width: '100%', padding: '10px 12px', border: '1px
solid #c0bdb5', borderRadius: 6, background: '#fff', color: '#1a1a18',
fontSize: 14 }}
  />
</div>
<div>
  <button
    onClick={generarAlertes}
    disabled={!dataInici || !dataFi || loading}
    style={{ background: '#dc3545', color: '#fff', border:
'none', borderRadius: 6, padding: '10px 24px', fontSize: 13,
fontWeight: 500, cursor: 'pointer', height: 42 }}
  >
    {loading ? 'Generant...' : '⚠ Generar alertes'}
  </button>
</div>
</div>

{missatge && (
  <div style={{ marginBottom: 12, padding: 10, borderRadius: 6,
background: '#fceaea', color: '#b83232', border: '1px solid
#e8a0a0' }}>
    {missatge.text}
  </div>
)}

{alertes && (
  <div style={{ background: '#fff', border: '1px solid #e0ddd5',
borderRadius: 10, overflow: 'hidden' }}>
    <div style={{ padding: '12px 16px', background: '#f5f4f0',
borderBottom: '1px solid #e0ddd5' }}>
      <strong>⚠ {alertes.total_alertes} alertes</strong>
      <span style={{ marginLeft: 16, fontSize: 12, color:
'#6b6a64' }}>
        {new Date(alertes.data_inici).toLocaleDateString('ca-
ES')} - {new Date(alertes.data_fi).toLocaleDateString('ca-ES')}
      </span>
    </div>
    {alertes.total_alertes === 0 && (
      <p style={{ padding: 40, textAlign: 'center', color:
'#1a7a4a', background: '#e8f5ee' }}>
        ✓ No hi ha alertes per als paràmetres seleccionats. Tots
els alumnes estan per sobre del llindar.

```

```

    </p>
  })
  {alertes.total_alertes > 0 && (
    <table style={{ width: '100%', borderCollapse: 'collapse',
fontSize: 13 }}>
      <thead>
        <tr style={{ background: '#fff', borderBottom: '1px
solid #e0ddd5' }}>
          <th style={{ textAlign: 'left', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Alumne</th>
          <th style={{ textAlign: 'left', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Grup</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Hores
teòriques</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Hores
assistides</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>%
Assistit</th>
          <th style={{ textAlign: 'center', padding: '12px
12px', color: '#6b6a64', fontWeight: 600, fontSize: 11 }}>Llindar</th>
        </tr>
      </thead>
      <tbody>
        {alertes.alertes.map(a => {
          let percentClass = '#fcea5a';
          let percentColor = '#b83232';
          if (a.percent_assistit < 50) {
            percentClass = '#dc3545';
            percentColor = '#fff';
          } else if (a.percent_assistit < 70) {
            percentClass = '#fcea5a';
            percentColor = '#b83232';
          }
          return (
            <tr key={` ${a.alumne_id}-${a.grup_id}`}
style={{ borderBottom: '1px solid #f0eee8' }}>
              <td style={{ padding: '10px 12px', color:
'#1a1a18' }}>{a.alumne_nom} {a.alumne_cognoms}</td>
              <td style={{ padding: '10px 12px', color:
'#1a1a18' }}><strong>{a.grup_nom}</strong></td>
              <td style={{ textAlign: 'center', padding: '10px
12px' }}>{a.hores_teoric}</td>
              <td style={{ textAlign: 'center', padding: '10px
12px' }}>{a.hores_assistit}</td>
              <td style={{ textAlign: 'center', padding: '10px
12px' }}>
                <span style={{
                  padding: '4px 10px',
                  borderRadius: 20,

```

```

        fontSize: 11,
        fontWeight: 500,
        background: percentClass,
        color: percentColor
    }}>
    {a.percent_assistit}%
  </span>
</td>
<td style={{ textAlign: 'center', padding: '10px
12px' }}>
    <span style={{
      padding: '4px 10px',
      borderRadius: 20,
      fontSize: 11,
      fontWeight: 500,
      background: '#f0eee8',
      color: '#6b6a64'
    }}>
    {a.llindar}%
  </span>
</td>
</tr>
);
    }}
  </tbody>
</table>
    )}
  </div>
    )}
  </div>
);
}

```

8. Annex: Idees de millora / TODO

Recull de mancances i possibles millores detectades durant la revisió del codi:

1. **Refactoritzar la lògica de càlcul d'hores teòriques i `formatHores`**, repetida en 4 rutes diferents (`/informes/assistencia`, `/informes/assistencia_materia`, `/informes/assistencia_grup`, `/informes/alertes`) → extreure-la a un mòdul compartit (`src/utils/informes.js`).
2. **Unificar el sistema de mòduls** al backend: `index.js` i `altaProfessor.js` usen ESM (`import/export`), mentre que `src/config/supabase.js`, `crearProfessor.js` i `testConnection.js` usen CommonJS (`require/module.exports`), incompatible amb `"type": "module"` del `package.json`.
3. **Eliminar `HorariManagement.jsx`** (component obsolet, sense autenticació, no enllaçat des de cap navegació).

4. **Variable d'entorn per a `API_URL`** al frontend (`src/api.js`), en lloc d'IP hardcoded.
5. **Rotar la clau `service_role`** exposada a `crearProfessor.js` i eliminar-la del codi font (veure secció 5.1).
6. **Restringir CORS** a l'origen real en producció.
7. **Eliminar el `console.log` del rol de la clau de servei** a l'arrencada del backend, o protegir-lo amb `NODE_ENV !== 'production'`.
8. **Contrasenyes temporals aleatòries** en la importació CSV de professors, en lloc de `temp123456` fix per a tots.
9. **Funcionalitats previstes** (segons context del projecte) encara no implementades en aquest snapshot del codi:
 - o Programació de sessions en blocs de 60/90/120 minuts en increments de 15 min (actualment la durada és un camp numèric lliure).
 - o Exportació/Importació Excel (actualment només CSV per a alumnes i professors).
 - o Exportació de PDF d'informes.
10. **Validació de format de `data`** a les rutes d'informes i assistència (actualment es confia en el format `YYYY-MM-DD` provinent del `<input type="date">` del frontend, sense validació explícita al backend).
11. **Paginació** a les llistes (`/professors`, `/alumnes`, `/grups`, etc.) — actualment es retornen tots els registres sense límit, el qual podria ser un problema de rendiment amb molts alumnes/grups.

9. Glossari de termes (català → concepte)

Terme	Significat
Alumne	Estudiant matriculat, pertany a un grup.
Grup	Classe/grup d'alumnes (ex: "1r ESO A"), amb un tutor i un llinar d'assistència.
Matèria	Assignatura/mòdul formatiu.
Horari	Franja setmanal recurrent (dia de la setmana + hora + durada + matèria) per a un grup.
Sessió	Una ocurrència concreta d'una franja horària en una data determinada.
Registre	Anotació d'assistència d'un alumne a una sessió concreta (minuts assistits/justificats).
Dia no lectiu	Data en què no hi ha classe per a un grup (festiu, excursió...), exclosa del càlcul d'hores teòriques.
Llinar d'assistència	Percentatge mínim d'assistència per sota del qual es genera una alerta.
Tutor	Professor responsable d'un grup concret; accés limitat al seu grup.
Admin	Rol amb accés total al sistema.